

INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E
TECNOLOGIA DE SÃO PAULO
CÂMPUS BIRIGUI

CAMILA ALBIERI MATTOS

PLATAFORMA PERSONALIZÁVEL PARA
CRIAÇÃO E COMPARTILHAMENTO DE
COLEÇÕES DIGITAIS

BIRIGUI

2025

LISTA DE FIGURAS

Figura 1 – Gerenciamento de Coleções	75
Figura 2 – Interações Com Itens da Coleção	76
Figura 3 – Personalização da Plataforma	76
Figura 4 – Layout e Templates	77
Figura 5 – Perfil e Social	78
Figura 6 – Autenticação	79
Figura 7 – Notificações	80
Figura 8 – Modelo de Domínio	143
Figura 9 – Esquema da tela inicial	145
Figura 10 – Esquema da tela de coleções do usuário	146
Figura 11 – Esquema da listagem de itens de uma coleção	146
Figura 12 – Esquema da biblioteca de layouts	147
Figura 13 – Esquema do editor de layout com a área de edição vazia	148
Figura 14 – Esquema do editor de layout com uma ficha composta	149
Figura 15 – Esquema da tela de amigos	150
Figura 16 – Esquema do perfil de outro usuário	151
Figura 17 – Esquema do perfil do próprio usuário	152
Figura 18 – Esquema da tela de configurações de aparência	153
Figura 19 – Diagrama de implantação do sistema	192
Figura 20 – Diagrama de atividades da publicação de uma nova versão	202

LISTA DE QUADROS

Quadro 1 – Configuração do equipamento utilizado no desenvolvimento . . .	16
Quadro 2 – Conceitos centrais do modelo de domínio	35
Quadro 3 – Descrição dos Campos dos Requisitos Funcionais	36
Quadro 4 – Grupos de Requisitos Funcionais	36
Quadro 5 – Classificação de Prioridade dos Requisitos	37
Quadro 6 – Classificação de Operação dos Requisitos	37
Quadro 7 – Atores do Sistema	37
Quadro 8 – RF-01 – Cadastrar Usuário	38
Quadro 9 – RF-02 – Autenticar Usuário (Login e Logout)	39
Quadro 10 – RF-03 – Alterar Senha na Área Restrita	40
Quadro 11 – RF-04 – Recuperar Senha via E-mail	41
Quadro 12 – RF-05 – Gerenciar Perfil do Usuário	42
Quadro 13 – RF-06 – Gerenciar Categoria	43
Quadro 14 – RF-07 – Gerenciar Coleção	44
Quadro 15 – RF-08 – Gerenciar Privacidade da Coleção	45
Quadro 16 – RF-09 – Gerenciar Item	46
Quadro 17 – RF-10 – Mover Item entre Coleções	47
Quadro 18 – RF-11 – Exibir Ficha de Item em Modo Leitura	48
Quadro 19 – RF-12 – Visualizar Itens de uma Coleção	49
Quadro 20 – RF-13 – Exibir Tela Principal de Categorias	50
Quadro 21 – RF-14 – Gerenciar Layout Personalizado	51
Quadro 22 – RF-15 – Associar Layout a Coleção	52
Quadro 23 – RF-16 – Compartilhar Layout	53
Quadro 24 – RF-17 – Modificar Layout de Item Individualmente	54
Quadro 25 – RF-18 – Gerenciar Aparência (Tema e Cores)	55
Quadro 26 – RF-19 – Gerenciar Tipografia Global	56
Quadro 27 – RF-20 – Gerenciar Amizade	57
Quadro 28 – RF-21 – Seguir Coleção Visível	58
Quadro 29 – RF-22 – Exibir Feed de Atualizações	59

Quadro 30 – RF-23 – Visualizar Perfis e Coleções de Usuários	60
Quadro 31 – RF-24 – Gerenciar Notificações do Sistema	62
Quadro 32 – RNF-01 – Usabilidade	63
Quadro 33 – RNF-02 – Desempenho de Carregamento	64
Quadro 34 – RNF-03 – Segurança de Senhas e Dados Pessoais	65
Quadro 35 – RNF-04 – Compatibilidade e Responsividade	66
Quadro 36 – RNF-05 – Personalização Visual sem Quebras de Layout	67
Quadro 37 – RNF-06 – Desempenho na Movimentação de Itens	68
Quadro 38 – RNF-07 – Desempenho do Editor de Layout	69
Quadro 39 – RNF-08 – Disponibilidade do Sistema	70
Quadro 40 – RNF-09 – Acessibilidade	71
Quadro 41 – RNF-10 – Segurança e Integridade dos Dados	72
Quadro 42 – Tecnologias e versões utilizadas na implementação	170
Quadro 43 – Estrutura de diretórios do <i>backend</i>	174
Quadro 44 – Estrutura de diretórios do <i>frontend</i>	176
Quadro 45 – Distribuição dos testes automatizados por módulo	181
Quadro 46 – Caso de teste CT01: validar a revogação da sessão no <i>logout</i> . .	182
Quadro 47 – Caso de teste CT02: validar a redefinição de senha por <i>token</i> temporário	183
Quadro 48 – Caso de teste CT03: validar o encerramento de acompanhamen- tos ao restringir a privacidade	184
Quadro 49 – Caso de teste CT04: validar o acesso a coleções conforme priva- cidade e amizade	185
Quadro 50 – Caso de teste CT05: validar a criação da ficha própria de um item	186
Quadro 51 – Caso de teste CT06: validar a paleta em uso	187
Quadro 52 – Caso de teste CT07: validar a resposta a uma solicitação de amizade	188
Quadro 53 – Caso de teste CT08: validar o formato real da imagem enviada	189
Quadro 54 – Alvos de implantação do sistema	195
Quadro 55 – Variáveis de ambiente de produção	196
Quadro 56 – Dockerfile da API	197
Quadro 57 – Declaração do contêiner da API e configuração do Caddy . . .	199

SUMÁRIO

1	INTRODUÇÃO	9
1.1	Justificativa	10
1.2	Objetivos	11
1.2.1	Objetivo Geral	11
1.2.2	Objetivos Específicos	11
1.3	Problema	12
1.4	Organização Deste Trabalho	12
2	MATERIAIS E MÉTODOS	14
2.1	Metodologia da Pesquisa	14
2.1.1	Pesquisa Bibliográfica	14
2.1.2	Pesquisa Aplicada	15
2.2	Materiais	15
2.2.1	Equipamentos	15
2.2.2	Softwares	16
2.3	Ferramentas Utilizadas	16
2.3.1	Visual Studio Code	16
2.3.2	Node.js e NestJS	17
2.3.3	PostgreSQL e Docker	18
2.3.4	Prisma	18
2.3.5	DBeaver	19
2.3.6	React e Vite	19
2.3.7	Figma	20
2.3.8	Astah Community	21
2.4	Métodos de Desenvolvimento	21
2.4.1	Elicitação e especificação de requisitos	21
2.4.2	Modelagem conceitual	22
2.4.3	Definição arquitetural	22
2.4.4	Prototipação	22

2.4.5	Implementação	23
2.4.6	Gestão do projeto	23
2.4.7	Estratégia de Testes	24
3	DESCRIÇÃO GERAL DO PRODUTO	25
3.1	Perspectiva do Produto	25
3.2	Funções do Produto	25
3.3	Características dos Usuários	26
3.4	Restrições Gerais	27
3.5	Diferenciais do Produto	27
4	ELICITAÇÃO DE REQUISITOS E ANÁLISE	28
4.1	Requisitos do Usuário	28
4.1.1	Organização em Pacotes	31
4.2	Requisitos do Sistema	33
4.2.1	Requisitos Funcionais	34
4.2.2	Requisitos Não-Funcionais	61
4.2.3	Restrições, Suposições e Dependências	72
4.2.4	Requisitos Adiados	73
4.3	Casos de Uso	74
4.3.1	Diagrama de Casos de Uso	74
4.3.2	Especificação dos Casos de Uso	80
4.3.2.1	Fazer Login	81
4.3.2.2	Realizar Logout	84
4.3.2.3	Alterar Senha	86
4.3.2.4	Recuperar Senha via E-mail	89
4.3.2.5	Editar Perfil	92
4.3.2.6	Personalizar Interface	95
4.3.2.7	Criar Categoria	98
4.3.2.8	Criar Coleção	102
4.3.2.9	Gerenciar Privacidade da Coleção	105
4.3.2.10	Criar Item	108
4.3.2.11	Exibir Ficha de Item em Modo Leitura	111

4.3.2.12	Visualizar Itens da Coleção	114
4.3.2.13	Visualizar Dashboard de Categorias	117
4.3.2.14	Criar Layout	120
4.3.2.15	Compartilhar Layout	124
4.3.2.16	Adicionar Amigo	128
4.3.2.17	Gerenciar Solicitações de Amizade	132
4.3.2.18	Visualizar Perfil de Usuário	135
4.3.2.19	Visualizar Feed de Atualizações	139
4.4	Modelo de Domínio	142
5	PROJETO DE SOFTWARE	144
5.1	Projeto de Interface	144
5.2	Projeto de Dados	153
5.2.1	Mapeamento Objeto-Relacional	154
5.2.2	Estrutura das Tabelas no Banco de Dados	155
5.2.2.1	Detalhamento da Estrutura das Tabelas	157
6	IMPLEMENTAÇÃO	169
6.1	Ambiente de Desenvolvimento	169
6.2	Servidor de Dados	170
6.3	Servidor de Aplicação	171
6.3.1	Armazenamento de Arquivos	173
6.4	Organização do Código do <i>Backend</i>	173
6.5	Aplicação Cliente	175
6.5.1	Comunicação com a API	176
6.5.2	Editor de Layouts	177
6.5.3	Aparência e Tipografia	178
6.6	Verificação Durante a Implementação	178
6.7	Limitações da Implementação	179
7	TESTE	180
7.1	Ambiente e Execução dos Testes	180
7.2	Visão Geral dos Testes	181
7.3	Casos de Teste	182

7.4	Resultados e Limitações	189
8	IMPLANTAÇÃO	191
8.1	Infraestrutura	191
8.2	Configuração dos Serviços	195
8.2.1	Contêiner da API	196
8.2.2	Superfície Exposta	198
8.2.3	Continuidade de Execução	200
8.3	Procedimento de Implantação	200
8.3.1	Provisionamento Inicial	200
8.3.2	Publicação de Novas Versões	201
8.3.3	Reversão	203
9	RESULTADOS E DISCUSSÃO	205
	REFERÊNCIAS BIBLIOGRÁFICAS	206

1 INTRODUÇÃO

Colecionar é uma prática presente em diferentes fases da vida e nos mais diversos interesses: há quem reúna livros lidos, filmes e séries assistidos, jogos concluídos, miniaturas de carros, moedas ou discos. Mais do que acumular objetos ou títulos, a coleção envolve escolher o que entra nela, organizá-la segundo critérios próprios e, muitas vezes, exibi-la a outras pessoas. Com a digitalização do cotidiano, essas etapas passaram a acontecer também em meio digital, e o ambiente em que a coleção é mantida pode tanto favorecer quanto dificultar as práticas de aquisição, curadoria e exibição que dão sentido a ela (WATKINS; SELLEN; LINDLEY, 2015).

Registrar uma coleção não se limita a listar o que se possui ou o que se consumiu. Para muitos colecionadores, o registro funciona como memória: a data em que um filme foi assistido, a nota dada a uma temporada de série, as impressões sobre um livro ou os detalhes de uma peça adquirida permitem revisitar a coleção mais tarde, comparar opiniões antigas com as atuais e perceber a própria trajetória. A forma como essas informações são apresentadas também importa, pois a coleção é, em boa medida, uma expressão de quem a mantém.

Nesse contexto, este trabalho apresenta uma plataforma *web* personalizável para criação e compartilhamento de coleções digitais. O acervo do usuário é organizado em três níveis: **Categorias**, que agrupam temas amplos (como “Filmes” ou “Carros”); **Coleções**, subdivisões de uma categoria onde os registros são cadastrados e onde se definem a privacidade e o layout; e **Itens**, que representam cada unidade da coleção. A apresentação dos itens é definida por um editor de layouts no estilo de ferramentas como o Canva, no qual o usuário posiciona e redimensiona Elementos de seis tipos (texto, imagem, forma, ícone, classificação e data) e monta modelos reutilizáveis para suas coleções. Completam a proposta a personalização global da interface, por meio de paletas de cores e tipografia, e uma dimensão social, na qual o usuário pode adicionar amigos, seguir coleções de outras pessoas e acompanhar suas atualizações em um *feed*.

1.1 Justificativa

A ideia deste trabalho surgiu de uma prática pessoal de registro de leituras. Ao concluir cada livro, anotavam-se detalhes como a data de término e as impressões sobre a história, com o objetivo de consultá-los depois, ao reler a obra ou simplesmente ao lembrar o que já havia sido lido. Voltar a esses registros permitia comparar a opinião de uma leitura anterior com a percepção atual sobre o enredo e os personagens.

Ao longo dos anos, diferentes meios foram experimentados para manter esse registro, sem que nenhum atendesse plenamente à necessidade. Modelos de acompanhamento de leituras encontrados na internet precisavam ser copiados à mão para o papel, o que dava trabalho e raramente alcançava o visual desejado. As planilhas eletrônicas resolviam o armazenamento, mas ofereciam poucas possibilidades de personalização, reduzindo cada registro a uma linha de tabela. Tampouco se encontrou um aplicativo dedicado que unisse o registro organizado à liberdade de definir sua aparência.

A mesma necessidade apareceu, em seguida, em outros tipos de coleção: filmes, séries, jogos e *hobbies* em geral. Cada um deles pede informações diferentes (um filme pode ter diretor e data em que foi assistido; um carro de coleção, fabricante, escala e ano), e as ferramentas de campos fixos não se adaptam bem a essa variedade. Faltava, portanto, um espaço em que o colecionador pudesse decidir não apenas *o que* registrar, mas *como* cada registro seria apresentado, expressando sua personalidade por meio de cores, fontes e do layout das fichas.

Essa motivação encontra respaldo na literatura sobre personalização de interfaces. Marathe e Sundar (2011) observaram que, ao personalizar aspectos como fontes e cores, os usuários passam a ver a interface como reflexo de sua identidade, e que esse sentimento reforça a percepção de controle sobre a interação. Uma plataforma que une a organização de coleções à personalização visual das fichas atende, assim, tanto à necessidade prática de registrar e consultar o acervo quanto ao desejo de que esse acervo tenha a cara de quem o mantém.

1.2 Objetivos

Esta seção apresenta o objetivo geral do trabalho e os objetivos específicos que o compõem.

1.2.1 Objetivo Geral

Desenvolver uma plataforma *web* que permita ao usuário criar e organizar coleções pessoais de qualquer natureza em categorias, coleções e itens, personalizar a apresentação de cada item por meio de layouts reutilizáveis e compartilhar suas coleções com outros usuários.

1.2.2 Objetivos Específicos

- Levantar e especificar os requisitos funcionais e não funcionais da plataforma, bem como seus casos de uso;
- Elaborar o modelo de domínio do sistema e, a partir dele, o projeto de interface e o projeto de dados;
- Implementar a organização do acervo em categorias, coleções e itens, com controle de privacidade no nível da coleção;
- Construir um editor visual de layouts baseado em Elementos de seis tipos, que permita associar layouts a coleções e personalizar a ficha de itens individualmente;
- Oferecer a personalização global da interface por meio de paletas de cores e tipografia;
- Implementar as funcionalidades sociais de amizade, acompanhamento de coleções, *feed* de atualizações e notificações;
- Verificar o funcionamento do sistema por meio de testes automatizados e testes manuais.

1.3 Problema

As ferramentas usualmente empregadas para registrar coleções pessoais, como planilhas eletrônicas, aplicativos de listas e cadernos, tratam cada registro como um conjunto de campos fixos com aparência padronizada. Essa rigidez dificulta a adaptação a acervos de naturezas distintas, que exigem informações diferentes, e não permite ao colecionador decidir como cada registro é apresentado. Em consequência, quem deseja manter coleções variadas precisa recorrer a várias ferramentas ou aceitar registros que não refletem a forma como gostaria de ver seu acervo.

O problema que este trabalho se propõe a resolver é, portanto, a ausência de uma plataforma que reúna, em um só lugar, a organização de coleções de qualquer tipo, a liberdade de definir a estrutura e a aparência de cada registro e a possibilidade de compartilhar essas coleções com outras pessoas.

1.4 Organização Deste Trabalho

Este texto está organizado em doze capítulos. O Capítulo ?? reúne a fundamentação teórica que sustenta o trabalho, abordando engenharia de requisitos, modelagem conceitual, arquitetura de aplicações *web* e personalização de interfaces. O Capítulo 2 descreve os materiais e os métodos empregados, da pesquisa bibliográfica à estratégia de testes, e apresenta o equipamento e as ferramentas utilizados no desenvolvimento. O Capítulo 3 traz a visão panorâmica do produto, com sua perspectiva, suas funções, os perfis de usuário atendidos e as restrições que delimitam o escopo. O Capítulo 4 especifica os requisitos do usuário e do sistema, os casos de uso e o modelo de domínio, e é nele que se definem os quatro conceitos centrais da plataforma. O Capítulo 5 trata do projeto de software, com o projeto de interface e o projeto de dados, detalhando o mapeamento objeto-relacional e a estrutura das tabelas do banco. O Capítulo 6 descreve a implementação, incluindo o ambiente de desenvolvimento, a organização do código e as decisões tomadas durante a construção do sistema. O Capítulo 7 apresenta os testes realizados e seus resultados. O Capítulo 8 trata da implantação da aplicação. O Capítulo 9 discute os resultados obtidos à luz dos objetivos estabelecidos. O Capítulo ?? analisa os

trabalhos e as ferramentas relacionados à proposta. Por fim, o Capítulo ?? apresenta as conclusões e aponta possibilidades de trabalhos futuros.

2 MATERIAIS E MÉTODOS

Este capítulo descreve os procedimentos metodológicos, os recursos materiais e as ferramentas computacionais empregadas no desenvolvimento da plataforma proposta. A seleção das abordagens levou em consideração tanto a natureza exploratória do problema abordado, que é a carência de soluções capazes de combinar a organização de coleções pessoais com a personalização aprofundada da interface, quanto os objetivos práticos deste trabalho, voltados à construção de um protótipo funcional.

2.1 Metodologia da Pesquisa

Esta seção caracteriza a pesquisa quanto aos procedimentos adotados para a construção do referencial teórico e quanto à sua natureza aplicada.

2.1.1 Pesquisa Bibliográfica

A construção do referencial teórico deste trabalho apoiou-se na revisão de literatura, recurso metodológico que, conforme (FONTELLES et al., 2009), abrange a análise de materiais previamente publicados em diversos suportes, incluindo livros, periódicos científicos, teses, documentos eletrônicos e demais fontes de conhecimento consolidado. Essa fundamentação é essencial para situar a pesquisa no estado da arte e identificar lacunas que justifiquem a proposta desenvolvida.

O processo de revisão foi conduzido em três momentos. No primeiro, delimitaram-se os eixos temáticos que sustentam o trabalho, a saber: engenharia de requisitos, modelagem conceitual com UML, padrões arquiteturais para aplicações *web*, frameworks de desenvolvimento *frontend* e *backend*, persistência relacional e princípios de usabilidade em interfaces personalizáveis. A escolha desses eixos refletiu as decisões técnicas que precisariam ser tomadas ao longo do projeto.

No segundo momento, procedeu-se à busca de materiais, utilizando-se como

bases primárias o Google Scholar e o Portal de Periódicos da CAPES, este último acessado via sistema CAFE. Complementarmente, foram realizadas consultas em documentações oficiais de frameworks e em publicações de conferências da área de engenharia de *software*. Privilegiaram-se produções dos últimos anos, sem, contudo, excluir obras seminais como as de (SOMMERVILLE, 2011), (PRESSMAN; MAXIM, 2016) e (LARMAN, 2007), cuja relevância para os temas tratados permanece indiscutível.

No terceiro momento, os materiais coletados passaram por triagem a partir da leitura dos resumos, seguida de leitura integral dos trabalhos considerados pertinentes. As informações extraídas foram organizadas em sínteses temáticas, revisadas em duas rodadas para assegurar precisão nas citações e consistência na argumentação desenvolvida ao longo do texto.

2.1.2 Pesquisa Aplicada

O trabalho adota também a perspectiva da pesquisa aplicada, caracterizada por (GERHARDT; SILVEIRA, 2009) como aquela cujo propósito é gerar conhecimento voltado à resolução de problemas concretos. No contexto desta pesquisa, tal abordagem materializa-se na transposição dos fundamentos teóricos levantados para decisões concretas de projeto e implementação do sistema.

Dessa articulação resultam três contribuições práticas principais: a especificação de requisitos fundamentada em (SOMMERVILLE, 2011) e (PRESSMAN; MAXIM, 2016); a elaboração do modelo conceitual baseada na abordagem de (LARMAN, 2007); e a definição da arquitetura do sistema em camadas, inspirada em (FOWLER, 2002), com comunicação entre *frontend* e *backend* pautada pelo estilo REST (FIELDING, 2000).

2.2 Materiais

2.2.1 Equipamentos

O desenvolvimento foi conduzido em um computador pessoal cuja configuração atende aos requisitos de execução simultânea do ambiente de desenvolvimento,

do contêiner de banco de dados, do servidor de aplicação e dos navegadores utilizados para validação da interface. O Quadro 1 sintetiza as especificações do equipamento.

Quadro 1 – Configuração do equipamento utilizado no desenvolvimento

Componente	Especificação
Processador	AMD Ryzen 5 5500 (6 núcleos, 12 <i>threads</i> , 3,6 GHz, <i>turbo</i> de 4,2 GHz)
Placa-mãe	MSI A520M-A PRO (soquete AM4, <i>chipset</i> AMD A520, M-ATX)
Memória RAM	16 GB DDR4 (2×8 GB Kingston Fury Beast, 3200 MHz, CL16)
Placa de vídeo	ASRock Radeon RX 6600 Challenger D (8 GB GDDR6, 128-bit)
Armazenamento	SSD Kingston NV3 de 1 TB (M.2 2280, PCIe NVMe)
Gabinete	Aigo Darkflash E330M (<i>mid-tower</i> , lateral de vidro)
Sistema operacional	Windows 11

Fonte: Elaborada pelo autor

2.2.2 Softwares

As ferramentas de *software* empregadas neste trabalho foram escolhidas considerando três critérios: aderência técnica ao escopo do projeto, disponibilidade gratuita ou licenciamento de código aberto, e existência de documentação consolidada que facilitasse a curva de aprendizado. As subseções da Seção 2.3 descrevem cada uma delas, contextualizando seu uso específico na construção da plataforma proposta.

2.3 Ferramentas Utilizadas

2.3.1 Visual Studio Code

O Visual Studio Code (VS Code) é um editor de código multiplataforma mantido pela Microsoft e distribuído gratuitamente. Sua arquitetura baseada em extensões permite estender suas capacidades para atender a diferentes ecossiste-

mas de desenvolvimento, aproximando-o da funcionalidade de um ambiente de desenvolvimento integrado (IDE) tradicional, porém com consumo de recursos mais enxuto.

Neste trabalho, a ferramenta concentrou três frentes distintas de uso: a edição do código *backend* em TypeScript, a edição do código *frontend* em React e a escrita do presente documento em LaTeX, viabilizada pela extensão *LaTeX Workshop*. A centralização dessas atividades em um único editor simplificou o fluxo de trabalho e reduziu o tempo dedicado a alternância entre ambientes.

2.3.2 Node.js e NestJS

O ecossistema Node.js consiste em um ambiente de execução JavaScript construído sobre o motor V8 do Chrome, que habilita a execução desta linguagem fora do navegador, particularmente em contextos de servidor. Por cima dele, o NestJS oferece um *framework* opinativo inspirado em conceitos do Angular, organizando a aplicação em módulos coesos compostos por *controllers*, *providers* e *services*. Essa organização modular alinha-se aos preceitos de separação de responsabilidades defendidos por (FOWLER, 2002).

No sistema proposto, essa dupla compõe a camada responsável pela lógica de negócio. Suas principais atribuições incluem a autenticação de usuários e a emissão de *tokens* de sessão, o processamento das operações sobre categorias, coleções, itens, *layouts* e Elementos, a aplicação das regras de privacidade configuradas no nível das coleções, o gerenciamento dos vínculos sociais (amizades e acompanhamentos) e o disparo de notificações decorrentes dessas interações.

Complementam o *backend* algumas bibliotecas do próprio ecossistema: o Passport e o módulo JWT do NestJS, responsáveis pela emissão e validação dos *tokens* de sessão; o bcrypt, utilizado no armazenamento das senhas em forma de *hash*, conforme exigido pelo RNF-03; o Nodemailer, encarregado do envio das mensagens de recuperação de senha; e um mecanismo de limitação de requisições, que restringe as tentativas de autenticação por endereço de origem.

2.3.3 PostgreSQL e Docker

O PostgreSQL, em sua versão 16, é um sistema gerenciador de banco de dados relacional maduro, de código aberto, reconhecido por sua fidelidade ao padrão SQL e por seu suporte robusto a transações. Sua integração com aplicações Node.js é bem consolidada no mercado, o que pesou em sua escolha como camada de persistência deste projeto.

A modelagem do banco espelha a organização do modelo de domínio, contemplando as entidades de usuários, categorias, coleções, itens, *layouts* e Elementos, sendo que o conteúdo e o estilo exibidos nas fichas residem nos próprios Elementos. Estruturas relacionadas à dimensão social (solicitações de amizade, acompanhamento de coleções, eventos de atualização e notificações), às preferências visuais globais e ao controle de sessões e de redefinição de senha também residem nessa camada.

A instalação do PostgreSQL foi realizada por meio do Docker, plataforma que abstrai o processo de provisionamento da instância em um contêiner isolado. Essa decisão trouxe dois benefícios imediatos: a reprodutibilidade do ambiente entre máquinas distintas e a facilidade de resetar a base durante os ciclos de teste, sem comprometer o sistema operacional hospedeiro.

2.3.4 Prisma

O Prisma é a ferramenta de mapeamento objeto-relacional adotada para intermediar o acesso do *backend* ao PostgreSQL. A partir de um arquivo de esquema, no qual as entidades são declaradas em uma linguagem própria, o Prisma gera um cliente tipado em TypeScript, de modo que consultas e escritas passam a ser verificadas pelo compilador antes da execução, um ganho relevante em um domínio com muitas associações entre entidades.

Duas características pesaram na escolha. A primeira é o mecanismo de migrações: cada alteração no esquema gera um arquivo versionado com o SQL correspondente, o que preserva o histórico da evolução do banco e permite reconstruí-lo do zero de forma determinística. A segunda é a possibilidade de dividir o esquema em vários arquivos, recurso empregado neste trabalho para separá-lo por assunto,

acompanhando os pacotes do modelo de domínio apresentados no Capítulo 4. O detalhamento do mapeamento entre o modelo de objetos e o modelo relacional é apresentado no Capítulo 5.

2.3.5 DBeaver

O DBeaver é um cliente gráfico multiplataforma voltado à administração de bancos de dados relacionais, compatível com os principais SGBDs disponíveis no mercado, entre eles o PostgreSQL. Por meio de sua interface visual, a ferramenta permite inspecionar estruturas, executar consultas SQL diretamente no banco e exportar resultados em diferentes formatos.

Ao longo do desenvolvimento, o DBeaver complementou o fluxo de trabalho ao oferecer uma alternativa visual ao terminal interativo do PostgreSQL. Foi particularmente útil para verificar a integridade das associações entre entidades, confirmando, por exemplo, que um item estava vinculado à coleção correta e utilizava os Elementos definidos pelo *layout* aplicável, e para explorar os dados inseridos durante as sessões de teste manual.

2.3.6 React e Vite

O React é uma biblioteca para construção de interfaces a partir de componentes, na qual a tela é descrita como função do estado da aplicação. O Vite, por sua vez, é a ferramenta de construção e o servidor de desenvolvimento adotados para executá-la: durante o desenvolvimento, serve os módulos diretamente ao navegador, o que torna a atualização em tela praticamente instantânea; para produção, gera um pacote otimizado. A aplicação resultante é uma *single-page application* (SPA), renderizada inteiramente no navegador.

A adoção do Vite em lugar do Next.js, alternativa natural no ecossistema React, foi deliberada, e apoia-se em três observações sobre a natureza deste sistema. Em primeiro lugar, os recursos de renderização no servidor e de geração estática que justificam o Next.js visam sobretudo a indexação por mecanismos de busca, e todo o conteúdo da plataforma fica atrás de autenticação, sem páginas públicas a indexar. Em segundo lugar, o modelo de autenticação do *backend* baseia-se no envio

do *token* no cabeçalho **Authorization**, ao passo que as abordagens mais recentes do Next.js pressupõem a leitura de *cookies* no servidor antes da renderização, o que exigiria trabalhar contra o desenho do *framework*. Por fim, a tela mais complexa do sistema, o editor de *layouts*, é integralmente interativa e depende do estado mantido no navegador, de modo que a renderização prévia no servidor não traria ganho. Manteve-se, assim, uma configuração mais simples e diretamente alinhada ao problema.

A camada de apresentação foi construída com essa combinação, complementada pelo React Router para o roteamento entre telas, pelo TanStack Query para a busca e o *cache* dos dados vindos da API e pelo Axios para as requisições HTTP. As telas contemplam o cadastro e a autenticação de usuários, a recuperação e a redefinição de senha, o *dashboard* que lista as categorias do usuário, a navegação entre categorias, coleções e itens, o editor visual de *layouts* (com funcionalidade de *drag-and-drop* de Elementos), a exibição e a edição de fichas individuais de itens, o painel de personalização visual global, as telas de interação social e o *feed* de atualizações. A renderização dinâmica das fichas, que varia conforme os Elementos configurados em cada *layout*, é um dos pontos em que as capacidades de composição do React foram mais intensamente exploradas.

2.3.7 Figma

O Figma é uma ferramenta colaborativa de *design* de interfaces executada diretamente no navegador, sem necessidade de instalação local. Seus recursos incluem a criação de componentes reutilizáveis, a definição de sistemas de *design* (tipografia, cores, espaçamentos) e a prototipação de fluxos interativos entre telas.

No presente trabalho, o Figma foi utilizado na fase de concepção visual da plataforma, antes da implementação efetiva. Essa etapa permitiu avaliar a disposição dos elementos, a hierarquia visual e a consistência entre telas, reduzindo a necessidade de refatorações estéticas posteriores no código. Foram prototipadas, entre outras, as telas do *dashboard*, do editor de *layouts* e das fichas de itens em modo de leitura e de edição.

2.3.8 Astah Community

O Astah Community, desenvolvido pela empresa japonesa Change Vision, é uma ferramenta voltada à modelagem UML que disponibiliza, em sua versão gratuita para uso acadêmico, recursos para a construção dos principais tipos de diagrama previstos pelo padrão.

Sua utilização neste trabalho concentrou-se na produção dos artefatos de análise: os diagramas de casos de uso, que formalizam as interações dos atores com o sistema, e o modelo de domínio, que evidencia as classes conceituais e suas associações. A organização dos diagramas em pacotes, facilitada pelo Astah, foi especialmente útil para representar visualmente os sete agrupamentos conceituais do modelo de domínio deste projeto (Usuário, Sistema, Coleções, Editor de Layout, Aparência, Interação Social e Notificações).

2.4 Métodos de Desenvolvimento

2.4.1 Elicitação e especificação de requisitos

A primeira etapa metodológica consistiu em identificar, analisar e especificar as funcionalidades que a plataforma precisaria oferecer aos seus usuários. Adotou-se para isso a classificação tradicional proposta por (SOMMERVILLE, 2011), que distingue requisitos funcionais, descrições do que o sistema deve fazer, de requisitos não funcionais, que tratam de propriedades como desempenho, segurança e usabilidade.

O levantamento partiu da análise crítica de limitações observadas em plataformas já existentes para gerenciamento de coleções pessoais, especialmente sua rigidez quanto à estrutura de dados e à aparência das fichas. Essa reflexão orientou a formulação de requisitos que valorizassem a autonomia do usuário sobre a organização e a apresentação de seu acervo. Cada requisito funcional foi descrito segundo um formato padronizado contendo ação, objeto, atributos, exemplos e restrições, com o verbo *gerenciar* adotado de forma recorrente para representar operações CRUD (cadastrar, editar e excluir). A relação completa dos requisitos é apresentada no Capítulo 4.

2.4.2 Modelagem conceitual

A partir dos requisitos elicitados, procedeu-se à modelagem conceitual do sistema, etapa na qual as entidades do mundo real relevantes ao domínio são representadas, juntamente com seus atributos e relações. Tomou-se como base a proposta de (LARMAN, 2007), para quem o modelo de domínio é uma ferramenta visual que evidencia os conceitos significativos antes de qualquer decisão de implementação.

Diante da quantidade de entidades identificadas, adotou-se a organização do diagrama em sete pacotes conceituais: Usuário, Sistema, Coleções, Editor de Layout, Aparência, Interação Social e Notificações. Esse agrupamento, além de reduzir a sobrecarga visual do diagrama, evidencia a separação de responsabilidades que seria posteriormente materializada tanto na divisão do esquema do banco em arquivos por assunto quanto na estrutura modular do *backend* implementado com NestJS.

2.4.3 Definição arquitetural

A arquitetura do sistema seguiu o modelo em camadas descrito por (FOWLER, 2002), com separação clara entre apresentação, lógica de negócio e persistência. A camada de apresentação é implementada em React, com Vite, e comunica-se com o *backend* em NestJS por meio de uma API baseada no estilo REST (FIELDING, 2000), enquanto este último acessa o PostgreSQL por meio do Prisma, que mapeia objetos para tabelas relacionais.

Essa separação em camadas foi uma decisão consciente para permitir que cada responsabilidade fosse testada e evoluída de forma independente. A adoção do estilo REST, por sua vez, ofereceu um vocabulário padronizado para a API (métodos HTTP, códigos de *status* e representações em JSON) que facilita tanto a documentação quanto eventuais evoluções futuras.

2.4.4 Prototipação

Antes da codificação, as telas centrais do sistema foram prototipadas no Figma, o que permitiu antecipar discussões sobre disposição de elementos, nomen-

clatura de botões e organização de conteúdo em cada tela. Particular atenção foi dada ao editor de *layouts*, já que a qualidade da interação com essa tela influencia diretamente a percepção que o usuário terá sobre a personalização oferecida pela plataforma.

A prototipação também serviu para definir o sistema de *design* de base, incluindo paleta de cores, tipografia e componentes reutilizáveis, que posteriormente orientaram a implementação visual em React.

2.4.5 Implementação

A codificação foi conduzida de forma iterativa e incremental, da camada de persistência em direção à interface. Iniciou-se pela modelagem do esquema do banco, estabelecendo as tabelas e relacionamentos derivados do modelo conceitual. Em seguida, foi implementada a camada de lógica de negócio, organizada em módulos NestJS que correspondem aos pacotes do modelo de domínio. Por fim, foi desenvolvida a camada de apresentação em React, consumindo as APIs disponibilizadas pelo *backend*.

A escolha pela ordem *backend-para-frontend* decorre do fato de que as telas dependem dos dados e regras já disponíveis na API. Durante toda a implementação, ajustes pontuais nos requisitos ou na modelagem foram feitos quando dificuldades práticas apontavam para oportunidades de refinamento.

2.4.6 Gestão do projeto

O planejamento do projeto foi estruturado em ciclos quinzenais, nos quais as tarefas eram definidas, priorizadas e acompanhadas ao longo de cada período. Essa abordagem possibilitou melhor controle do tempo, organização das entregas e adaptação contínua às necessidades do projeto.

O versionamento do código e do próprio documento foi feito com o Git, em repositórios separados para o *backend*, o *frontend* e a monografia. Além de preservar o histórico das alterações, essa prática serviu ao registro das decisões de projeto: cada mudança relevante de rumo ficou associada ao momento em que foi tomada, o que facilitou recuperar as justificativas na redação deste texto.

2.4.7 Estratégia de Testes

A verificação do sistema apoiou-se em duas frentes complementares. A primeira é a de testes automatizados no *backend*, escritos com o Jest e organizados junto ao código que verificam: cada serviço e cada controlador possui seu próprio arquivo de teste, no qual as dependências externas são substituídas por implementações simuladas, de modo que a regra de negócio seja exercitada de forma isolada. Essa estratégia é particularmente relevante nas regras de privacidade e de acompanhamento de coleções, cujas combinações de vínculo de amizade e nível de visibilidade seriam trabalhosas de cobrir manualmente. A segunda frente é a de testes manuais exploratórios, conduzidos sobre a interface a cada funcionalidade concluída, com apoio do DBeaver para inspecionar o estado do banco após cada operação. Essa verificação direta foi o meio adotado para validar aspectos visuais e de interação que os testes automatizados não alcançam, como o comportamento do editor de *layouts* durante o arrastar e soltar de Elementos. Os procedimentos de teste e seus resultados são detalhados no capítulo dedicado aos testes.

3 DESCRIÇÃO GERAL DO PRODUTO

Este capítulo apresenta uma visão panorâmica da plataforma proposta, descrevendo seu propósito, os perfis de usuários atendidos e as fronteiras que delimitam o escopo do desenvolvimento. A descrição aqui contida serve como base para o detalhamento técnico e a especificação de requisitos apresentados nos capítulos subsequentes.

3.1 Perspectiva do Produto

O produto consiste em uma plataforma web de gerenciamento e catalogação de acervos pessoais. O sistema surge como uma solução para o problema da rigidez em softwares de inventário tradicionais, que geralmente oferecem campos fixos que não se adaptam adequadamente a diferentes tipos de itens.

A perspectiva central do produto é a **flexibilidade estrutural**. Por meio de uma arquitetura baseada em **Elementos** visuais reutilizáveis, o usuário deixa de ser apenas um consumidor de formulários prontos para se tornar o arquiteto de sua própria organização. O sistema permite que colecionadores de diferentes nichos (como livros, moedas ou jogos) utilizem a mesma plataforma, mas com experiências de preenchimento e visualização completamente distintas e adequadas a cada domínio.

3.2 Funções do Produto

As funcionalidades do sistema estão organizadas em cinco eixos principais que sustentam a proposta de valor da plataforma:

- **Organização Multinível:** Estruturação lógica dos dados em **Categorias** (agrupadores temáticos), **Coleções** (subdivisões onde os itens residem) e **Itens** (a unidade individual de conteúdo). A Coleção é o nível em que se

configuram a privacidade e o layout, de modo que um mesmo acervo pode reunir partes públicas e partes reservadas sob uma única Categoria.

- **Customização Estrutural (Editor de Layout):** Motor que permite criar e editar modelos (*templates*) visuais. O usuário pode adicionar, reposicionar e redimensionar **Elementos** dos seis tipos oferecidos pelo editor: texto, imagem, forma, ícone, classificação e data.
- **Interação Social:** Funcionalidades que permitem adicionar amigos, seguir coleções visíveis de outros usuários e visualizar atualizações de acervos em um *feed* cronológico.
- **Personalização Visual Global:** Controle sobre a estética da interface por meio de uma biblioteca de paletas de cores e tipografia, com estilo claro, escuro ou automático, garantindo que o ambiente digital reflita o estilo do colecionador.
- **Notificação:** Comunicação ao usuário dos eventos que o envolvem, como solicitações de amizade e atualizações nas coleções que ele acompanha, reunidas em uma área dedicada dentro da plataforma.

3.3 Características dos Usuários

O produto foi concebido para atender a dois perfis principais de interação, identificados durante a fase de concepção:

- **Usuário Organizador (Casual):** Utiliza a plataforma para manter listas simples de seus pertences. Prefere associar às suas coleções os *layouts* padrão oferecidos pelo sistema, focando na velocidade e praticidade do cadastro.
- **Usuário Curador (Entusiasta):** Possui um envolvimento profundo com seus acervos. Este perfil dedica tempo ao **Editor de Layout** para criar fichas técnicas detalhadas e esteticamente personalizadas, buscando uma experiência de catalogação profissional.

3.4 Restrições Gerais

O desenvolvimento e a operação do sistema estão condicionados aos seguintes fatores limitadores:

- **Conectividade:** Por se tratar de uma aplicação *web* cliente-servidor, com os dados mantidos no servidor, o acesso ao acervo e às funcionalidades sociais exige conexão estável com a internet.
- **Compatibilidade de Interface:** O sistema é otimizado para navegadores modernos que suportam tecnologias de renderização dinâmica. Embora seja responsivo, a experiência avançada do **Editor de Layout** é otimizada para dispositivos de médio e grande porte, como *tablets* e *desktops*.
- **Escopo de Mídia:** O armazenamento de arquivos (imagens) está sujeito a limites de tamanho definidos para garantir a performance de carregamento e a viabilidade da infraestrutura do servidor.

3.5 Diferenciais do Produto

Diferente de planilhas eletrônicas ou aplicativos de listas estáticas, o grande diferencial desta plataforma é o **acoplamento entre dado e forma**. Enquanto outras ferramentas tratam o registro de um item como uma simples linha em uma tabela, este produto trata cada item como uma experiência visual única. O usuário define não apenas *o que* guardar, mas *como* aquela informação deve ser apresentada, proporcionando uma ferramenta de curadoria que evolui junto com a coleção.

4 ELICITAÇÃO DE REQUISITOS E ANÁLISE

A engenharia de requisitos é uma etapa fundamental no processo de desenvolvimento de software, sendo responsável por definir o que o sistema deve fazer e sob quais restrições ele deve operar (VALENTE, 2020). Segundo (SOMMERVILLE, 2011), essa disciplina engloba as atividades de descoberta, análise, documentação e validação dos requisitos, constituindo a base sobre a qual todo o projeto será construído. (PRESSMAN; MAXIM, 2016) complementam essa visão ao afirmar que a engenharia de requisitos envolve a coleta de informações sobre o sistema a ser desenvolvido, a análise do problema e a descrição, classificação e priorização dos requisitos identificados.

O processo de elicitação de requisitos, também chamado de levantamento ou descoberta de requisitos, consiste em reunir informações junto aos stakeholders para compreender suas necessidades e expectativas em relação ao sistema (SOMMERVILLE, 2011). Diversas técnicas podem ser empregadas nessa etapa, como entrevistas, questionários, workshops, prototipação e análise de cenários de uso (VALENTE, 2020)

Este capítulo apresenta os requisitos identificados para o sistema proposto, organizados em requisitos do usuário e requisitos do sistema, sendo estes últimos subdivididos em requisitos funcionais e não funcionais.

4.1 Requisitos do Usuário

Requisitos de usuário são declarações, em linguagem natural com diagramas, dos serviços que o sistema deverá fornecer aos seus usuários e das restrições com as quais ele deve operar (SOMMERVILLE, 2011). Pressman e Maxim (2016) destacam que um documento de requisitos de sistema precisa ser claro, consistente e completo, pois servirá de referência para desenvolvedores, gerentes de projeto e engenheiros

de testes ao longo de todo o ciclo de vida do software.

Antes de apresentar os requisitos propriamente ditos, é importante compreender os **quatro** conceitos centrais que estruturam a modelagem de dados da plataforma proposta: **Elemento**, **TipoElemento**, **Layout** e **Item**. Para facilitar a compreensão do papel de cada um no sistema, vale partir de uma analogia com ferramentas de edição visual consolidadas no mercado, como o *Canva*. Nesses editores, o usuário monta uma tela de forma livre, acrescentando componentes gráficos a partir de uma paleta, como caixas de texto, imagens e ícones, e ajustando em seguida a posição e o tamanho de cada um. A plataforma proposta adota essa mesma dinâmica de montagem visual, mas com um diferencial fundamental: a tela montada pelo usuário não é uma arte gráfica isolada, e sim um **modelo de dados reutilizável**, que pode ser compartilhado por dezenas ou centenas de itens de uma coleção, preservando a individualidade das informações de cada um.

A partir dessa analogia, os quatro conceitos podem ser entendidos na mesma ordem. O **Elemento** é cada peça que o usuário acrescenta à tela, como uma caixa de texto, uma imagem ou uma classificação por estrelas. É nele que residem tanto o conteúdo exibido quanto os atributos que determinam onde ele aparece: posição, largura, altura e camada de sobreposição. O modo como ele aparece fica a cargo do **EstiloDeElemento**, conjunto de propriedades visuais associado a cada Elemento, tratado adiante. O **TipoElemento** é a classificação desse Elemento dentro do conjunto fixo de tipos que o editor oferece. Trata-se de uma enumeração, e não de uma entidade com atributos próprios, de modo que dois Elementos do mesmo tipo, como “Autor” e “Gênero”, ambos de texto, são registros distintos, cada um com suas próprias coordenadas. O **Layout** é a tela montada, isto é, o conjunto de Elementos organizados visualmente, funcionando como um modelo reutilizável associado a uma coleção. E o **Item** é uma unidade concreta do acervo do usuário, por exemplo o livro “Duna”, que parte desse modelo e pode manter um layout próprio. O conteúdo exibido pode estar diretamente nos Elementos textuais, como em “Autor:” ou “Autor: Frank Herbert”, e o usuário pode separar rótulos e valores criando Elementos independentes quando desejar.

Uma analogia complementar, mais próxima da estrutura de dados, é a de um **molde reutilizável**. O *Layout* funciona como um molde já desenhado, que

define quais Elementos existem, sua posição e seu tamanho, mas não o que cada um exibe; os Elementos são as peças desenhadas nesse molde; e o *Item* é a unidade que parte dele dentro de uma coleção. Quando necessário, um item pode receber um layout próprio por meio de *override*, sem alterar o layout da coleção nem os demais itens.

Em termos práticos, considere um exemplo concreto. Suponha que o usuário crie um Layout chamado “*Card de Livro*” contendo quatro elementos: um de *Imagem* para a capa, um de *Texto* para o autor, um elemento de *Classificação* de um a cinco estrelas e outro de *Texto* para notas pessoais. Cada elemento possui posição e tamanho próprios dentro da área de edição do layout. Ao cadastrar o item “*Duna*”, o sistema gera os elementos visuais correspondentes para preenchimento e leitura: para o elemento *Imagem*, a capa; para o primeiro *Texto*, o autor “*Frank Herbert*”; para o elemento *Classificação*, o valor 5; e para o elemento *Texto*, a nota “*Obra-prima do gênero*”. Desse modo, o Layout e seus elementos permanecem reutilizáveis por diferentes itens da coleção, enquanto cada item preserva sua própria ficha visual.

Essa separação traz três benefícios diretos para o projeto:

- (i) permite que o usuário reutilize o mesmo layout para múltiplos itens sem duplicação de estrutura;
- (ii) viabiliza que cada item preserve valores distintos, respeitando a individualidade de cada obra cadastrada; e
- (iii) permite modificações individuais em um item específico, como adicionar um Elemento extra apenas a um único livro, sem afetar o layout original nem os demais itens (RF-17).

O Quadro 2 apresenta, de forma resumida, a definição dos quatro conceitos centrais e de dois termos que deles derivam e são usados ao longo do texto: o *EstiloDeElemento*, que reúne as propriedades visuais de um Elemento, e o layout padrão, oferecido pelo sistema.

Cabe esclarecer dois pontos antes de apresentar os requisitos. Em primeiro lugar, o conjunto de tipos de elemento é **fixo** e definido por uma enumeração

(TipoElemento), e não um cadastro gerido em tempo de execução. Essa decisão segue o modelo de editores visuais consolidados, como o Canva, nos quais os *tipos* de elemento (texto, imagem, formas, entre outros) fazem parte das capacidades nativas da ferramenta, enquanto o que o usuário cria livremente são *instâncias* desses tipos: quantos elementos quiser, acrescentando-os pela paleta e depois posicionando-os e redimensionando-os na tela. A eventual inclusão de um tipo inteiramente novo (por exemplo, vídeo incorporado) caracteriza uma evolução do próprio editor, e não um dado a ser cadastrado pelo usuário final.

Em segundo lugar, cabe registrar onde reside o conteúdo de cada item, pois é disso que depende a relação entre o Layout e o Item. O conteúdo não é guardado em uma entidade à parte: fica no próprio Elemento, ao lado da posição, do tamanho e das propriedades visuais. O Layout associado a uma coleção define apenas quais Elementos existem e como se organizam, sem guardar o que qualquer item exhibe. A individualização ocorre em outro nível: o Item recebe um **layout próprio**, gerado a partir do layout da coleção quando ela possui um, ou vazio quando não possui (RF-17). Cada Item passa a ter, dessa forma, seus próprios Elementos, nos quais conteúdo, posição e estilo permanecem indissociáveis. Chama-se **layout efetivo** de um item o seu layout próprio, quando existe, ou, na falta dele, o layout da coleção.

4.1.1 Organização em Pacotes

Devido à quantidade de classes conceituais envolvidas, o modelo de domínio foi organizado em sete pacotes, de modo a agrupar entidades que compartilham propósito semântico e reduzir a complexidade visual do diagrama. Essa segmentação segue a recomendação de (LARMAN, 2007), segundo o qual o agrupamento de classes em pacotes facilita a leitura do modelo, evidencia suas fronteiras conceituais e favorece o raciocínio modular sobre o sistema. Os pacotes adotados neste trabalho são:

- **Usuário:** concentra a entidade central da plataforma, responsável pelos dados de identificação e pelo vínculo com todas as demais entidades do sistema. Foi isolada em um pacote próprio por atuar como ponto de referência para os demais agrupamentos.

- **Coleções:** reúne as entidades que compõem a estrutura de organização do acervo em três níveis hierárquicos: Categoria, Coleção e Item. Inclui também a enumeração de privacidade. Representa o núcleo de domínio da aplicação, pois é nele que o usuário efetivamente cadastra e organiza seu conteúdo.
- **Editor de Layout:** agrupa as entidades responsáveis pela personalização visual das fichas de itens, incluindo Layout, Elemento, o EstiloDeElemento, que reúne as propriedades visuais de cada Elemento, e a enumeração TipoElemento, que define o conjunto fixo de tipos de elemento disponíveis no editor (texto, imagem, forma, ícone, classificação e data). É este pacote que sustenta o diferencial de customização da plataforma.
- **Interação Social:** concentra as entidades ligadas aos vínculos entre usuários, como as solicitações de amizade (Amizade e StatusAmizade) e o acompanhamento de coleções de outros usuários (Seguir Coleção).
- **Notificações:** reúne as entidades responsáveis pelo registro e pela exibição de eventos relevantes ao usuário: a própria entidade Notificação, a enumeração TipoNotificacao, a entidade AtualizaçãoEvento e a enumeração TipoEvento. A separação deste pacote em relação ao de Interação Social reflete a distinção entre o vínculo social em si e o mecanismo de comunicação dessas ocorrências ao usuário, além de isolar a lógica de geração e exibição de notificações para facilitar futuras extensões do sistema.
- **Aparência:** reúne as entidades que definem a configuração visual global da interface: a biblioteca de paletas do usuário, cada uma com suas cores, sua tipografia e seu estilo claro ou escuro.
- **Sistema:** reúne as entidades de apoio à operação da plataforma, que não pertencem ao domínio do acervo nem às interações entre usuários: a sessão de usuário, que permite revogar o acesso no momento do logout, e o token de redefinição de senha, de uso único e com prazo de validade. Foi isolado em pacote próprio por sustentar os requisitos de acesso sem participar das regras de negócio das coleções.

Essa organização reflete a separação de responsabilidades identificada durante a elicitação de requisitos: a entidade central do pacote *Usuário* sustenta o

cadastro e o perfil (RF-01 e RF-05), enquanto as do pacote *Sistema* viabilizam a autenticação, a alteração e a recuperação de senha (RF-02 a RF-04); as classes do pacote *Coleções* sustentam os requisitos funcionais de gestão de conteúdo (RF-06 a RF-13); as do pacote *Editor de Layout* suportam os requisitos de personalização estrutural (RF-14 a RF-17); as do pacote *Aparência* viabilizam a biblioteca de paletas oferecida no cadastro e gerenciada pelo usuário (RF-18 e RF-19); as do pacote *Interação Social* dão suporte às funcionalidades de rede entre usuários (RF-20 a RF-23); e as do pacote *Notificações* sustentam o mecanismo de comunicação de eventos do sistema ao usuário (RF-24).

Cabe observar que os grupos de requisitos e os pacotes conceituais são recortes distintos e não coincidem termo a termo. O grupo *Acesso ao Sistema* (RF-01 a RF-05), por exemplo, distribui-se entre dois pacotes: os dados de identificação do usuário residem em *Usuário*, ao passo que as estruturas que sustentam sessão e redefinição de senha residem em *Sistema*. O agrupamento em pacotes obedece à afinidade conceitual entre as classes, e não à organização adotada na especificação dos requisitos.

O modelo de domínio resultante é apresentado na Seção 4.4, ao final deste capítulo, depois de detalhados os requisitos e os casos de uso que o originaram.

4.2 Requisitos do Sistema

Sommerville (2011) afirma que os requisitos do sistema são tradicionalmente classificados em requisitos **funcionais**, que descrevem as funcionalidades que o sistema deve oferecer, e requisitos **não funcionais**, que definem restrições de qualidade como desempenho, segurança e usabilidade. Eles constituem o núcleo da especificação do sistema, pois delimitam o escopo das funcionalidades a serem implementadas, e sua classificação permite uma organização sistemática das especificações, facilitando tanto o desenvolvimento quanto a validação do sistema.

Assim, esta seção apresenta os requisitos identificados neste trabalho, organizados nas subseções a seguir.

4.2.1 Requisitos Funcionais

Segundo Sommerville (2011), requisitos funcionais descrevem as funções ou serviços que o sistema deve fornecer, ou seja, o que o sistema deve fazer. Eles especificam as ações que o sistema deve ser capaz de executar em resposta a entradas específicas ou em determinadas condições.

Os atores que interagem com o sistema são: **Usuário**, que representa a pessoa cadastrada na plataforma e responsável por criar, editar e gerenciar suas categorias, coleções, itens e interações sociais; e o **Sistema**, que atua de forma autônoma em funcionalidades como exibição de fichas de itens, envio de notificações, atualização do feed em tempo real e organização visual dos dados.

Para padronizar a especificação dos requisitos, adotou-se o verbo **gerenciar** como forma genérica de representar operações do tipo CRUD (cadastrar, editar e excluir), sempre que aplicável. Essa padronização reduz a repetição textual e torna a descrição dos requisitos mais objetiva.

Os requisitos funcionais estão organizados em seis grupos, apresentados de forma sequencial e seguindo uma ordem lógica de dependência entre funcionalidades, alinhada à jornada natural do usuário no sistema: iniciam pelo **Acesso ao Sistema** (cadastro, autenticação, recuperação e alteração de senha e gerenciamento de perfil); prosseguem com a **Coleção**, núcleo do sistema (categorias, coleções, privacidade, itens, suas fichas e formas de visualização); avançam para o **Editor de Layout** (criação de layouts, associação a coleções, compartilhamento entre usuários e personalização individual); seguem para a **Aparência**, camada de personalização visual que se aplica sobre o conteúdo já organizado (paletas de cores e tipografia, com estilo claro ou escuro); contemplam a **Interação Social** (amizade, acompanhamento de coleções e visualização de perfis); e culminam na **Notificação**, mecanismo de comunicação de eventos ao usuário. Os quadros 3 a 7 apresentam a definição dos campos e das classificações utilizadas na especificação, e os requisitos funcionais identificados para este sistema estão apresentados nos Quadros 8 a 31.

Quadro 2 – Conceitos centrais do modelo de domínio

Conceito	Descrição
Elemento	Componente visual reutilizável utilizado para compor layouts, como textos, imagens, classificações, datas e ícones. Cada elemento possui nome, tipo, conteúdo opcional, posição, largura, altura e camada de sobreposição, além do seu EstiloDeElemento.
EstiloDeElemento	Conjunto de propriedades visuais que define a aparência de um Elemento, sem interferir na sua posição nem no seu conteúdo. Reúne, para o texto, fonte, tamanho, peso, itálico, sublinhado, riscado, alinhamento horizontal e vertical, nível de título, entrelinha, espaçamento entre letras e apresentação em lista; para a forma e a imagem, forma do elemento, arredondamento e borda (cor, espessura e traço); para a forma, ainda, curva e ponta da linha; para a imagem, ajuste; para a classificação, nota máxima e ícone; para a data, formato e estética; e, para qualquer tipo, cor, cor de fundo, opacidade e rotação. É um conjunto extensível: novas propriedades acompanham a evolução do editor, sem alterar os demais conceitos.
TipoElemento	Conjunto fixo e predefinido de categorias de elemento que o usuário pode acrescentar no editor, à semelhança dos elementos básicos de ferramentas como o Canva. Compreende: texto, imagem, forma, ícone, classificação e data. É representado por uma enumeração, e não por uma entidade com atributos próprios, pois constitui um conjunto estável que faz parte das capacidades do próprio editor.
Layout	Estrutura personalizada que define quais elementos compõem a ficha de um item e como eles se organizam visualmente, sem guardar o conteúdo de nenhum item em particular. Funciona como um <i>template</i> reutilizável, que pode ser associado a uma coleção; um mesmo layout pode ser reutilizado por diferentes coleções. A associação é opcional: os itens de uma coleção que possui layout partem dele, e os de uma coleção sem layout partem de uma ficha vazia.
Layout padrão	Layout já montado, oferecido pelo sistema na biblioteca de layouts do usuário desde o cadastro (RF-01). Serve de facilitador para quem não deseja compor uma ficha do zero: pode ser associado a uma coleção como qualquer outro layout, e igualmente editado ou duplicado (RF-14 e RF-15). Não é aplicado automaticamente.
Item	Unidade individual de conteúdo cadastrada pelo usuário (por exemplo, um livro, filme ou jogo específico). Todo item pertence a uma coleção e parte do layout associado a ela, quando existe, podendo receber um layout próprio conforme RF-17.

Fonte: Elaborada pelo autor

Quadro 3 – Descrição dos Campos dos Requisitos Funcionais

Campo	Descrição
Código	Identificador único do requisito funcional (ex.: RF-01).
Grupo	Categoria à qual o requisito pertence, organizando funcionalidades semelhantes (ex.: Acesso ao Sistema, Coleção, etc.).
Descrição	Detalhamento do requisito estruturado em ação, objeto, atributos, exemplos e restrições.
Prioridade	Define o nível de importância do requisito (Essencial, Importante ou Desejável).
Operação	Classifica o tipo de operação (Entrada, Processamento ou Saída).
Ator	Entidade responsável pela execução ou interação (Usuário ou Sistema).

Fonte: Elaborada pelo autor

Quadro 4 – Grupos de Requisitos Funcionais

Grupo	Descrição
Acesso ao Sistema	Cadastro, autenticação, recuperação e alteração de senha e gerenciamento do perfil do usuário.
Coleção	Gerenciamento de categorias, coleções, privacidade, itens e respectivas formas de visualização.
Editor de Layout	Criação, associação, compartilhamento e personalização de layouts e Elementos.
Aparência	Configuração visual global da interface por meio de paletas de cores e tipografia, com estilo claro ou escuro.
Interação Social	Vínculos de amizade, acompanhamento de coleções e visualização de perfis de outros usuários.
Notificação	Geração, exibição e gerenciamento das notificações enviadas ao usuário.

Fonte: Elaborada pelo autor

Quadro 5 – Classificação de Prioridade dos Requisitos

Prioridade	Descrição
Essencial	Requisito indispensável para o funcionamento do sistema.
Importante	Requisito necessário para o funcionamento adequado, mas não crítico.
Desejável	Requisito opcional, podendo ser implementado posteriormente.

Fonte: Adaptado do modelo de requisitos

Quadro 6 – Classificação de Operação dos Requisitos

Operação	Descrição
Entrada	Dados fornecidos pelo usuário ao sistema.
Processamento	Tratamento e validação realizados pelo sistema.
Saída	Exibição ou retorno de informações ao usuário.

Fonte: Adaptado do modelo de requisitos

Quadro 7 – Atores do Sistema

Ator	Descrição
Usuário	Pessoa que utiliza o sistema para gerenciar suas coleções e interações.
Sistema	Responsável por execuções automáticas e processamento de dados.

Fonte: Elaborada pelo autor

Quadro 8 – RF-01 – Cadastrar Usuário

Código	RF-01
Grupo	Acesso ao Sistema
Descrição	Ação: Cadastrar Objeto: Usuário Atributos: nome (100 caracteres), email (150 caracteres), senha (mínimo 8 caracteres), foto de perfil (JPEG/PNG, máx. 5 MB), descrição (500 caracteres), código de usuário (gerado automaticamente pelo sistema, único e imutável). Exemplos: Usuário preenche nome ‘Camila Albieri Mattos’, email ‘camilamattos.mila@gmail.com’, senha ‘Camila@2024’ e conclui o cadastro com sucesso; o sistema gera automaticamente o código de usuário ‘camila#4827’. Usuário tenta cadastrar com o email ‘camilamattos.mila@gmail.com’, que já existe, e recebe mensagem de erro informando que o email já está em uso. Restrições: 1. Nome, email e senha são obrigatórios; foto de perfil e descrição são opcionais. 2. O email deve estar em formato válido (ex.: usuario@dominio.com). 3. Não podem ser registrados dois ou mais usuários com o mesmo email. 4. A senha deve ter no mínimo 8 caracteres. 5. A senha deve ser armazenada de forma criptografada (RNF-03). 6. O código de usuário é gerado automaticamente pelo sistema no momento do cadastro, deve ser único em toda a plataforma e não pode ser alterado posteriormente. 7. O código de usuário é o identificador público utilizado para buscas e envio de solicitações de amizade. 8. Ao concluir o cadastro, o sistema disponibiliza na biblioteca de aparência do usuário as paletas predefinidas que oferece, três claras e três escuras, deixando em uso a paleta ‘Claro’ ou a paleta ‘Escuro’, conforme o tema do sistema operacional no momento do cadastro (RF-18). 9. Ao concluir o cadastro, o sistema também disponibiliza na biblioteca de layouts do usuário os layouts padrão que oferece, prontos para serem associados a coleções, editados ou duplicados (RF-14 e RF-15).
Prioridade	Essencial
Operação	Entrada
Ator	Usuário

Fonte: Elaborada pelo autor

Quadro 9 – RF-02 – Autenticar Usuário (Login e Logout)

Código	RF-02
Grupo	Acesso ao Sistema
Descrição	Ação: Autenticar (login e logout) Objeto: Usuário Atributos: email (150 caracteres), senha (mínimo 8 caracteres), token de sessão (gerado automaticamente pelo sistema). Exemplos: Usuário informa ‘camilamattos.mila@gmail.com’ e ‘Camila@2024’, o sistema valida as credenciais e concede acesso à área restrita. Usuário informa email não cadastrado e recebe a mensagem ‘Credenciais de login inválidas’. Usuário clica em ‘Sair’ e tem sua sessão encerrada imediatamente, sendo redirecionado para a tela de login. Restrições: 1. Email e senha são obrigatórios para realizar o login. 2. O sistema deve verificar se o email está cadastrado e se a senha confere com o hash armazenado. 3. Não conferindo, o sistema deve exibir ‘Credenciais de login inválidas’ sem especificar qual campo está errado. 4. A cada autenticação bem-sucedida o sistema deve registrar uma sessão própria e vinculá-la ao token emitido. 5. O token de sessão deve ter prazo de validade definido, expirando automaticamente ao final desse prazo. 6. A validade da sessão deve ser verificada pelo sistema a cada requisição autenticada, de modo que uma sessão revogada deixe de conceder acesso imediatamente, mesmo que o token ainda não tenha expirado. 7. O logout deve revogar imediatamente a sessão ativa, tornando o token inutilizável a partir desse momento.
Prioridade	Essencial
Operação	Processamento
Ator	Usuário

Fonte: Elaborada pelo autor

Quadro 10 – RF-03 – Alterar Senha na Área Restrita

Código	RF-03
Grupo	Acesso ao Sistema
Descrição	Ação: Alterar senha Objeto: Usuário (autenticado) Atributos: senha atual (para confirmação), nova senha (mínimo 8 caracteres), confirmação da nova senha. Exemplos: Camila, já autenticada, acessa ‘Configurações de Conta’, informa a senha atual, define a nova senha ‘NovaSenha@2025’, confirma e o sistema atualiza a credencial. Camila informa a senha atual incorreta e recebe a mensagem ‘Senha atual inválida’. Camila digita a nova senha e a confirmação divergentes e recebe aviso de que os campos não coincidem. Restrições: 1. Esta operação só está disponível para o usuário autenticado, dentro da área restrita. 2. O usuário deve informar corretamente a senha atual antes de definir a nova senha. 3. A nova senha deve ter no mínimo 8 caracteres e ser diferente da senha atual. 4. A nova senha e sua confirmação devem coincidir. 5. A nova senha deve ser armazenada de forma criptografada (RNF-03). 6. Após a alteração, as demais sessões ativas do usuário devem ser invalidadas por segurança.
Prioridade	Importante
Operação	Entrada
Ator	Usuário

Fonte: Elaborada pelo autor

Quadro 11 – RF-04 – Recuperar Senha via E-mail

Código	RF-04
Grupo	Acesso ao Sistema
Descrição	Ação: Recuperar senha Objeto: Usuário (não autenticado) Atributos: email cadastrado, token de redefinição (gerado automaticamente, único e temporário), nova senha (mínimo 8 caracteres), confirmação da nova senha. Exemplos: Camila, sem estar logada, clica em ‘Esqueci minha senha’, informa seu email e recebe uma mensagem com link de redefinição. Camila abre o link recebido, define a nova senha e passa a conseguir autenticar-se com a nova credencial. Camila tenta usar um link de redefinição expirado e recebe a mensagem de que o link não é mais válido, com opção de solicitar um novo. Restrições: 1. A recuperação é iniciada por usuário não autenticado, a partir da tela de login. 2. O sistema solicita o email cadastrado e, se ele existir, envia um link de redefinição contendo um token único e temporário (RF-24). 3. Por segurança, a mensagem exibida ao solicitante deve ser a mesma independentemente de o email existir ou não, para não revelar quais emails estão cadastrados. 4. O token de redefinição deve expirar em no máximo 24 horas e ser de uso único. 5. Ao acessar o link, o usuário define a nova senha e sua confirmação, que devem coincidir e ter no mínimo 8 caracteres. 6. A nova senha deve ser armazenada de forma criptografada (RNF-03). 7. Concluída a redefinição, o token é invalidado e as sessões ativas anteriores são encerradas.
Prioridade	Importante
Operação	Entrada
Ator	Usuário

Fonte: Elaborada pelo autor

Quadro 12 – RF-05 – Gerenciar Perfil do Usuário

Código	RF-05
Grupo	Acesso ao Sistema
Descrição	Ação: Editar Objeto: Usuário Atributos: nome de exibição (100 caracteres), email (150 caracteres), foto de perfil (JPEG/PNG, máx. 5 MB), descrição/bio (500 caracteres), token de verificação de e-mail (gerado automaticamente pelo sistema quando há alteração de e-mail). Exemplos: Usuário acessa ‘Editar Perfil’, altera o nome de exibição para ‘Cami Mattos’, adiciona bio ‘Leitora e colecionadora de histórias’ e salva as alterações. Usuário altera o e-mail para ‘cami.mattos@gmail.com’; o sistema envia um link de verificação ao novo endereço e mantém o e-mail antigo ativo até a confirmação. Restrições: 1. O nome de exibição deve ter entre 3 e 100 caracteres e é obrigatório. 2. A foto de perfil deve estar em formato JPEG ou PNG com tamanho máximo de 5 MB. 3. A descrição/bio é opcional e pode ser deixada em branco. 4. Alterações só têm efeito após o usuário salvar explicitamente. 5. Ao alterar o e-mail, o sistema deve enviar um link de verificação para o novo endereço antes de efetivar a mudança. 6. O e-mail antigo permanece ativo até que o novo seja verificado pelo usuário. 7. O token de verificação de e-mail deve expirar em até 24 horas após o envio. 8. Não é permitido alterar o e-mail para um endereço já cadastrado por outro usuário. 9. A alteração de senha não é feita por este requisito, e sim pelo RF-03 (área restrita). 10. O código de usuário (RF-01) não pode ser alterado por este caso de uso.
Prioridade	Importante
Operação	Entrada
Ator	Usuário

Fonte: Elaborada pelo autor

Quadro 13 – RF-06 – Gerenciar Categoria

Código	RF-06
Grupo	Coleção
Descrição	Ação: Gerenciar Objeto: Categoria (agrupador temático de alto nível que reúne coleções relacionadas) Atributos: nome (100 caracteres), tipo/tema (texto livre de até 50 caracteres, com ‘livros’, ‘filmes’, ‘séries’, ‘jogos’ e ‘músicas’ oferecidos como sugestão; o usuário pode definir o seu próprio), ícone (emoji Unicode, opcional), imagem de capa (JPEG/PNG, opcional; enviada com até 10 MB e enquadrada pelo usuário na proporção do banner, e o recorte gravado tem até 2 MB), imagem original da capa (guardada apenas para permitir o reenquadramento, nunca exibida), área do recorte (posição e tamanho em pixels do original), cor de fundo do cartão (hexadecimal, opcional), descrição (300 caracteres), data de criação (gerada automaticamente). Exemplos: Usuário Camila cria a categoria ‘Livros’ com ícone personalizado e descrição ‘Tudo que leio ou pretendo ler’; a categoria é criada vazia, pronta para receber coleções. Camila edita a categoria, renomeando para ‘Biblioteca Pessoal’. Camila exclui a categoria ‘Jogos Antigos’ e confirma a exclusão permanente no diálogo de confirmação, que alerta sobre a remoção de todas as coleções e itens vinculados. Restrições: 1. O nome da categoria é obrigatório; ícone, imagem de capa e descrição são opcionais. 2. Não podem existir duas categorias com o mesmo nome para o mesmo usuário. 3. A categoria, ao ser criada, permanece vazia e não contém itens diretamente; os itens só podem ser cadastrados dentro de suas coleções (RF-07 e RF-09). 4. A categoria serve apenas como agrupador temático; a definição de layout ocorre no nível da coleção, e não no da categoria (RF-07 e RF-15). 5. A categoria não possui nível de privacidade próprio; a privacidade é configurada individualmente em cada coleção (RF-08). 6. A exclusão deve exibir diálogo de confirmação alertando que todas as coleções e itens vinculados serão removidos permanentemente. 7. A data de criação é gerada automaticamente pelo sistema e não pode ser editada. 8. O tema é gravado em letras minúsculas, sem espaços nas extremidades e sem acentuação, de modo que grafias diferentes da mesma palavra sejam tratadas como o mesmo tema. A exibição apresenta a primeira letra em maiúscula. 9. O ícone e a imagem de capa coexistem e são escolhidos separadamente: a capa ocupa o banner do cartão, e o emoji aparece sobre ela e no modo lista. Difere da coleção (RF-07), onde as duas formas são alternativas entre si. 10. Categoria sem ícone e sem capa é exibida apenas com a cor de

Quadro 14 – RF-07 – Gerenciar Coleção

Código	RF-07
Grupo	Coleção
Descrição	Ação: Gerenciar Objeto: Coleção (subdivisão estruturada que pertence a uma categoria e agrupa itens) Atributos: nome (100 caracteres), categoria pai (referência obrigatória), ícone (emoji Unicode ou imagem JPEG/PNG, opcional; a imagem é enviada com até 10 MB e enquadrada pelo usuário, e o recorte gravado tem até 2 MB), imagem original do ícone (guardada apenas para permitir o reenquadramento, nunca exibida), área do recorte (posição e tamanho em pixels do original), cor de fundo do cartão (hexadecimal, opcional), descrição (300 caracteres), layout associado (referência opcional a um layout do usuário), nível de privacidade (pública, somente amigos ou privada), tags (de nenhuma a dez, texto livre de até 30 caracteres cada, opcionais), data de criação (gerada automaticamente). Exemplos: Usuário cria a coleção ‘Lidos’ dentro da categoria ‘Livros’, definindo a privacidade como ‘pública’; a coleção passa a aceitar itens. Camila edita a coleção ‘Lidos’ para associar o layout ‘Card de Mangá’, que passa a valer para os itens dessa coleção. Camila exclui a coleção ‘Rascunhos’ e confirma no diálogo a remoção de todos os itens vinculados. Camila marca as coleções ‘Lidos’ e ‘Relidos’ com a tag ‘fantasia’ e, clicando nela, vê as duas reunidas, ainda que pertençam a categorias diferentes. Ao escrever uma tag no formulário, Camila recebe como sugestão ‘ficcao cientifica’, palavra já utilizada por outro usuário do sistema. Restrições: 1. O nome e a referência à categoria pai são obrigatórios. 2. Não podem existir duas coleções com o mesmo nome dentro da mesma categoria para o mesmo usuário. 3. Toda coleção pertence a exatamente uma categoria. 4. Por padrão, a privacidade de uma nova coleção é ‘pública’, podendo ser alterada conforme RF-08. 5. Toda coleção pode ter um layout associado, que determina os elementos exibidos e preenchíveis nos itens dessa coleção; um mesmo layout pode ser reutilizado por diferentes coleções (RF-15). 6. A exclusão deve exibir diálogo de confirmação alertando que todos os itens vinculados serão removidos permanentemente. 7. A data de criação é gerada automaticamente pelo sistema e não pode ser editada. 8. O usuário deve poder alternar entre os modos de visualização ‘grade’ e ‘lista’ a qualquer momento; o modo padrão é ‘grade’ e a escolha deve ser persistida entre sessões. 9. O usuário deve poder ordenar as coleções por pelo menos quatro critérios: nome crescente, nome decrescente, data de criação crescente e data de criação decrescente. 10. A ordenação padrão é por data de criação decrescente (mais recentes primeiro)

Quadro 15 – RF-08 – Gerenciar Privacidade da Coleção

Código	RF-08
Grupo	Coleção
Descrição	Ação: Configurar Objeto: Nível de visibilidade da coleção Atributos: nível de privacidade (privada, somente amigos, pública), coleção alvo (referência). Exemplos: Camila define a coleção ‘Lidos’ como ‘pública’, tornando-a visível a qualquer usuário. Camila altera a coleção ‘Diário’ para ‘privada’; ela deixa de aparecer para qualquer outro usuário. Camila define ‘Favoritos’ como ‘somente amigos’; apenas amigos confirmados passam a visualizá-la. Restrições: 1. A privacidade é configurada no nível da coleção; a categoria não possui privacidade própria (RF-06). 2. Coleções ‘privadas’ são visíveis unicamente pelo proprietário. 3. Coleções ‘somente amigos’ liberam visualização e acompanhamento apenas para usuários com vínculo de amizade aceito (RF-20). 4. Coleções ‘públicas’ ficam acessíveis a qualquer usuário cadastrado. 5. Por padrão, toda coleção recém-criada nasce como ‘pública’ (RF-07). 6. Se uma coleção ‘pública’ ou ‘somente amigos’ for alterada para ‘privada’, o sistema deve encerrar automaticamente o acompanhamento de todos os seguidores e remover seus eventos do feed (RF-21 e RF-22). 7. Se uma coleção ‘pública’ for alterada para ‘somente amigos’, o sistema deve manter o acompanhamento dos amigos confirmados e encerrar o dos demais seguidores (RF-20 e RF-21).
Prioridade	Essencial
Operação	Entrada
Ator	Usuário

Fonte: Elaborada pelo autor

Quadro 16 – RF-09 – Gerenciar Item

Código	RF-09
Grupo	Coleção
Descrição	Ação: Gerenciar Objeto: Item dentro de uma coleção Atributos: nome (200 caracteres), coleção pai (referência obrigatória), cor de fundo do cartão (hexadecimal, opcional), layout visual efetivo do item, Elementos herdados ou personalizados (Texto, Imagem, Forma, Ícone, Classificação e Data), conteúdo e estilo armazenados nos próprios Elementos, data de criação (gerada automaticamente). Exemplos: Camila cria o item ‘Duna’ na coleção ‘Lidos’ (dentro da categoria ‘Livros’); o item abre com o layout aplicável e Camila edita visualmente os textos, imagens, datas e a classificação que aparecem na ficha, como faria em um canvas. Camila adiciona individualmente um Elemento de Imagem e um texto extra ‘Personagem Favorito: Paul Atreides’, sem afetar os demais itens nem o layout da coleção. Posteriormente edita a classificação de 5 para 4 estrelas e, por fim, exclui o item ‘Duna – Rascunho’ confirmando no diálogo de confirmação. Restrições: 1. O nome do item é obrigatório; todos os demais campos são opcionais. 2. Todo item deve pertencer a uma coleção existente (RF-07); não é possível criar itens diretamente em uma categoria. 3. Ao ser criado, o item adota automaticamente a estrutura visual do layout associado à sua coleção (RF-15); caso a coleção não tenha layout, a ficha do item começa vazia e o próprio usuário monta o layout desse item (RF-17). O sistema não desenha uma ficha padrão: o que ele oferece são os layouts padrão da biblioteca, que o usuário pode associar à coleção quando não quiser compor do zero (RF-15). 4. O usuário pode adicionar, remover ou reposicionar Elementos individualmente no item sem afetar o layout da coleção nem os demais itens (RF-17). 5. Não podem existir dois itens com o mesmo nome dentro da mesma coleção. 6. Imagens enviadas devem estar em formato JPEG ou PNG com tamanho máximo de 5 MB por arquivo. 7. A exclusão deve exibir diálogo de confirmação informando que a ação é irreversível. 8. A data de criação é gerada automaticamente e não pode ser editada pelo usuário.
Prioridade	Essencial
Operação	Entrada
Ator	Usuário

Fonte: Elaborada pelo autor

Quadro 17 – RF-10 – Mover Item entre Coleções

Código	RF-10
Grupo	Coleção
Descrição	Ação: Mover Objeto: Item entre coleções, dentro da mesma categoria ou entre categorias distintas Atributos: item de origem (nome, categoria e coleção atuais), destino (coleção existente, podendo pertencer a uma categoria diferente). Exemplos: Camila arrasta ‘A Metamorfose’ da coleção ‘Para Ler’ para ‘Lidos’, ambas na categoria ‘Livros’; como as duas coleções usam o mesmo layout, o item some da origem e aparece no destino imediatamente, sem mudança de estrutura. Camila move ‘O Hobbit’ da coleção ‘Lidos’ para a coleção ‘Releituras Planejadas’. Camila move um item para uma coleção que usa um layout diferente e o sistema avisa que a estrutura de elementos pode mudar, exibindo prévia dos Elementos preservados. Restrições: 1. Um item pode pertencer a apenas uma coleção por vez. 2. A movimentação deve ser concluída com no máximo três comandos (ex.: arrastar, soltar e confirmar). 3. Se a coleção de destino usa o mesmo layout da coleção de origem, todos os dados e a estrutura de elementos são preservados automaticamente. 4. Se o item ainda usa o layout da coleção, ao ser movido ele passa a seguir o layout da coleção de destino; se já possui layout próprio, esse layout é preservado (RF-17). 5. A operação deve ser reversível via ação de desfazer (Ctrl+Z ou botão equivalente) em até 30 segundos após a confirmação. 6. As contagens de itens das coleções de origem e destino devem ser atualizadas imediatamente.
Prioridade	Importante
Operação	Entrada
Ator	Usuário

Fonte: Elaborada pelo autor

Quadro 18 – RF-11 – Exibir Ficha de Item em Modo Leitura

Código	RF-11
Grupo	Coleção
Descrição	Ação: Exibir em modo leitura Objeto: Ficha individual de item Atributos: layout efetivo do item, Elementos com conteúdo e estilo, data de criação do item, modo de acesso (dono, com acesso à edição do layout do item; visitante, somente leitura). Exemplos: Camila acessa a ficha do item ‘Duna’ da coleção ‘Lidos’ e visualiza todos os Elementos visuais, com a opção de editar o layout do item disponível. Um amigo visita a categoria pública ‘Livros’ de Camila, abre a coleção ‘Lidos’, acessa a ficha do item ‘Duna’ e visualiza os mesmos Elementos, porém sem nenhuma opção de edição. Elementos não preenchidos continuam visíveis para visitantes com indicação visual de que estão vazios, sem permitir edição. Restrições: 1. A ficha exibe os Elementos do layout efetivo do item; quando existir layout próprio do item, ele prevalece sobre o layout da coleção (RF-17). 2. Elementos sem conteúdo devem ser exibidos para dono e visitantes com indicação visual de que estão vazios; isso não concede permissão de edição ao visitante. 3. Para o dono do item, a ficha deve oferecer acesso direto à edição da própria ficha, isto é, do layout do item (RF-17). As ações de alterar os dados do item e de excluí-lo não ficam na ficha: elas pertencem ao menu de ações do item na listagem da coleção, de onde valem para qualquer item sem precisar abri-lo. 4. Para visitantes, a ficha é estritamente somente leitura, sem nenhuma opção de edição visível. 5. A ficha deve respeitar a aparência (tema e cores) configurada pelo dono (RF-18). 6. O sistema deve fornecer ao <i>front-end</i> as informações necessárias para exibir a ficha final do item em modo leitura, diferenciando visualmente o acesso do dono e o de visitantes.
Prioridade	Essencial
Operação	Saída
Ator	Sistema

Fonte: Elaborada pelo autor

Quadro 19 – RF-12 – Visualizar Itens de uma Coleção

Código	RF-12
Grupo	Coleção
Descrição	Ação: Visualizar Objeto: Itens de uma coleção Atributos: lista de itens da coleção selecionada, critérios de ordenação (nome A–Z, nome Z–A, data de criação crescente, data de criação decrescente, classificação crescente, classificação decrescente), critérios de filtro (conteúdo dos Elementos do layout efetivo de cada item). Exemplos: Camila abre a coleção ‘Lidos’ da categoria ‘Livros’ e visualiza todos os 30 itens ordenados por data de criação decrescente. Camila ordena por classificação decrescente e filtra por gênero ‘Ficção Científica’, vendo apenas os livros desse gênero do melhor ao pior classificado. Camila visualiza a coleção ‘Favoritos’ usando o mesmo filtro; o sistema mantém a preferência de ordenação, mas os filtros são independentes por coleção. Restrições: 1. A visualização deve exibir todos os itens da coleção selecionada. 2. A ordenação padrão é por data de criação decrescente (mais recentes primeiro). 3. Os critérios de filtro disponíveis derivam dos Elementos presentes no layout efetivo dos itens, isto é, o layout da coleção ou, havendo personalização individual, o layout próprio do item (RF-17). 4. A ordenação aplicada deve ser persistida por usuário e válida para todas as coleções até ser alterada. 5. Os filtros aplicados são mantidos durante a navegação dentro da mesma coleção e reiniciados ao sair dela. 6. A tag não é critério de filtro nesta tela. A tag pertence à coleção (RF-07), e não ao item: como todos os itens de uma mesma coleção compartilhariam as mesmas tags, o filtro não separaria nada. A busca por tag reúne coleções e está descrita no RF-07.
Prioridade	Essencial
Operação	Saída
Ator	Usuário

Fonte: Elaborada pelo autor

Quadro 20 – RF-13 – Exibir Tela Principal de Categorias

Código	RF-13
Grupo	Coleção
Descrição	Ação: Exibir Objeto: Tela principal (dashboard) de categorias do usuário Atributos: lista de categorias do usuário autenticado (nome, ícone, imagem de capa, cor de fundo do cartão, contagem total de coleções, contagem total de itens somados entre todas as coleções), modo de exibição (grade ou lista), critérios de ordenação (nome A–Z, nome Z–A, data de criação crescente, data de criação decrescente), critérios de filtro (tipo/tema da categoria), atalhos de navegação para coleções e itens. Exemplos: Ao fazer login, Camila vê na tela inicial os cartões ‘Livros (2 coleções, 20 itens)’, ‘Filmes (2 coleções, 30 itens)’ e ‘Cafés (1 coleção, 7 itens)’ em modo grade. Camila clica em ‘Livros’ e visualiza as coleções ‘Lidos (20, pública)’, ‘Para Ler (30, somente amigos)’ e ‘Diário (2, privada)’, podendo navegar para os itens de cada uma. Camila alterna para o modo lista e ordena as categorias por data de criação decrescente; a preferência é mantida na próxima sessão. Camila, em seu primeiro acesso, vê mensagem de boas-vindas e botão ‘Criar primeira categoria’. Restrições: 1. A tela deve exibir todas as categorias do usuário autenticado em uma única tela dedicada. 2. Cada categoria deve exibir nome, ícone e as contagens de coleções e de itens. Em modo grade, a imagem de capa (RF-06) ocupa o topo do cartão, com o nome sobre ela. 3. A categoria não possui nível de privacidade próprio; o indicador individual de privacidade aparece no <i>card</i> de cada coleção dentro da categoria, e não no <i>card</i> da categoria (RF-08). 4. A navegação até as coleções de uma categoria deve ser acessível com um único clique ou toque; a navegação até um item, com no máximo três. 5. Usuários sem categorias devem ver mensagem de boas-vindas com atalho para criação da primeira categoria. 6. Ao acessar uma categoria vazia (sem coleções), o sistema deve exibir mensagem informativa e atalho para criação da primeira coleção. 7. O usuário deve poder alternar entre os modos de visualização ‘grade’ e ‘lista’ a qualquer momento. 8. O modo de visualização padrão é ‘grade’; a escolha do usuário deve ser persistida entre sessões. 9. O usuário deve poder ordenar as categorias por pelo menos quatro critérios: nome crescente, nome decrescente, data de criação crescente e data de criação decrescente. 10. A ordenação padrão é por data de criação decrescente (mais recentes primeiro). 11. Os filtros e a ordenação aplicados devem ser persistidos entre sessões do mesmo usuário. 12. O filtro por tema opera sobre o texto livre definido pelo usuário.

Quadro 21 – RF-14 – Gerenciar Layout Personalizado

Código	RF-14
Grupo	Editor de Layout
Descrição	Ação: Gerenciar e duplicar Objeto: Layout personalizado (template) Atributos: nome do layout (100 caracteres), fundo da ficha (uma cor hexadecimal, ou um degradê de duas cores com um ângulo e um ponto de início), Elementos disponíveis (conforme os tipos fixos definidos pela enumeração TipoElemento: texto, imagem, forma, ícone, classificação e data), conteúdo textual opcional do Elemento, EstiloDeElemento de cada Elemento (fonte, tamanho, peso, itálico, sublinhado, riscado, alinhamento horizontal e vertical, nível de título, entrelinha, espaçamento entre letras, cor, cor de fundo, arredondamento, opacidade, rotação, borda (cor, espessura e traço), forma do elemento, curva e ponta da linha, ajuste de imagem, apresentação em lista, nota máxima e ícone da classificação, formato e estética da data), camada de sobreposição, posição e tamanho de cada Elemento na área de edição, layout de origem (referência opcional, preenchida apenas na operação de duplicação). Exemplos: Camila cria o layout ‘Card de Livro’ com Elemento de Imagem para capa, textos livres como ‘Autor: Frank Herbert’, Classificação de 1 a 5 estrelas, dois Elementos de Data (início e fim de leitura) e área de Texto para notas. Camila reposiciona o Elemento de Classificação via <i>drag-and-drop</i> e visualiza o resultado em tempo real na prévia. Camila seleciona o layout ‘Card de Livro’ e clica em ‘Duplicar’; o sistema cria ‘Card de Livro (cópia)’ com a mesma estrutura, que Camila renomeia para ‘Card de Livro – Edição Especial’ e ajusta sem afetar o original. Camila exclui o layout ‘Card Antigo’ que não está associado a nenhuma coleção. Restrições: 1. O nome do layout é obrigatório e deve ser único por usuário. 2. A biblioteca de layouts do usuário reúne três origens: os layouts que ele mesmo cria, os layouts padrão oferecidos pelo sistema desde o cadastro (RF-01) e os layouts importados de outros usuários (RF-16). Todos podem ser igualmente editados, duplicados e associados a coleções. 3. Pelo menos um Elemento deve ser adicionado para que o layout possa ser salvo. 4. Os tipos de Elemento disponíveis para inclusão são os definidos pela enumeração TipoElemento (texto, imagem, forma, ícone, classificação e data), à semelhança dos elementos fixos oferecidos por editores visuais como o Canva. 5. Elementos podem armazenar conteúdo visual próprio, como ‘Autor:’ ou ‘Autor: Frank Herbert’, além de propriedades de fonte, cor e alinhamento. 6. Elementos podem ser reposicionados e redimensionados via arrastar e soltar (<i>drag-and-drop</i>). 7. O fundo da ficha é escolhido pelo usuário, como cor única ou como degradê entre duas cores, e vale para o layout inteiro, e não para um Elemento em particular.

Quadro 22 – RF-15 – Associar Layout a Coleção

Código	RF-15
Grupo	Editor de Layout
Descrição	Ação: Associar Objeto: Layout (template) a uma coleção Atributos: coleção de destino (existente, pertencente ao usuário), layout selecionado na biblioteca do usuário (criado por ele, padrão do sistema ou importado de outro usuário, podendo já estar associado a outras coleções). Exemplos: Camila associa o layout ‘Card de Livro’ à coleção ‘Lidos’; todos os itens dessa coleção passam a usar esse layout. Camila associa o mesmo layout ‘Card de Livro’ também à coleção ‘Para Ler’; as duas coleções reutilizam o mesmo template, de forma independente. Camila associa à coleção ‘Mangás’ um layout padrão oferecido pelo sistema, por não desejar compor a ficha do zero. Camila troca o layout da coleção ‘Mangás’ para ‘Card de Mangá’; os novos itens passam a usar o novo layout, e os itens já existentes mantêm seus dados (RF-17). Restrições: 1. Um layout só pode ser associado a coleções existentes e pertencentes ao mesmo usuário. 2. O layout é associado no nível da coleção; a categoria não recebe layout (RF-06). 3. A associação é opcional. O usuário pode escolher qualquer layout de sua biblioteca, seja criado por ele, padrão do sistema ou importado de outro usuário (RF-14 e RF-16); nenhum layout é aplicado automaticamente. 4. Uma coleção pode ter, no máximo, um layout ativo por vez. 5. Um mesmo layout pode ser associado a várias coleções simultaneamente, sendo reutilizado como template. 6. O layout aplicável a um item é o da sua coleção; na ausência de layout associado, a ficha do item nasce vazia e ele pode receber um layout próprio, montado ali mesmo. Modificações individuais por item são tratadas como <i>override</i> (RF-17). 7. A troca ou remoção do layout da coleção não altera os dados nem os Elementos individuais (<i>override</i>) dos itens já cadastrados (RF-17). 8. Ao associar um layout, o sistema deve exibir prévia de como os novos itens serão criados. 9. O usuário pode remover a associação de layout da coleção a qualquer momento.
Prioridade	Importante
Operação	Entrada
Ator	Usuário

Fonte: Elaborada pelo autor

Quadro 23 – RF-16 – Compartilhar Layout

Código	RF-16
Grupo	Editor de Layout
Descrição	Ação: Compartilhar e importar Objeto: Layout (template) compartilhado entre usuários Atributos: layout de origem (pertencente ao usuário que compartilha), identificador do layout compartilhado (utilizado em link ou em requisição de importação), usuário importador, layout importado (cópia gerada na biblioteca do importador), finalidade interna do layout (<i>template</i> importado), que registra que a cópia nasceu de uma importação. Exemplos: Camila cria o layout ‘Card de Livro’ e envia para uma amiga um link contendo o identificador desse layout. A amiga importa o layout pela biblioteca de layouts; o sistema cria uma cópia na conta dela, que pode ser usada e editada livremente como se tivesse sido criada por ela. Camila altera o layout original depois do compartilhamento; as cópias já importadas por outros usuários permanecem intactas. Restrições: 1. O compartilhamento simples usa o próprio identificador do layout em link ou em requisição de importação; não há código separado, ativação nem cancelamento de compartilhamento neste fluxo. 2. A importação cria uma cópia independente do layout na biblioteca do usuário importador; alterações na cópia não afetam o layout de origem, e vice-versa. 3. Layouts importados devem aparecer na biblioteca do usuário como <i>templates</i> utilizáveis, mas ficam marcados internamente com a finalidade de <i>template</i> importado. 4. O compartilhamento abrange apenas a estrutura visual do layout, não os itens do dono nem as personalizações individuais feitas em itens. 5. A importação está disponível a partir do identificador ou do link de compartilhamento, independentemente de haver vínculo de amizade entre os usuários. 6. O usuário não deve importar um layout da própria biblioteca por este fluxo; para isso deve usar a duplicação (RF-14).
Prioridade	Desejável
Operação	Entrada
Ator	Usuário

Fonte: Elaborada pelo autor

Quadro 24 – RF-17 – Modificar Layout de Item Individualmente

Código	RF-17
Grupo	Editor de Layout
Descrição	Ação: Modificar individualmente Objeto: Layout herdado em um item específico Atributos: Elementos herdados do layout aplicável ao item (base de partida), conteúdo e estilo dos Elementos, Elementos adicionados pelo usuário no item (<i>override</i>), Elementos removidos pelo usuário no item (<i>override</i>), Elementos reposicionados ou redimensionados no item (<i>override</i>). Exemplos: Camila cria o item ‘Harry Potter e a Pedra Filosofal’ na coleção ‘Lidos’ (categoria ‘Livros’); o item abre com o layout aplicável herdado; Camila adiciona Elemento extra de texto ‘Data de Início de Leitura’ apenas nesse item. Camila remove o Elemento de Forma de um item específico pois não se aplica, sem afetar o layout da coleção nem os outros itens. Depois da personalização, o item mantém seu próprio layout, sem alterar o layout usado pela coleção. Restrições: 1. Todo item, ao ser criado, adota o layout da sua coleção como ponto de partida (RF-09 e RF-15). Caso a coleção não possua layout, o item começa sem layout, e o usuário pode montá-lo a partir da própria ficha. 2. O usuário pode adicionar, remover ou reposicionar Elementos no item sem afetar o layout da coleção ou os demais itens. 3. O layout aplicável deve ser exibido como referência visual durante a edição individual do item. 4. As modificações individuais são exclusivas do item; nenhuma alteração feita no item reflete no layout da coleção. 5. Ao iniciar a edição visual individual, o sistema deve criar uma cópia própria do layout da coleção para aquele item, preservando Elementos, conteúdo e estilos como ponto de partida. Não havendo layout na coleção, a cópia é criada vazia: a existência de um layout na coleção não é pré-requisito para personalizar a ficha de um item. 6. Após a criação da cópia individual, alterações de conteúdo, estilo, posição, tamanho, adição ou remoção de Elementos devem afetar apenas aquele item, sem modificar o layout da coleção nem os demais itens. 7. Depois de personalizado, o item mantém seu layout próprio, mesmo que o layout da coleção seja alterado posteriormente.
Prioridade	Desejável
Operação	Entrada
Ator	Usuário

Fonte: Elaborada pelo autor

Quadro 25 – RF-18 – Gerenciar Aparência (Tema e Cores)

Código	RF-18
Grupo	Aparência
Descrição	Ação: Gerenciar aparência Objeto: Paletas de aparência do usuário e configuração visual global da interface Atributos: cor primária (hex), subcores (12 valores hex: fundo da página, cartões, títulos, barra lateral, cabeçalho, modais e um texto para cada superfície, isto é, texto no fundo, nos cartões, na barra lateral, no cabeçalho, nos modais e dentro dos botões de destaque), paletas predefinidas (lista do sistema), paletas personalizadas (nome de até 50 caracteres), paleta em uso, estilo global (claro / escuro / automático conforme sistema operacional). Exemplos: No momento do cadastro (RF-01), o sistema disponibiliza ao usuário seis paletas predefinidas ('Claro', 'Escuro', 'Outono', 'Oceano', 'Floresta' e 'Lavanda'); como o sistema operacional de Camila está no tema escuro, a paleta 'Escuro' já nasce em uso. Camila seleciona a paleta predefinida 'Oceano', de tons azulados e estilo escuro; toda a interface é atualizada imediatamente. Camila cria a paleta personalizada 'Meu Rosa' a partir da paleta em uso, altera a cor primária para #E91E8C, confere o resultado na área de prévia e salva. Com a paleta clara 'Meu Rosa' em uso, Camila muda o estilo global para escuro; as cores da paleta passam a ser exibidas com a claridade invertida, preservando os tons. Restrições: 1. Todo usuário recém-cadastrado recebe as paletas predefinidas do sistema (RF-01). Nasce em uso a paleta 'Escuro' se o sistema operacional estiver no tema escuro no momento do cadastro, e a paleta 'Claro' nos demais casos; mudanças posteriores no sistema operacional não trocam a paleta em uso. 2. O usuário pode criar, editar, duplicar, renomear e excluir paletas, e selecionar a paleta em uso a qualquer momento. As paletas predefinidas pertencem ao usuário e podem ser editadas e excluídas como as personalizadas. O nome de uma paleta não pode se repetir entre as paletas do mesmo usuário, mas pode coincidir com o de paletas de outros usuários. 3. A paleta em uso deve ser aplicada globalmente a todas as telas do sistema de forma imediata. 4. A paleta em uso não pode ser excluída, o que garante que o usuário tenha sempre ao menos uma paleta. 5. Uma paleta é sempre completa: a cor primária e todas as subcores devem estar preenchidas. Cada superfície (fundo da página, cartões, barra lateral, cabeçalho e modais) tem sua própria cor de texto, escolhida pelo usuário; o sistema exibe exatamente a cor escolhida, sem substituí-la. 6. O sistema classifica cada paleta como clara ou escura a partir da cor de fundo. Quando o estilo global em vigor é o oposto ao da paleta em uso, as cores são exibidas com a claridade invertida, preservando os tons. 7. As paletas e a paleta em uso devem ser salvas no perfil e persis-

Quadro 26 – RF-19 – Gerenciar Tipografia Global

Código	RF-19
Grupo	Aparência
Descrição	Ação: Alterar Objeto: Tipografia e aparência textual global Atributos: família de fonte (lista predefinida com no mínimo 5 opções), tamanho base em px (mínimo 12 px, máximo 24 px), tamanho de título em px (mínimo 16 px, máximo 48 px), espaço entre linhas (1.0 a 2.0), espaço entre letras, peso da fonte (300 a 800), itálico. Exemplos: Camila edita a paleta ‘Meu Rosa’ e altera a fonte para ‘Georgia’, o tamanho base para 16 px e o espaço de linha para 1.6; a prévia reflete as mudanças antes da confirmação. Restrições: 1. As fontes disponíveis devem incluir ao menos cinco opções tipográficas distintas (ex.: serif, sans-serif, monospace). 2. As alterações devem ser exibidas em área de prévia antes de o usuário confirmar a aplicação. 3. A tipografia faz parte de cada paleta (RF-18): é aplicada globalmente junto com a paleta em uso e persistida entre sessões e dispositivos.
Prioridade	Desejável
Operação	Entrada
Ator	Usuário

Fonte: Elaborada pelo autor

Quadro 27 – RF-20 – Gerenciar Amizade

Código	RF-20
Grupo	Interação Social
Descrição	Ação: Adicionar, aceitar e visualizar Objeto: Amizade entre usuários Atributos: usuário solicitante (nome, código de usuário e email), usuário receptor (nome, código de usuário e email), critério de busca (código de usuário ou nome), status da solicitação (pendente / aceita / recusada / cancelada), data da solicitação e data da resposta. Exemplos: Camila informa o código de usuário ‘joao#1203’ no campo de busca e envia a solicitação de amizade; João aceita e ambos aparecem mutuamente na lista de amigos. Camila busca por ‘João Silva’ pelo nome e o sistema exibe a lista de usuários correspondentes; Camila seleciona o usuário correto e envia a solicitação. Camila recusa a solicitação de um usuário desconhecido; o status é atualizado para ‘recusada’ sem gerar notificação ao solicitante. Restrições: 1. A busca por usuários pode ser realizada tanto pelo código de usuário (busca exata) quanto pelo nome (busca parcial). 2. A busca por código de usuário retorna no máximo um resultado, pois o código é único. 3. A busca por nome pode retornar múltiplos resultados; o sistema exibe nome, foto e código de cada usuário para desambiguação. 4. Não é possível enviar solicitação de amizade para si mesmo. 5. Uma solicitação pendente não pode ser reenviada enquanto aguarda resposta. 6. O remetente pode cancelar uma solicitação ainda pendente, que passa ao status ‘cancelada’ e deixa de aparecer para o destinatário. 7. A lista de amigos exibe apenas amizades com status ‘aceita’. 8. O usuário pode desfazer uma amizade aceita a qualquer momento, removendo o vínculo para ambos os lados. 9. Envio, aceite e desfazer amizade devem gerar notificação ao usuário envolvido, conforme RF-24; a recusa é silenciosa.
Prioridade	Importante
Operação	Entrada
Ator	Usuário

Fonte: Elaborada pelo autor

Quadro 28 – RF-21 – Seguir Coleção Visível

Código	RF-21
Grupo	Interação Social
Descrição	Ação: Seguir Objeto: Coleção visível de outro usuário Atributos: coleção seguida (com privacidade pública ou ‘somente amigos’), categoria pai da coleção (referência), usuário seguidor, data de início do acompanhamento. Exemplos: Camila visita o perfil de ‘João Silva’, com quem não possui amizade, vê a coleção pública ‘Filmes Assistidos’ (dentro da categoria ‘Filmes’) e clica em ‘Seguir’; as atualizações dessa coleção passam a aparecer no feed de Camila, mas as demais coleções da categoria ‘Filmes’ (como ‘Quero Ver’) não, a menos que sejam seguidas individualmente. Após João aceitar a amizade de Camila, ela também passa a visualizar e pode seguir a coleção ‘Favoritos’, configurada como ‘somente amigos’. Camila tenta acessar uma coleção privada de João e não encontra a opção de acompanhamento, pois a coleção sequer aparece no perfil visitado. Restrições: 1. Qualquer usuário autenticado pode seguir uma coleção ‘pública’, independentemente de amizade com o proprietário. 2. Coleções ‘somente amigos’ podem ser seguidas apenas enquanto existir vínculo de amizade confirmado (RF-20). 3. Coleções ‘privadas’ não aparecem e não podem ser seguidas por outros usuários, nem mesmo por amigos. 4. O proprietário não pode seguir a própria coleção. 5. O acompanhamento ocorre no nível da coleção: o usuário pode seguir múltiplas coleções da mesma categoria, de forma independente, sem que isso implique seguir as demais. 6. O usuário pode deixar de seguir uma coleção a qualquer momento, sem restrições. 7. Se a coleção seguida for alterada para ‘privada’, todos os acompanhamentos devem ser encerrados automaticamente (RF-08). 8. Se uma coleção ‘pública’ for alterada para ‘somente amigos’, acompanhamentos de usuários sem amizade confirmada devem ser encerrados; os de amigos permanecem ativos. 9. Ao desfazer uma amizade, acompanhamentos de coleções ‘públicas’ permanecem ativos e acompanhamentos de coleções ‘somente amigos’ entre os dois usuários são encerrados.
Prioridade	Importante
Operação	Entrada
Ator	Usuário

Fonte: Elaborada pelo autor

Quadro 29 – RF-22 – Exibir Feed de Atualizações

Código	RF-22
Grupo	Interação Social
Descrição	Ação: Exibir Objeto: Feed de atualizações das coleções seguidas Atributos: tipo de evento (adição, edição ou remoção de item em coleção seguida), item afetado ou registro mínimo do item quando ele já tiver sido removido, coleção seguida, categoria pai, usuário autor da ação, data e hora do evento e descrição textual auxiliar. Exemplos: Feed de Camila exibe ‘João adicionou Oppenheimer na coleção Filmes Assistidos (categoria Filmes) – há 2 horas’. Feed exibe ‘Ana avaliou o item Breaking Bad na coleção Séries Concluídas – há 5 minutos’. Camila rola o feed e carrega eventos mais antigos progressivamente. Restrições: 1. O feed deve exibir apenas eventos de coleções que o usuário segue ativamente (RF-21). 2. Os eventos devem ser ordenados cronologicamente, do mais recente ao mais antigo. 3. O feed deve ser atualizado automaticamente em tempo real, sem necessidade de recarregar a página. 4. Se a privacidade de uma coleção seguida for alterada para ‘privada’, os eventos anteriores dessa coleção não devem mais ser exibidos, e o acompanhamento é encerrado automaticamente (RF-21). 5. Eventos de criação ou exclusão de coleção em si não são exibidos no feed; a descoberta de novas coleções ocorre por meio da visita a perfis de usuários (RF-23). 6. O feed deve suportar carregamento progressivo (paginação) para históricos longos. 7. O registro de evento deve usar campos estruturados (coleção, item quando aplicável e usuário autor) como fonte principal dos dados; a descrição textual é apenas informação auxiliar. 8. Eventos de remoção de item devem preservar um registro mínimo do item afetado, de modo que o histórico continue compreensível após a exclusão do item.
Prioridade	Essencial
Operação	Saída
Ator	Sistema

Fonte: Elaborada pelo autor

Quadro 30 – RF-23 – Visualizar Perfis e Coleções de Usuários

Código	RF-23
Grupo	Interação Social
Descrição	Ação: Visualizar Objeto: Perfis, categorias, coleções acessíveis e itens de outros usuários Atributos: perfil do usuário visitado (nome, foto, bio), coleções acessíveis (com privacidade ‘pública’ ou ‘somente amigos’), categorias agrupadoras dessas coleções, itens pertencentes a cada coleção visível. Exemplos: Camila visita o perfil de ‘João Silva’ e visualiza, agrupadas pelas respectivas categorias, as coleções públicas e somente para amigos de João – por exemplo, ‘Filmes / Assistidos’ (pública), ‘Filmes / Favoritos’ (somente amigos) e ‘Séries / Concluídas’ (pública); dentro de cada coleção, vê os itens cadastrados. Camila não encontra a coleção ‘Diário Pessoal’ de João (categoria ‘Diários’), pois ela é privada; como é a única coleção da categoria, a categoria ‘Diários’ inteira não aparece para Camila. A categoria ‘Jogos’ de João aparece para Camila apenas com as coleções não-privadas; coleções privadas dessa mesma categoria continuam ocultas. Restrições: 1. Apenas coleções com privacidade ‘pública’ ou ‘somente amigos’ são exibidas no perfil visitado, respeitando o vínculo de amizade entre os usuários (coleções ‘somente amigos’ só aparecem para amigos confirmados). 2. Coleções ‘privadas’ e seus itens não devem ser listados nem acessíveis, mesmo que o amigo compartilhe um link direto. 3. Categorias servem apenas como agrupadores visuais no perfil visitado e não possuem privacidade própria. 4. Categorias que, para o visitante, não tenham nenhuma coleção visível devem ser ocultadas integralmente do perfil, de modo que o visitante não tenha conhecimento de sua existência. 5. Todos os itens dentro de uma coleção visível são acessíveis ao visitante; não há privacidade individual por item. 6. A navegação nas categorias, coleções e itens de outros usuários é somente leitura; não é possível editar ou excluir dados.
Prioridade	Importante
Operação	Saída
Ator	Usuário

Fonte: Elaborada pelo autor

4.2.2 Requisitos Não-Funcionais

Segundo Sommerville (2011), os requisitos não funcionais são requisitos que não estão diretamente relacionados com os serviços específicos oferecidos pelo sistema a seus usuários. Logo, não dizem respeito diretamente às funcionalidades fornecidas pelo sistema, mas sim às suas propriedades e restrições de qualidade. Pressman e Maxim (2016), dizem que esses requisitos especificam atributos como desempenho, usabilidade, segurança, confiabilidade e portabilidade, sendo fundamentais para garantir que o sistema atenda aos padrões de qualidade esperados. Assim, esta seção apresenta os requisitos não-funcionais identificados neste trabalho, consolidados nos Quadros de 32 a 41.

Quadro 31 – RF-24 – Gerenciar Notificações do Sistema

Código	RF-24
Grupo	Notificação
Descrição	Ação: Gerar, exibir e gerenciar Objeto: Notificações do sistema Atributos: tipo de notificação (solicitação de amizade recebida, solicitação de amizade aceita, amizade desfeita, atualização em coleção seguida, verificação de e-mail, recuperação de senha), usuário destinatário, usuário autor (quando aplicável), mensagem (texto descritivo gerado pelo sistema), evento estruturado vinculado quando for atualização de coleção seguida (com coleção, categoria, item e registro mínimo), data e hora de geração, status (não lida / lida), data e hora de leitura quando aplicável, canal de entrega (in-app, sempre; e-mail, para verificação de e-mail e recuperação de senha). Exemplos: João envia solicitação de amizade para Camila; o sistema gera notificação in-app para Camila com o texto ‘João Silva (joao#1203) enviou uma solicitação de amizade’ e exibe indicador visual de notificação não lida. Camila aceita a solicitação; o sistema gera notificação in-app para João com o texto ‘Camila Mattos aceitou sua solicitação de amizade’. Camila solicita recuperação de senha; o sistema envia notificação via e-mail com link de redefinição (conforme RF-04). Restrições: 1. Envio, aceite e desfazimento de amizade devem gerar notificação in-app ao usuário envolvido; recusas são silenciosas (RF-20). 2. As notificações in-app devem ser exibidas em uma área dedicada (ex.: ícone de sino no menu principal) com indicador visual de quantidade de notificações não lidas. 3. As notificações devem ser ordenadas cronologicamente, da mais recente à mais antiga. 4. O usuário pode marcar uma notificação como lida, marcar todas como lidas ou limpar sua área de notificações sem excluir os registros do sistema: um campo indica o estado atual de leitura, outro registra quando a leitura ocorreu, e a limpeza apenas oculta as notificações da listagem do usuário. 5. Notificações de verificação de e-mail (RF-05) e de recuperação de senha (RF-04) devem ser enviadas também pelo canal de e-mail, contendo link único e com prazo de expiração. 6. Notificações referentes a coleções cuja privacidade tenha sido alterada para ‘privada’ devem ser ocultadas automaticamente do destinatário (RF-08). 7. As notificações in-app devem ser atualizadas em tempo real, sem necessidade de recarregar a página. 8. Notificações lidas devem ser mantidas por no mínimo 30 dias antes de qualquer limpeza automática.
Prioridade	Importante
Operação	Saída
Ator	Sistema

Quadro 32 – RNF-01 – Usabilidade

Código	RNF-01
Categoria	Usabilidade
Descrição	Descrição: A interface deve ser intuitiva, facilitando o uso do editor de layout e das funcionalidades de personalização visual. Métrica: Usuários iniciantes devem ser capazes de criar uma coleção e adicionar um item em até 5 minutos sem consultar documentação externa. Exemplos: Um novo usuário acessa o sistema pela primeira vez e consegue criar a coleção ‘Meus Filmes’, associar um layout e cadastrar o item ‘Interestelar’ seguindo apenas as indicações visuais da interface. Um usuário utiliza o editor de layout pela primeira vez e compreende como arrastar e posicionar elementos sem necessitar de tutorial. Restrições: 1. Todas as ações principais devem ser acessíveis em no máximo três cliques a partir da tela inicial. 2. Elementos interativos devem possuir rótulos descritivos e tooltips explicativos. 3. Mensagens de erro devem ser claras, indicando o problema e a ação corretiva. 4. O sistema deve utilizar ícones e terminologia consistentes em todas as telas.
Prioridade	Essencial
Operação	Saída

Fonte: Elaborada pelo autor

Quadro 33 – RNF-02 – Desempenho de Carregamento

Código	RNF-02
Categoria	Desempenho
Descrição	Descrição: O sistema deve carregar categorias, coleções e itens em tempo adequado sob condições normais de uso. Métrica: O tempo de carregamento de qualquer tela principal (categorias, coleções, lista de itens) não deve exceder 3 segundos em conexão de banda larga padrão (10 Mbps). Exemplos: Camila acessa sua coleção ‘Livros’ com 42 itens e a listagem completa é exibida em menos de 2 segundos. Camila abre o feed de atualizações com 50 eventos recentes e o carregamento inicial ocorre em menos de 3 segundos. Restrições: 1. Imagens de capa e perfil devem ser otimizadas automaticamente pelo sistema para reduzir o tempo de carregamento. 2. Listas com mais de 20 itens devem utilizar carregamento progressivo (paginação ou scroll infinito). 3. Requisições à API devem retornar resposta em no máximo 2 segundos para operações de leitura. 4. Operações de escrita (cadastro, edição, exclusão) devem ser concluídas em no máximo 3 segundos.
Prioridade	Essencial
Operação	Processamento

Fonte: Elaborada pelo autor

Quadro 34 – RNF-03 – Segurança de Senhas e Dados Pessoais

Código	RNF-03
Categoria	Segurança
Descrição	Descrição: O sistema deve criptografar senhas e proteger informações pessoais dos usuários de acordo com boas práticas de segurança. Métrica: Todas as senhas devem ser armazenadas utilizando algoritmo de hash seguro (bcrypt ou argon2) com salt único por usuário. Exemplos: A senha ‘Camila@2026’ é armazenada como hash irreversível no banco de dados; mesmo com acesso direto à base, não é possível recuperar a senha original. Uma tentativa de login com credenciais inválidas retorna mensagem genérica sem revelar se o erro é no email ou na senha. Restrições: 1. Senhas nunca devem ser armazenadas em texto puro ou com criptografia reversível. 2. Toda comunicação entre cliente e servidor deve utilizar protocolo HTTPS/TLS. 3. Tokens de sessão devem ser gerados com entropia suficiente, ter prazo de validade definido e poder ser revogados a qualquer momento pelo servidor, independentemente da expiração. 4. Dados sensíveis (email, senha) não devem ser expostos em logs do sistema. 5. O sistema deve limitar tentativas de login a 5 por minuto por endereço IP para mitigar ataques de força bruta.
Prioridade	Essencial
Operação	Processamento

Fonte: Elaborada pelo autor

Quadro 35 – RNF-04 – Compatibilidade e Responsividade

Código	RNF-04
Categoria	Compatibilidade
Descrição	Descrição: O sistema deve oferecer compatibilidade com navegadores modernos e dispositivos móveis, mantendo responsividade. Métrica: O sistema deve funcionar corretamente nas duas versões mais recentes dos navegadores Google Chrome, Mozilla Firefox, Safari e Microsoft Edge, além de dispositivos com tela a partir de 320 px de largura. Exemplos: Camila acessa o sistema pelo navegador Chrome no desktop e todas as funcionalidades operam normalmente. Camila acessa o mesmo sistema pelo Safari no iPhone e consegue navegar, criar coleções e adicionar itens com a interface adaptada ao tamanho da tela. Restrições: 1. O layout deve ser responsivo, adaptando-se automaticamente a resoluções de 320 px até 2560 px de largura. 2. Funcionalidades de arrastar e soltar (drag-and-drop) devem possuir alternativa por toque em dispositivos móveis. 3. O editor de layout deve ser funcional tanto em dispositivos desktop quanto em tablets. 4. Nenhuma funcionalidade essencial deve depender de plugins ou extensões de navegador.
Prioridade	Essencial
Operação	Saída

Fonte: Elaborada pelo autor

Quadro 36 – RNF-05 – Personalização Visual sem Quebras de Layout

Código	RNF-05
Categoria	Personalização
Descrição	Descrição: O sistema deve permitir que o usuário personalize temas, cores, subcores e fontes sem comprometer o funcionamento das telas. Métrica: Qualquer combinação de tema e tipografia escolhida pelo usuário deve manter todas as funcionalidades do sistema operacionais, sem quebras de layout ou sobreposição de elementos. Exemplos: Camila aplica a paleta personalizada ‘Meu Rosa’ com fonte ‘Georgia’ 18 px e todas as telas do sistema permanecem funcionais e legíveis. Camila alterna entre os estilos claro e escuro e nenhum texto fica ilegível ou botão fica inacessível. Restrições: 1. As personalizações visuais devem ser aplicadas exclusivamente via variáveis CSS, sem alterar a estrutura HTML ou a lógica do sistema. 2. O sistema deve validar combinações de cores antes de aplicá-las, alertando sobre problemas de contraste. 3. Personalizações devem ser aplicadas em tempo real, sem necessidade de recarregar a página. 4. As configurações devem ser persistidas no perfil do usuário e sincronizadas entre dispositivos.
Prioridade	Importante
Operação	Processamento

Fonte: Elaborada pelo autor

Quadro 37 – RNF-06 – Desempenho na Movimentação de Itens

Código	RNF-06
Categoria	Desempenho
Descrição	Descrição: A movimentação de itens entre coleções deve ocorrer com no máximo três comandos e sem atraso perceptível. Métrica: A operação de mover um item deve ser concluída em no máximo 500 milissegundos desde a ação do usuário até a atualização visual na interface. Exemplos: Camila arrasta o item ‘Duna’ da coleção ‘Para Ler’ para ‘Lidos’ e a mudança é refletida instantaneamente na tela, sem indicador de carregamento. Camila move ‘O Hobbit’ para outra coleção via menu de contexto (selecionar destino e confirmar) e a contagem de itens é atualizada imediatamente em ambas as coleções. Restrições: 1. A interface deve aplicar a movimentação de forma otimista (atualizar visualmente antes da confirmação do servidor). 2. Em caso de falha na comunicação com o servidor, o sistema deve reverter a movimentação e informar o usuário. 3. A operação de desfazer (Ctrl+Z) deve estar disponível por até 30 segundos após a movimentação, conforme RF-10. 4. As contagens de itens nas coleções de origem e destino devem ser recalculadas imediatamente.
Prioridade	Importante
Operação	Processamento

Fonte: Elaborada pelo autor

Quadro 38 – RNF-07 – Desempenho do Editor de Layout

Código	RNF-07
Categoria	Desempenho
Descrição	Descrição: O editor de layout deve refletir alterações de elementos em tempo real, sem travamentos ou recarregamentos completos da página. Métrica: Qualquer operação no editor (adicionar, remover, reposicionar ou redimensionar elemento) deve ser refletida na prévia em no máximo 200 milissegundos. Exemplos: Camila arrasta um elemento de Imagem para uma nova posição no editor e a prévia atualiza instantaneamente sem piscar ou travar. Camila adiciona cinco elementos consecutivos ao layout e o editor mantém fluidez durante todas as operações. Restrições: 1. O editor deve utilizar renderização parcial, atualizando apenas os componentes afetados pela alteração. 2. O salvamento do layout deve ocorrer de forma assíncrona, sem bloquear a interação do usuário com o editor. 3. A prévia deve manter fidelidade visual com o resultado final exibido na ficha do item. 4. O editor deve suportar pelo menos 15 elementos simultâneos sem degradação perceptível de desempenho.
Prioridade	Essencial
Operação	Processamento

Fonte: Elaborada pelo autor

Quadro 39 – RNF-08 – Disponibilidade do Sistema

Código	RNF-08
Categoria	Disponibilidade
Descrição	Descrição: O sistema deve manter alta disponibilidade para que atualizações de amigos e sincronização de dados ocorram continuamente. Métrica: O sistema deve garantir disponibilidade mínima de 99% do tempo (uptime), excluindo janelas de manutenção programada comunicadas com antecedência mínima de 24 horas. Exemplos: Camila acessa o sistema às 23h de um domingo e consegue visualizar o feed de atualizações dos amigos normalmente. Durante uma manutenção programada, o sistema exibe página informativa com previsão de retorno, comunicada previamente por email. Restrições: 1. O sistema deve possuir mecanismo de recuperação automática em caso de falha de serviço. 2. Manutenções programadas devem ser realizadas em horários de menor uso (madrugada) e comunicadas com antecedência. 3. O feed de atualizações deve funcionar com conexão intermitente, exibindo dados em cache quando offline. 4. Dados não sincronizados devem ser enviados automaticamente quando a conexão for restabelecida.
Prioridade	Importante
Operação	Processamento

Fonte: Elaborada pelo autor

Quadro 40 – RNF-09 – Acessibilidade

Código	RNF-09
Categoria	Acessibilidade
Descrição	Descrição: O sistema deve adotar padrões de acessibilidade, como contraste adequado, textos legíveis e navegação clara. Métrica: O sistema deve atender aos critérios de conformidade WCAG 2.1 nível AA, incluindo relação de contraste mínima de 4,5:1 para texto normal e 3:1 para texto grande. Exemplos: Camila, que possui baixa visão, ativa o estilo de alto contraste e consegue ler todos os textos e identificar todos os botões da interface. Um usuário utiliza o sistema exclusivamente via teclado (navegação por Tab e Enter) e consegue acessar todas as funcionalidades principais. Restrições: 1. Todos os elementos interativos devem ser acessíveis via navegação por teclado. 2. Imagens informativas devem possuir texto alternativo (alt text) descritivo. 3. O sistema deve alertar o usuário ao selecionar combinações de cores que não atendam ao contraste mínimo. 4. Fontes devem possuir tamanho mínimo de 12 px e ser escaláveis sem perda de funcionalidade até 200% de zoom.
Prioridade	Importante
Operação	Saída

Fonte: Elaborada pelo autor

Quadro 41 – RNF-10 – Segurança e Integridade dos Dados

Código	RNF-10
Categoria	Segurança
Descrição	Descrição: O sistema deve armazenar dados em servidor seguro e garantir integridade das coleções criadas pelo usuário. Métrica: O sistema deve realizar backups automáticos diários do banco de dados, com retenção mínima de 30 dias, e garantir que nenhum dado de coleção seja corrompido ou perdido sem ação explícita do usuário. Exemplos: Uma falha no servidor é corrigida e todos os dados de coleções, itens e configurações de Camila são restaurados integralmente a partir do backup mais recente. Camila exclui acidentalmente uma coleção; o sistema solicita confirmação antes de executar e, uma vez confirmada, a exclusão é permanente e registrada em log de auditoria. Restrições: 1. O banco de dados deve ser hospedado em servidor com certificação de segurança e acesso restrito. 2. Backups automáticos devem ser realizados diariamente e armazenados em localização separada do servidor principal. 3. Operações de exclusão devem ser registradas em log de auditoria com identificação do usuário, data e hora. 4. O sistema deve utilizar transações atômicas para garantir que operações de escrita não resultem em dados inconsistentes. 5. Arquivos de mídia (imagens) devem ser armazenados com redundância para evitar perda.
Prioridade	Essencial
Operação	Processamento

Fonte: Elaborada pelo autor

4.2.3 Restrições, Suposições e Dependências

Esta seção apresenta os fatores que podem limitar decisões de desenvolvimento ou influenciar diretamente a implementação dos requisitos definidos.

- O sistema depende de conexão com a internet para sincronizar dados e exibir atualizações de amigos.

- O desempenho do editor de layout está condicionado à capacidade do dispositivo do usuário.
- A troca de dados entre usuários exige um servidor ativo e funcional.
- A expansão das funcionalidades sociais depende da adoção do sistema por múltiplos usuários.

4.2.4 Requisitos Adiados

Alguns requisitos foram identificados durante a elicitação, porém não serão implementados nesta primeira versão do sistema. Eles poderão ser considerados em iterações futuras.

- Implementação de chat privado entre usuários.
- Recomendações automáticas baseadas nas coleções dos amigos.
- Exportação de coleções em PDF ou outros formatos.
- Mecanismos avançados de descoberta pública (explorar, ranking, tendências).

Além desses, dois requisitos especificados neste capítulo foram mantidos na especificação, por integrarem o comportamento pretendido da plataforma, mas não chegaram a ser implementados na etapa concluída até aqui:

- **Verificação de e-mail na alteração de endereço** (RF-05 e RF-24). O envio de mensagens por e-mail já está disponível na plataforma, pois é utilizado na recuperação de senha (RF-04); o fluxo específico de confirmação do novo endereço, contudo, ficou para uma etapa posterior. Até lá, a alteração de e-mail tem efeito imediato.
- **Atualização em tempo real do *feed* e das notificações** (RF-22 e RF-24). As notificações e os eventos de atualização são persistidos e disponibilizados por consulta à API, mas a entrega por fluxo contínuo de eventos, que dispensaria o recarregamento da página, permanece fora desta etapa.

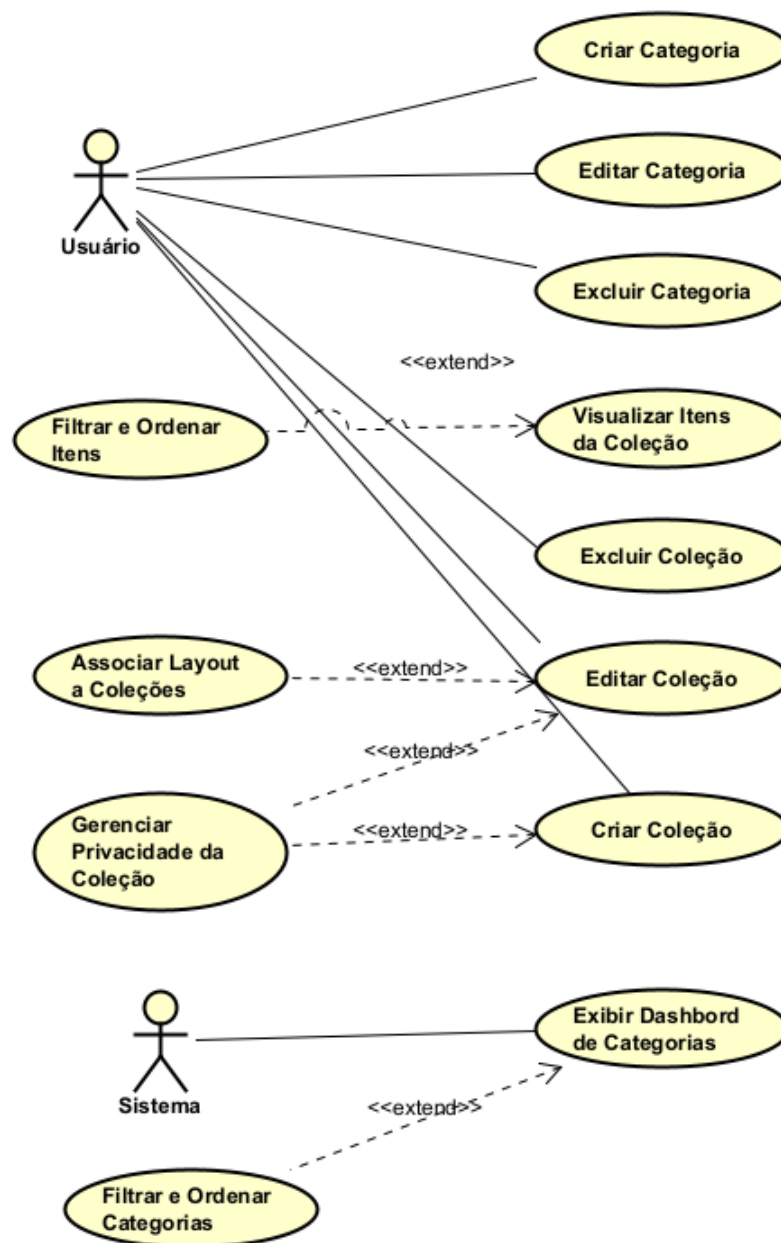
4.3 Casos de Uso

Conforme Valente (2020), um caso de uso descreve a sequência de interações realizadas por um ator com um sistema para atingir um objetivo específico, sendo esse ator, em geral, uma entidade externa, como um usuário humano ou outro sistema. Logo, um caso de uso é uma descrição de um cenário específico de interação entre um ator e o sistema, detalhando as ações realizadas e os resultados esperados. Ele é utilizado para capturar os requisitos funcionais do sistema, fornecendo uma visão clara e estruturada das funcionalidades que o sistema deve oferecer.

4.3.1 Diagrama de Casos de Uso

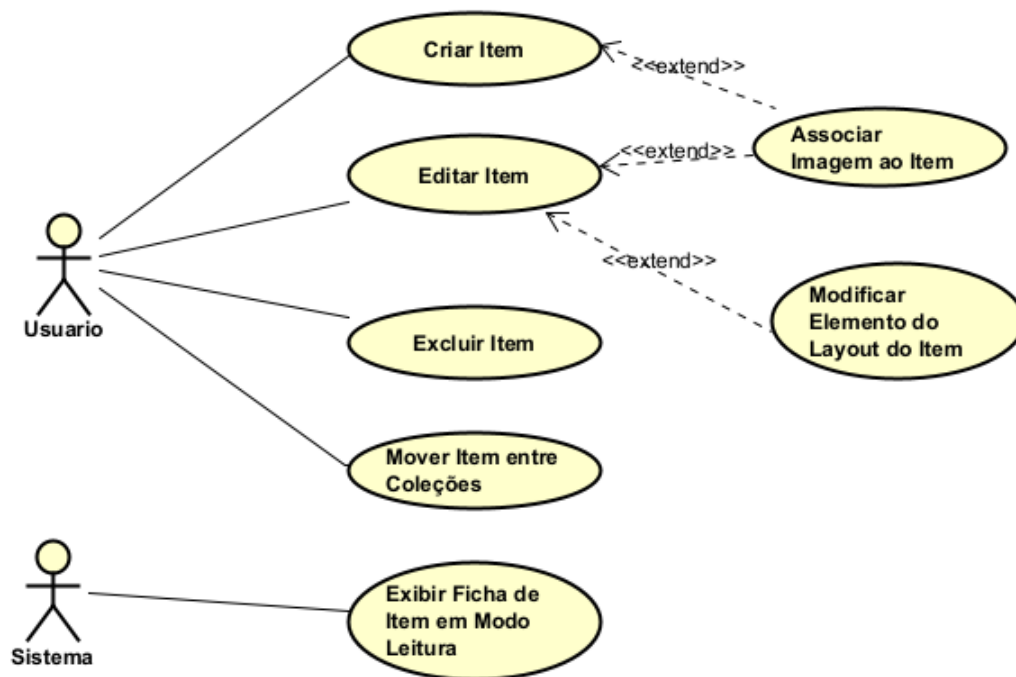
O diagrama de casos de uso elaborado neste trabalho descreve as principais interações do usuário com a plataforma personalizável para criação e compartilhamento de coleções digitais, representando ações como o gerenciamento de categorias, coleções e itens, a montagem de layouts no editor visual, a configuração da aparência da interface e as interações sociais entre usuários. Essas interações são essenciais para garantir que a plataforma atenda às necessidades dos usuários, proporcionando uma experiência eficaz e acessível. Dada a quantidade de casos de uso identificados, o diagrama foi dividido por área funcional e está apresentado nas Figuras de 1 a 7.

Figura 1 – Gerenciamento de Coleções



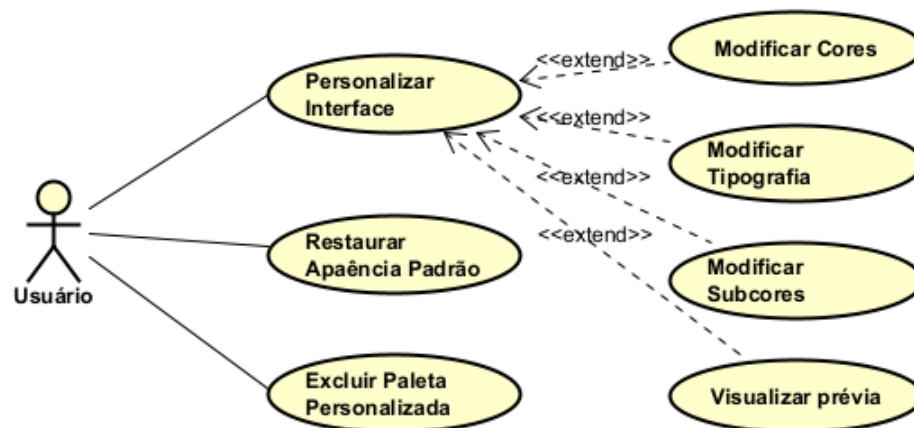
Fonte: Elaborada pelo autor

Figura 2 – Interações Com Itens da Coleção



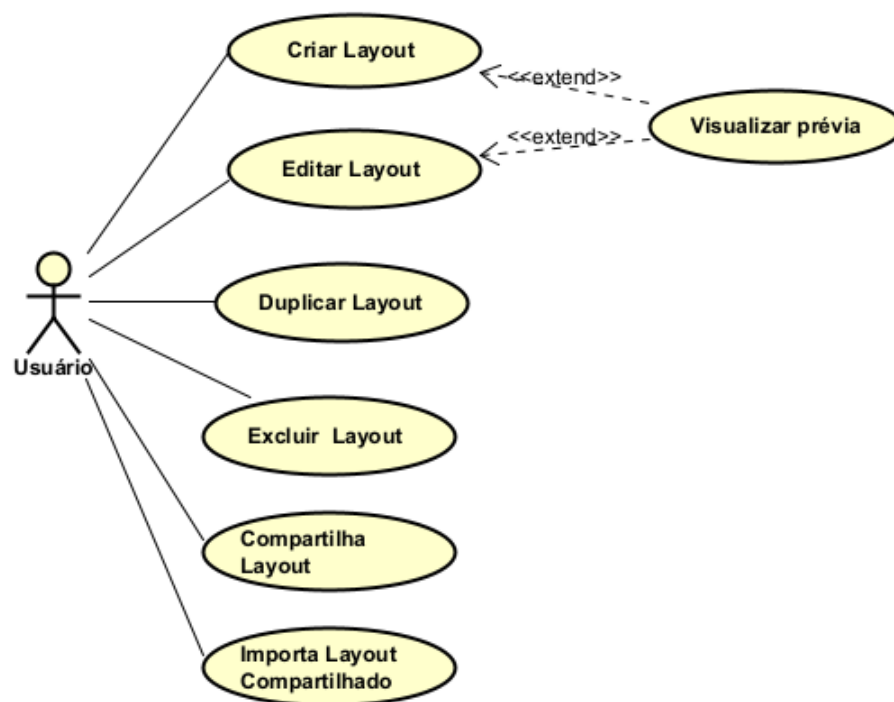
Fonte: Elaborada pelo autor

Figura 3 – Personalização da Plataforma



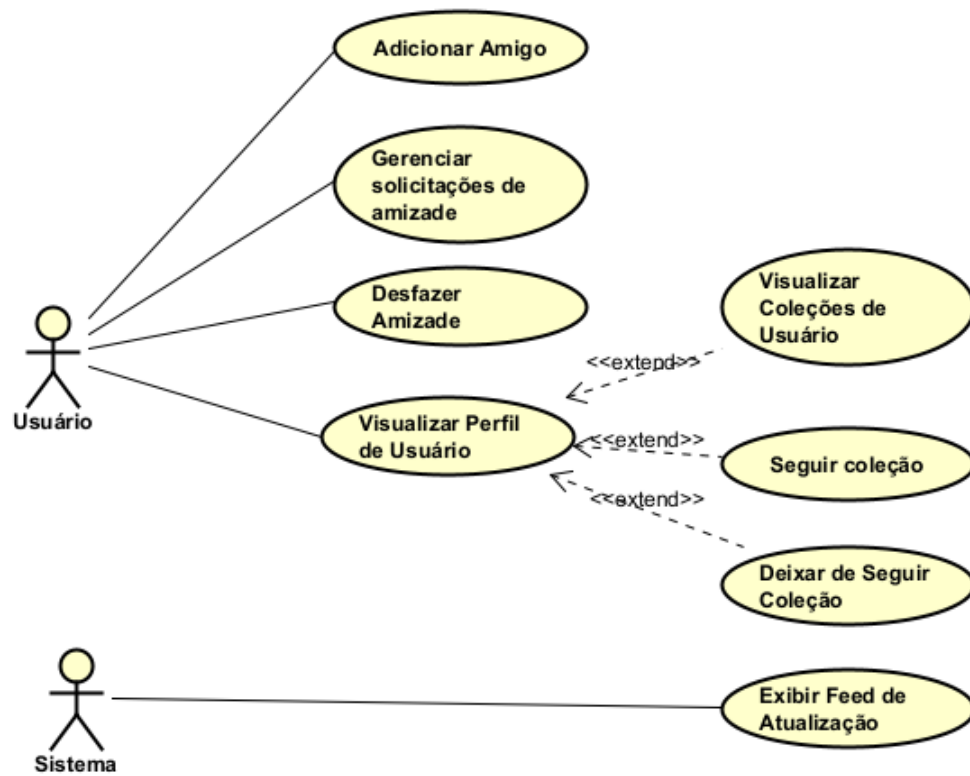
Fonte: Elaborada pelo autor

Figura 4 – Layout e Templates



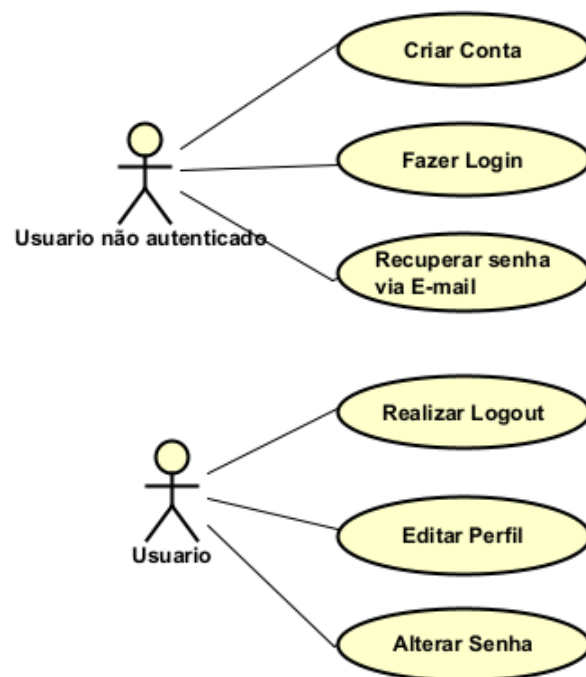
Fonte: Elaborada pelo autor

Figura 5 – Perfil e Social



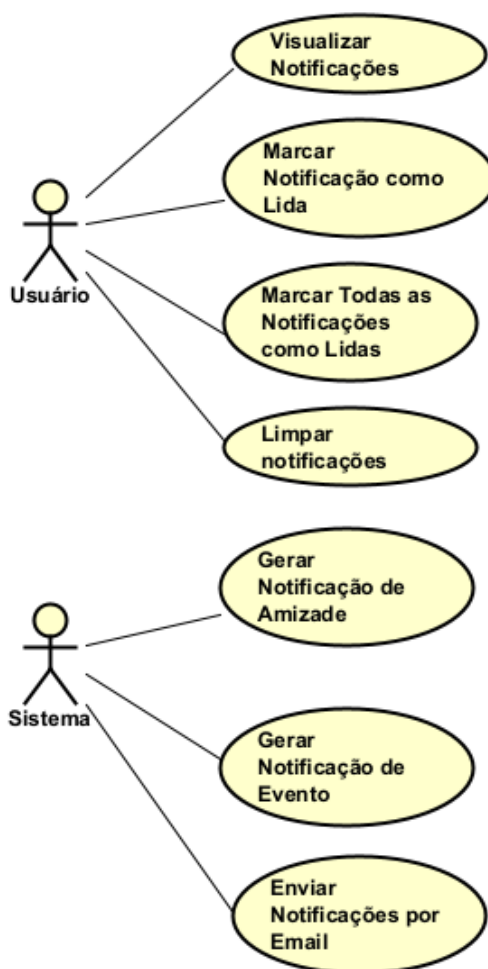
Fonte: Elaborada pelo autor

Figura 6 – Autenticação



Fonte: Elaborada pelo autor

Figura 7 – Notificações



Fonte: Elaborada pelo autor

4.3.2 Especificação dos Casos de Uso

Uma especificação de caso de uso fornece a descrição textual detalhada do comportamento do sistema a partir da perspectiva do usuário, permitindo compreender como as funcionalidades são executadas e quais resultados são esperados. As especificações apresentam os fluxos principais, fluxos alternativos, condições prévias e pós-condições associadas a cada interação.

Neste trabalho, foram elaboradas dezenove especificações textuais, cobrindo os casos de uso que possuem fluxo próprio e alternativas relevantes. Os demais casos

de uso presentes nos diagramas não receberam especificação por representarem variações, detalhamentos ou funcionalidades já incluídas nos fluxos básicos aqui descritos, como é o caso das operações de edição e exclusão, cujo fluxo espelha o de criação com a diferença do registro previamente selecionado. O modelo adotado para a especificação dos casos de uso é apresentado nos Quadros 1 a 19, adaptado da IBM.

Cabe destacar a estrutura hierárquica adotada pelo sistema, que organiza os dados em três níveis: **Categoria** (agrupador temático de alto nível, ex.: “Livros”), **Coleção** (subdivisão de uma categoria onde os itens são efetivamente cadastrados e onde o layout é associado, ex.: “Lidos”, “Para Ler”) e **Item** (unidade individual de conteúdo, ex.: o livro “Duna”). Essa separação permite que o usuário organize seu acervo em duas camadas antes de cadastrar qualquer item, facilitando a categorização e a personalização por meio do layout definido em cada coleção.

A seguir, são apresentadas as especificações detalhadas dos casos de uso principais identificados no sistema.

4.3.2.1 Fazer Login

Quadro 1 – Especificação do Caso de Uso: Fazer Login

Item	Descrição
Nome do Caso de Uso	Fazer Login
Descrição Resumida	Este caso de uso permite que o usuário já cadastrado se autentique no sistema informando seu e-mail e senha, obtendo acesso às funcionalidades restritas da plataforma por meio de um token de sessão gerado automaticamente (RF-02).

Continua na próxima página

Quadro 1 – Continuação da página anterior

Item	Descrição
Fluxo Básico	FB01: Autenticação de usuário 1. O usuário acessa a tela de login do sistema. 2. O usuário insere seu e-mail e senha. 3. O usuário confirma clicando em “Entrar”. 4. O sistema valida o formato do e-mail inserido [include: Validar Formato de E-mail]. 5. O sistema verifica se o e-mail está cadastrado na base de dados. 6. O sistema compara a senha informada com o hash armazenado. 7. O sistema gera um token de sessão [include: Gerar Token de Sessão]. 8. O sistema registra a sessão ativa do usuário. 9. O sistema redireciona o usuário para o dashboard de categorias.

Continua na próxima página

Quadro 1 – Continuação da página anterior

Item	Descrição
Fluxos Alternativos	FA01: Credenciais inválidas 1. Nos passos 5 ou 6 do FB01, o sistema identifica que o e-mail não está cadastrado ou a senha não corresponde ao hash armazenado. 2. O sistema exibe a mensagem genérica: “Credenciais de login inválidas”, sem especificar qual campo está incorreto. 3. O sistema retorna ao passo 2 do FB01. FA02: Campos não preenchidos 1. No passo 3 do FB01, o sistema identifica que um ou ambos os campos estão vazios. 2. O sistema exibe mensagem de erro indicando os campos obrigatórios. 3. O sistema retorna ao passo 2 do FB01. FA03: Sessão expirada ou revogada durante o uso 1. O token de sessão atinge o fim de seu prazo de validade, ou a sessão correspondente é revogada pelo sistema, por exemplo após um logout ou uma alteração de senha (RF-03). 2. Na próxima ação do usuário, o sistema verifica a sessão vinculada ao token e a identifica como inválida. 3. O sistema exibe mensagem: “Sua sessão expirou. Por favor, faça login novamente.” 4. O sistema redireciona o usuário para a tela de login.

Continua na próxima página

Quadro 1 – Continuação da página anterior

Item	Descrição
Requisitos Especiais	RNF-02 - A autenticação deve ser concluída em até 3 segundos. RNF-03 - A senha nunca deve ser trafegada ou armazenada em texto plano. RNF-03 - O token de sessão deve ter prazo de validade definido e poder ser revogado pelo servidor a qualquer momento.
Condições Prévias	O usuário deve estar cadastrado no sistema (RF-01). O usuário não deve possuir sessão ativa.
Pós-Condições	O usuário está autenticado e possui sessão ativa. Um token de sessão válido é gerado e armazenado. O usuário tem acesso às funcionalidades restritas da plataforma.
Pontos de Extensão	Nenhum ponto de extensão identificado para este caso de uso.

Fonte: Elaborada pelo autor

4.3.2.2 Realizar Logout

Quadro 2 – Especificação do Caso de Uso: Realizar Logout

Item	Descrição
Nome do Caso de Uso	Realizar Logout
Descrição Resumida	Este caso de uso permite que o usuário autenticado encerre sua sessão ativa na plataforma, invalidando o token de sessão e impedindo acesso não autorizado posterior (RF-02).

Continua na próxima página

Quadro 2 – Continuação da página anterior

Item	Descrição
Fluxo Básico	FB01: Encerramento de sessão 1. O usuário clica na opção “Sair” disponível no menu do sistema. 2. O sistema invalida imediatamente o token de sessão ativo [include: Invalidar Token de Sessão]. 3. O sistema encerra a sessão do usuário no servidor. 4. O sistema redireciona o usuário para a tela de login. 5. O sistema exibe mensagem informativa: “Você saiu com sucesso.”
Fluxos Alternativos	FA01: Erro ao encerrar sessão 1. No passo 3 do FB01, ocorre falha na comunicação com o servidor. 2. O sistema exibe mensagem de erro: “Não foi possível encerrar sua sessão. Tente novamente.” 3. O sistema mantém o usuário na tela atual.
Requisitos Especiais	RNF-02 - O logout deve ser concluído em até 2 segundos. RNF-03 - O token invalidado não deve ser reutilizável em nenhuma hipótese.
Condições Prévias	O usuário deve estar autenticado com sessão ativa.
Pós-Condições	A sessão do usuário é encerrada. O token de sessão é invalidado e não pode ser reutilizado. O usuário é redirecionado para a tela de login.

Continua na próxima página

Quadro 2 – Continuação da página anterior

Item	Descrição
Pontos de Extensão	Nenhum ponto de extensão identificado para este caso de uso.

Fonte: Elaborada pelo autor

4.3.2.3 Alterar Senha

Quadro 3 – Especificação do Caso de Uso: Alterar Senha

Item	Descrição
Nome do Caso de Uso	Alterar Senha
Descrição Resumida	Este caso de uso permite que o usuário autenticado troque sua senha dentro da área restrita, informando a senha atual para confirmação de identidade antes de definir a nova credencial (RF-03).

Continua na próxima página

Quadro 3 – Continuação da página anterior

Item	Descrição
Fluxo Básico	FB01: Alteração de senha 1. O usuário acessa as configurações de conta. 2. O usuário seleciona a opção “Alterar Senha”. 3. O sistema exibe o formulário com os campos de senha atual, nova senha e confirmação. 4. O usuário informa a senha atual. 5. O usuário define a nova senha e a repete no campo de confirmação. 6. O usuário confirma clicando em “Salvar”. 7. O sistema compara a senha atual informada com o hash armazenado. 8. O sistema valida a nova senha quanto ao comprimento mínimo e à divergência em relação à atual. 9. O sistema armazena a nova senha de forma criptografada. 10. O sistema revoga as demais sessões ativas do usuário. 11. O sistema exibe mensagem de sucesso: “Senha alterada com sucesso.”

Continua na próxima página

Quadro 3 – Continuação da página anterior

Item	Descrição
Fluxos Alternativos	FA01: Senha atual incorreta 1. No passo 7 do FB01, o sistema identifica que a senha atual não corresponde ao hash armazenado. 2. O sistema exibe a mensagem: “Senha atual inválida”. 3. O sistema retorna ao passo 4 do FB01. FA02: Nova senha e confirmação divergentes 1. No passo 8 do FB01, o sistema identifica que os dois campos não coincidem. 2. O sistema exibe aviso de que a nova senha e sua confirmação devem ser iguais. 3. O sistema retorna ao passo 5 do FB01. FA03: Nova senha inválida 1. No passo 8 do FB01, o sistema identifica que a nova senha tem menos de 8 caracteres ou é igual à senha atual. 2. O sistema exibe mensagem indicando a regra não atendida. 3. O sistema retorna ao passo 5 do FB01.
Requisitos Especiais	RNF-03 - A senha deve ser armazenada com algoritmo de hash seguro e nunca trafegar em texto plano. RNF-02 - A operação deve ser concluída em até 3 segundos.
Condições Prévias	O usuário deve estar autenticado com sessão ativa.
Pós-Condições	A senha é atualizada no banco de dados em forma criptografada. As demais sessões ativas do usuário são revogadas, permanecendo válida apenas a sessão corrente.

Continua na próxima página

Quadro 3 – Continuação da página anterior

Item	Descrição
Pontos de Extensão	Nenhum ponto de extensão identificado para este caso de uso.

Fonte: Elaborada pelo autor

4.3.2.4 Recuperar Senha via E-mail

Quadro 4 – Especificação do Caso de Uso: Recuperar Senha via E-mail

Item	Descrição
Nome do Caso de Uso	Recuperar Senha via E-mail
Descrição Resumida	Este caso de uso permite que um usuário não autenticado, que tenha esquecido sua senha, redefina a credencial por meio de um link enviado ao e-mail cadastrado, contendo um token único e temporário (RF-04 e RF-24).

Continua na próxima página

Quadro 4 – Continuação da página anterior

Item	Descrição
Fluxo Básico	FB01: Redefinição de senha por e-mail 1. O usuário acessa a tela de login e seleciona “Esqueci minha senha”. 2. O usuário informa o e-mail cadastrado. 3. O sistema exibe mensagem informando que, se o e-mail estiver cadastrado, um link de redefinição será enviado. 4. O sistema verifica se o e-mail corresponde a uma conta existente. 5. O sistema gera um token único e temporário e o armazena de forma protegida, associado à conta [include: Gerenciar Notificações do Sistema]. 6. O sistema envia ao endereço cadastrado uma mensagem contendo o link de redefinição. 7. O usuário acessa o link recebido. 8. O sistema valida o token quanto à existência, ao prazo de validade e ao uso prévio. 9. O usuário define a nova senha e a repete no campo de confirmação. 10. O sistema valida a nova senha e a armazena de forma criptografada. 11. O sistema invalida o token e encerra as sessões ativas anteriores do usuário. 12. O sistema redireciona o usuário para a tela de login com mensagem de sucesso.

Continua na próxima página

Quadro 4 – Continuação da página anterior

Item	Descrição
Fluxos Alternativos	FA01: E-mail não cadastrado 1. No passo 4 do FB01, o sistema identifica que o e-mail informado não corresponde a nenhuma conta. 2. O sistema não envia mensagem alguma, mas mantém a mesma resposta exibida no passo 3, de modo a não revelar quais endereços estão cadastrados. 3. O caso de uso é encerrado. FA02: Token expirado 1. No passo 8 do FB01, o sistema identifica que o token ultrapassou o prazo de 24 horas. 2. O sistema exibe mensagem informando que o link não é mais válido. 3. O sistema oferece a opção de solicitar um novo link, retornando ao passo 2 do FB01. FA03: Token já utilizado 1. No passo 8 do FB01, o sistema identifica que o token já foi usado em uma redefinição anterior. 2. O sistema exibe mensagem informando que o link não é mais válido. 3. O sistema oferece a opção de solicitar um novo link, retornando ao passo 2 do FB01. FA04: Nova senha inválida ou divergente 1. No passo 10 do FB01, o sistema identifica que a nova senha tem menos de 8 caracteres ou não coincide com a confirmação. 2. O sistema exibe mensagem indicando a regra não atendida. 3. O sistema retorna ao passo 9 do FB01, mantendo o token válido.

Continua na próxima página

Quadro 4 – Continuação da página anterior

Item	Descrição
Requisitos Especiais	RNF-03 - O token deve ser gerado com entropia suficiente e armazenado de forma protegida; a nova senha nunca deve trafegar em texto plano. RNF-02 - O envio da mensagem deve ser disparado em até 3 segundos.
Condições Prévias	O usuário não deve estar autenticado. O usuário deve possuir uma conta cadastrada (RF-01) e acesso ao e-mail informado no cadastro.
Pós-Condições	A senha é atualizada em forma criptografada. O token utilizado é invalidado e não pode ser reaproveitado. As sessões ativas anteriores são encerradas, exigindo nova autenticação.
Pontos de Extensão	Nenhum ponto de extensão identificado para este caso de uso.

Fonte: Elaborada pelo autor

4.3.2.5 Editar Perfil

Quadro 5 – Especificação do Caso de Uso: Editar Perfil

Item	Descrição
Nome do Caso de Uso	Editar Perfil

Continua na próxima página

Quadro 5 – Continuação da página anterior

Item	Descrição
Descrição Resumida	Este caso de uso permite que o usuário visualize e edite as informações de seu perfil, incluindo dados cadastrais básicos (nome, e-mail) e personalizações visuais (foto de perfil, descrição, biografia). Alterações no e-mail exigem verificação por meio de link enviado ao novo endereço (RF-05).
Fluxo Básico	FB01: Edição de perfil 1. O usuário acessa o menu de configurações ou clica em seu avatar. 2. O usuário seleciona a opção “Editar Perfil”. 3. O sistema exibe o formulário de edição com as informações atuais do usuário. 4. O usuário modifica os campos desejados (nome de exibição, biografia, foto) [E01]. 5. O usuário pode alterar seu e-mail; o sistema envia link de verificação ao novo endereço e mantém o e-mail antigo ativo até a confirmação [include: Gerenciar Notificações do Sistema]. 6. O usuário pode alterar sua senha [E02]. 7. O usuário confirma as alterações clicando em “Salvar”. 8. O sistema valida os dados inseridos. 9. O sistema atualiza as informações no banco de dados. 10. O sistema exibe mensagem de sucesso: “Perfil atualizado com sucesso.” 11. O sistema retorna à visualização do perfil com as informações atualizadas.

Continua na próxima página

Quadro 5 – Continuação da página anterior

Item	Descrição
Fluxos Alternativos	FA01: E-mail já cadastrado 1. No passo 8 do FB01, o sistema identifica que o novo e-mail já está cadastrado por outro usuário. 2. O sistema exibe mensagem de erro: “Este e-mail já está em uso. Por favor, utilize outro e-mail.” 3. O sistema retorna ao passo 5 do FB01. FA02: Formato de e-mail inválido 1. No passo 8 do FB01, o sistema identifica que o e-mail inserido possui formato inválido. 2. O sistema exibe mensagem de erro: “Por favor, insira um e-mail válido.” 3. O sistema retorna ao passo 5 do FB01. FA03: Imagem de perfil com formato não suportado 1. Durante o passo 4 do FB01, o usuário tenta fazer upload de imagem com formato não suportado. 2. O sistema exibe mensagem de erro: “Formato de imagem não suportado. Use PNG, JPG ou JPEG.” 3. O sistema retorna ao passo 4 do FB01. FA04: Imagem de perfil muito grande 1. Durante o passo 4 do FB01, o usuário tenta fazer upload de imagem maior que 5MB. 2. O sistema exibe mensagem de erro: “A imagem deve ter no máximo 5MB.” 3. O sistema retorna ao passo 4 do FB01. FA05: Link de verificação de e-mail expirado 1. O usuário tenta confirmar a alteração de e-mail após o prazo de 24 horas. 2. O sistema identifica que o token de verificação expirou. 3. O sistema exibe mensagem: “O link de verificação expirou. Por favor, solicite um novo.” 4. O e-mail original é mantido como endereço ativo. FA06: Cancelamento da operação 1. Em qualquer momento entre os passos 4 e 7 do FB01, o usuário pode selecionar “Cancelar”. 2. O sistema exibe mensagem de confirmação se houver alterações não salvas. 3. Se o usuário confirmar, o sistema descarta as alterações e

Quadro 5 – Continuação da página anterior

Item	Descrição
Requisitos Especiais	RNF-01 - A interface deve ser intuitiva. RNF-02 - As alterações devem ser salvas em até 3 segundos. RNF-03 - O sistema deve proteger informações pessoais. RNF-04 - Compatibilidade com dispositivos móveis.
Condições Prévias	O usuário deve estar autenticado no sistema.
Pós-Condições	As informações do perfil são atualizadas no banco de dados. As alterações são refletidas em todas as áreas do sistema que exibem informações do usuário. Se o e-mail foi alterado, um e-mail de verificação é enviado e o e-mail antigo permanece ativo até a confirmação. O código de usuário (RF-01) permanece inalterado.
Pontos de Extensão	E01 - Personalizar Informações (permite modificar foto, nome, descrição). E02 - Alterar senha (fluxo específico com verificação de senha atual).

Fonte: Elaborada pelo autor

4.3.2.6 Personalizar Interface

Quadro 6 – Especificação do Caso de Uso: Personalizar Interface

Item	Descrição
Nome do Caso de Uso	Personalizar Interface

Continua na próxima página

Quadro 6 – Continuação da página anterior

Item	Descrição
Descrição Resumida	Este caso de uso permite que o usuário gerencie a aparência visual global da plataforma por meio de paletas: selecionar a paleta em uso e criar, editar, duplicar, renomear e excluir paletas, cada uma com cor primária, subcores e tipografia. O estilo global (claro, escuro ou automático) é escolhido no menu da conta. Enquanto uma paleta é editada, a interface passa a exibi-la, para que o usuário veja o resultado exato antes de confirmar; nada é gravado até salvar, e tudo persiste entre sessões e dispositivos (RF-18 e RF-19).
Fluxo Básico	FB01: Seleção da paleta em uso 1. O usuário seleciona a opção “Aparência” no menu da conta. 2. O sistema exibe a biblioteca de paletas do usuário, com a paleta em uso destacada. 3. O usuário seleciona uma paleta da biblioteca. 4. O sistema define a paleta selecionada como paleta em uso, adotando o estilo global associado a ela. 5. O sistema aplica a paleta globalmente a todas as telas do sistema de forma imediata.

Continua na próxima página

Quadro 6 – Continuação da página anterior

Item	Descrição
Fluxos Alternativos	FA01: Criação de paleta 1. No passo 3 do FB01, o usuário seleciona “Nova paleta”. 2. O sistema exibe o editor de paleta com as cores e a tipografia da paleta em uso, e passa a exibir a interface com a paleta em edição. 3. O usuário informa o nome e modifica cores e tipografia [E01][E02]; a cada alteração o sistema atualiza a interface exibida, sem gravar. 4. O usuário clica em “Salvar”. 5. O sistema classifica a paleta como clara ou escura pela cor de fundo, salva-a na biblioteca e retorna ao passo 2 do FB01. FA02: Edição de paleta 1. No passo 3 do FB01, o usuário seleciona “Editar” no menu de uma paleta. 2. O sistema exibe o editor de paleta com os valores atuais da paleta, e passa a exibir a interface com a paleta em edição. 3. O usuário modifica nome, cores e tipografia [E01][E02] e clica em “Salvar”. 4. O sistema salva a paleta; se ela for a paleta em uso, as alterações são aplicadas globalmente de forma imediata. 5. O fluxo retorna ao passo 2 do FB01. FA03: Cancelamento da edição 1. Em qualquer momento no editor de paleta, antes de salvar, o usuário seleciona “Cancelar”. 2. Havendo alterações não salvas, o sistema exibe mensagem de confirmação: “Sair sem salvar?” 3. Se o usuário confirmar, o sistema descarta as alterações, devolve a interface à paleta em uso e retorna ao passo 2 do FB01; caso contrário, permanece no editor. FA04: Duplicação de paleta 1. No passo 3 do FB01, o usuário seleciona “Duplicar” no menu de uma paleta. 2. O sistema cria uma cópia da paleta, com “(cópia)” no nome, e a exibe na biblioteca. FA05: Renomeação de paleta

Quadro 6 – Continuação da página anterior

Item	Descrição
Requisitos Especiais	RNF-01 - A biblioteca e o editor de paletas devem ser intuitivos e acessíveis. RNF-07 - As alterações devem ser refletidas na interface em tempo real, sem recarregar a página. RNF-04 - As paletas devem ser persistidas entre sessões e dispositivos.
Condições Prévias	O usuário deve estar autenticado no sistema.
Pós-Condições	As paletas e a paleta em uso são salvas no perfil do usuário. A paleta em uso é aplicada globalmente e imediatamente a todas as telas, e persiste em futuras sessões e em outros dispositivos.
Pontos de Extensão	E01 - Modificar Cores (estende o FA01 e o FA02, conforme RF-18). E02 - Modificar Fontes (estende o FA01 e o FA02, conforme RF-19).

Fonte: Elaborada pelo autor

4.3.2.7 Criar Categoria

Quadro 7 – Especificação do Caso de Uso: Criar Categoria

Item	Descrição
Nome do Caso de Uso	Criar Categoria

Continua na próxima página

Quadro 7 – Continuação da página anterior

Item	Descrição
Descrição Resumida	Este caso de uso permite que o usuário crie uma nova categoria (agrupador temático de alto nível) para organizar coleções e itens relacionados a um tema, como livros, filmes, séries ou jogos. A categoria é criada vazia e servirá como container para coleções que serão criadas posteriormente. A categoria não possui nível de privacidade próprio; a privacidade é configurada individualmente em cada coleção (RF-06 e RF-08).

Continua na próxima página

Quadro 7 – Continuação da página anterior

Item	Descrição
Fluxo Básico	FB01: Criação de categoria 1. O usuário acessa a funcionalidade de gerenciamento de categorias no dashboard. 2. O usuário seleciona a opção “Criar Categoria”. 3. O sistema exibe o formulário de criação de categoria. 4. O usuário insere o nome da categoria. 5. O usuário seleciona um ícone para representar a categoria (opcional). 6. O usuário insere uma descrição para a categoria (opcional). 7. O usuário informa o tipo/tema (opcional), podendo escolher uma das sugestões oferecidas ou digitar o seu próprio. 8. O usuário confirma a criação da categoria. 9. O sistema valida os dados inseridos. 10. O sistema cria a categoria vazia no banco de dados. 11. O sistema exibe mensagem de sucesso. 12. O sistema exibe a categoria recém-criada com atalho destacado para “Criar primeira coleção”.

Continua na próxima página

Quadro 7 – Continuação da página anterior

Item	Descrição
Fluxos Alternativos	FA01: Nome da categoria já existe 1. No passo 9 do FB01, o sistema identifica que já existe uma categoria com o mesmo nome para o usuário. 2. O sistema exibe mensagem de erro: “Já existe uma categoria com este nome. Por favor, escolha outro nome.” 3. O sistema retorna ao passo 4 do FB01. FA02: Campos obrigatórios não preenchidos 1. No passo 9 do FB01, o sistema identifica que o campo nome não foi preenchido. 2. O sistema exibe mensagem de erro: “O nome da categoria é obrigatório.” 3. O sistema retorna ao passo 4 do FB01. FA03: Cancelamento da operação 1. Em qualquer momento entre os passos 4 e 8 do FB01, o usuário pode selecionar “Cancelar”. 2. O sistema descarta as informações inseridas. 3. O sistema retorna ao dashboard de categorias. 4. O caso de uso é encerrado.
Requisitos Especiais	RNF-01 - A interface deve ser intuitiva. RNF-02 - O sistema deve criar a categoria em até 3 segundos. RNF-04 - Compatibilidade com navegadores modernos e dispositivos móveis.
Condições Prévias	O usuário deve estar autenticado no sistema.

Continua na próxima página

Quadro 7 – Continuação da página anterior

Item	Descrição
Pós-Condições	Uma nova categoria é criada e armazenada no banco de dados. A categoria aparece no dashboard do usuário. A categoria está pronta para receber coleções, que terão sua própria privacidade individual (padrão pública, conforme RF-08). Nenhum item pode ser cadastrado diretamente na categoria; é necessário criar ao menos uma coleção antes.
Pontos de Extensão	Nenhum ponto de extensão identificado para este caso de uso.

Fonte: Elaborada pelo autor

4.3.2.8 Criar Coleção

Quadro 8 – Especificação do Caso de Uso: Criar Coleção

Item	Descrição
Nome do Caso de Uso	Criar Coleção
Descrição Resumida	Este caso de uso permite que o usuário crie uma nova coleção dentro de uma categoria existente. A coleção é onde os itens serão efetivamente cadastrados e é também a unidade onde a privacidade é configurada (pública, somente amigos ou privada) e onde o layout é associado (RF-07, RF-15 e RF-08).

Continua na próxima página

Quadro 8 – Continuação da página anterior

Item	Descrição
Fluxo Básico	FB01: Criação de coleção 1. O usuário acessa uma categoria específica no dashboard. 2. O usuário seleciona a opção “Criar Coleção”. 3. O sistema exibe o formulário de criação de coleção, com privacidade preenchida como “pública” por padrão. 4. O usuário insere o nome da coleção. 5. O usuário seleciona um ícone (opcional). 6. O usuário insere uma descrição (opcional). 7. O usuário associa um layout à coleção (opcional) [E01]. 8. O usuário configura a privacidade da coleção (pública, somente amigos ou privada) [E02]. 9. O usuário escreve as tags da coleção (opcional), recebendo como sugestão o vocabulário já existente no sistema (RF-07). 10. O usuário confirma a criação da coleção. 11. O sistema valida os dados inseridos. 12. O sistema cria a coleção vinculada à categoria pai, com a privacidade configurada, e vincula as tags informadas ao vocabulário, criando as que ainda não existirem. 13. O sistema exibe mensagem de sucesso. 14. O sistema redireciona para a visualização da coleção, pronta para receber itens.

Continua na próxima página

Quadro 8 – Continuação da página anterior

Item	Descrição
Fluxos Alternativos	FA01: Nome da coleção já existe na categoria 1. No passo 11 do FB01, o sistema identifica que já existe uma coleção com o mesmo nome na categoria pai. 2. O sistema exibe mensagem de erro: “Já existe uma coleção com este nome nesta categoria. Por favor, escolha outro nome.” 3. O sistema retorna ao passo 4 do FB01. FA02: Campos obrigatórios não preenchidos 1. No passo 11 do FB01, o sistema identifica que o campo nome não foi preenchido. 2. O sistema exibe mensagem de erro: “O nome da coleção é obrigatório.” 3. O sistema retorna ao passo 4 do FB01. FA03: Cancelamento da operação 1. Em qualquer momento entre os passos 4 e 9 do FB01, o usuário pode selecionar “Cancelar”. 2. O sistema descarta as informações inseridas. 3. O sistema retorna à categoria pai. 4. O caso de uso é encerrado.
Requisitos Especiais	RNF-01 - A interface deve ser intuitiva. RNF-02 - A coleção deve ser criada em até 3 segundos.
Condições Prévias	O usuário deve estar autenticado. A categoria pai deve existir e pertencer ao usuário.

Continua na próxima página

Quadro 8 – Continuação da página anterior

Item	Descrição
Pós-Condições	Uma nova coleção é criada vinculada à categoria pai, com sua privacidade individual configurada. A coleção está pronta para receber itens. Caso um layout tenha sido associado, os itens dessa coleção passam a adotá-lo; caso contrário, a ficha de cada item nasce vazia e o usuário monta o layout de cada um no editor (RF-17). A contagem de coleções da categoria é atualizada. Se a privacidade configurada for ‘pública’ ou ‘somente amigos’, a coleção poderá ser seguida por amigos (RF-21).
Pontos de Extensão	E01 - Associar Layout à Coleção (estende o passo 7 do FB01, conforme RF-15). E02 - Configurar Privacidade da Coleção (estende o passo 8 do FB01, conforme RF-08; padrão pública, alterável para privada ou somente amigos).

Fonte: Elaborada pelo autor

4.3.2.9 Gerenciar Privacidade da Coleção

Quadro 9 – Especificação do Caso de Uso: Gerenciar Privacidade da Coleção

Item	Descrição
Nome do Caso de Uso	Gerenciar Privacidade da Coleção
Descrição Resumida	Este caso de uso permite que o usuário defina quem pode visualizar e acompanhar cada uma de suas coleções, escolhendo entre os níveis pública, somente amigos e privada. A privacidade é configurada no nível da coleção; a categoria não possui nível próprio (RF-08 e RF-06).

Continua na próxima página

Quadro 9 – Continuação da página anterior

Item	Descrição
Fluxo Básico	FB01: Alteração do nível de privacidade 1. O usuário acessa uma coleção de sua propriedade. 2. O usuário seleciona a opção de edição da coleção. 3. O sistema exibe o nível de privacidade vigente entre as três opções disponíveis. 4. O usuário seleciona o novo nível desejado. 5. O usuário confirma a alteração. 6. O sistema atualiza a privacidade da coleção. 7. O sistema avalia os acompanhamentos existentes e encerra os que deixaram de ser permitidos [E01]. 8. O sistema exibe mensagem de sucesso e atualiza o indicador de privacidade exibido no <i>card</i> da coleção.

Continua na próxima página

Quadro 9 – Continuação da página anterior

Item	Descrição
Fluxos Alternativos	FA01: Restrição para privada 1. No passo 7 do FB01, o novo nível escolhido é “privada”. 2. O sistema encerra o acompanhamento de todos os seguidores da coleção. 3. O sistema remove os eventos dessa coleção dos <i>feeds</i> em que apareciam e oculta as notificações relacionadas (RF-22 e RF-24). 4. O fluxo continua no passo 8 do FB01. FA02: Restrição de pública para somente amigos 1. No passo 7 do FB01, a coleção passa de “pública” para “somente amigos”. 2. O sistema mantém o acompanhamento dos seguidores com amizade confirmada. 3. O sistema encerra o acompanhamento dos demais seguidores (RF-21). 4. O fluxo continua no passo 8 do FB01. FA03: Ampliação da visibilidade 1. No passo 7 do FB01, a coleção passa a um nível mais permissivo, como de “privada” para “pública”. 2. O sistema não restaura acompanhamentos encerrados anteriormente; a coleção volta a poder ser seguida, mas cada usuário precisa segui-la novamente. 3. O fluxo continua no passo 8 do FB01.

Continua na próxima página

Quadro 9 – Continuação da página anterior

Item	Descrição
Requisitos Especiais	RNF-03 - As regras de visibilidade devem ser aplicadas no servidor, de modo que uma coleção restrita não seja acessível nem por link direto. RNF-02 - A alteração deve ser concluída em até 3 segundos.
Condições Prévias	O usuário deve estar autenticado e ser o proprietário da coleção.
Pós-Condições	A coleção passa a ter o novo nível de privacidade. Os acompanhamentos incompatíveis com o novo nível são encerrados, e os eventos correspondentes deixam de ser exibidos nos <i>feeds</i> dos usuários afetados.
Pontos de Extensão	E01 - Encerrar Acompanhamentos (estende o passo 7 do FB01, conforme RF-21 – executado apenas quando a mudança restringe a visibilidade).

Fonte: Elaborada pelo autor

4.3.2.10 Criar Item

Quadro 10 – Especificação do Caso de Uso: Criar Item

Item	Descrição
Nome do Caso de Uso	Criar Item

Continua na próxima página

Quadro 10 – Continuação da página anterior

Item	Descrição
Descrição Resumida	Este caso de uso permite que o usuário cadastre um novo item dentro de uma coleção específica. O item adota automaticamente o layout associado à coleção; caso a coleção não tenha layout, a ficha do item nasce vazia e o próprio usuário monta o layout desse item (RF-09, RF-15 e RF-17).
Fluxo Básico	FB01: Criação de item 1. O usuário acessa uma coleção específica (dentro de uma categoria). 2. O usuário seleciona a opção “Adicionar Item”. 3. O sistema determina o layout aplicável ao item: o layout associado à coleção, se houver; caso contrário, nenhum, e a ficha do item nasce vazia. 4. O sistema exibe a ficha do item com os Elementos do layout aplicável. 5. O usuário insere o nome do item. 6. O usuário altera na ficha os Elementos que desejar, escrevendo sobre eles diretamente, como nos Elementos de Texto, de Data e de Classificação. 7. O usuário associa uma imagem ao item (opcional) [E02]. 8. O usuário confirma a criação do item. 9. O sistema valida os dados inseridos. 10. O sistema cria o item e o vincula à coleção. 11. O sistema exibe mensagem de sucesso. 12. O sistema exibe a ficha completa do item criado [E01].

Continua na próxima página

Quadro 10 – Continuação da página anterior

Item	Descrição
Fluxos Alternativos	FA01: Nenhum layout aplicável 1. No passo 3 do FB01, o sistema identifica que a coleção não possui layout associado. 2. O sistema exibe a ficha do item sem Elementos e disponibiliza o editor para que o usuário monte o layout desse item (RF-17). 3. O fluxo continua no passo 5 do FB01. FA02: Nome do item já existe na coleção 1. No passo 9 do FB01, o sistema identifica que já existe um item com o mesmo nome na coleção. 2. O sistema exibe mensagem de alerta: “Já existe um item com este nome nesta coleção. Deseja criar mesmo assim?” 3. Se o usuário confirmar, o fluxo continua no passo 10 do FB01. 4. Se o usuário cancelar, o sistema retorna ao passo 5 do FB01. FA03: Campos obrigatórios não preenchidos 1. No passo 9 do FB01, o sistema identifica que o campo nome não foi preenchido. 2. O sistema exibe mensagem de erro indicando os campos que precisam ser preenchidos. 3. O sistema retorna ao passo 5 do FB01. FA04: Cancelamento da operação 1. Em qualquer momento entre os passos 5 e 8 do FB01, o usuário pode selecionar “Cancelar”. 2. O sistema descarta as informações inseridas. 3. O sistema retorna à visualização da coleção. 4. O caso de uso é encerrado.

Continua na próxima página

Quadro 10 – Continuação da página anterior

Item	Descrição
Requisitos Especiais	RNF-01 - A interface deve ser intuitiva. RNF-02 - O sistema deve criar o item em até 3 segundos. RNF-07 - Alterações devem refletir em tempo real.
Condições Prévias	O usuário deve estar autenticado. A coleção de destino deve existir dentro de uma categoria válida.
Pós-Condições	Um novo item é criado e vinculado à coleção. O item adota o layout da coleção ou, não havendo layout associado, permanece com a ficha vazia, cabendo ao usuário montar o layout do item (RF-17). A ficha individual do item passa a ser exibida em modo leitura (RF-11). O item aparece na visualização da coleção. A contagem de itens da coleção e da categoria pai é atualizada.
Pontos de Extensão	E01 - Exibir Ficha de Item em Modo Leitura (estende o passo 12 do FB01, conforme RF-11). E02 - Associar Imagem ao Item (estende o passo 7 do FB01).

Fonte: Elaborada pelo autor

4.3.2.11 Exibir Ficha de Item em Modo Leitura

Quadro 11 – Especificação do Caso de Uso: Exibir Ficha de Item em Modo Leitura

Item	Descrição
Nome do Caso de Uso	Exibir Ficha de Item em Modo Leitura

Continua na próxima página

Quadro 11 – Continuação da página anterior

Item	Descrição
Descrição Resumida	Este caso de uso descreve a exibição da ficha individual de um item, com os Elementos do seu layout efetivo, tanto para o dono do acervo quanto para visitantes. A ficha é a tela de leitura do item: para o dono, é dela que se acessa a edição do layout do próprio item; para o visitante, ela é estritamente somente leitura (RF-11 e RF-17).
Fluxo Básico	FB01: Exibição da ficha para o dono 1. O usuário seleciona um item na listagem da coleção. 2. O sistema determina o layout efetivo do item, nesta ordem: o layout próprio do item, quando ele foi modificado individualmente (RF-17); na falta dele, o layout associado à sua coleção (RF-15); e, na falta dos dois, nenhum layout, restando à ficha apenas o nome e a data de criação do item. 3. O sistema recupera os Elementos desse layout, com seu conteúdo, posição, tamanho, camada e propriedades visuais. 4. O sistema exibe a ficha, desenhando cada Elemento na posição e com a aparência definidas, e respeitando a aparência global configurada pelo dono (RF-18). 5. O sistema marca os Elementos sem conteúdo, identificando cada um pelo seu nome. 6. O sistema disponibiliza ao dono o acesso à edição do layout do item [E01]. 7. O usuário retorna à listagem da coleção.

Continua na próxima página

Quadro 11 – Continuação da página anterior

Item	Descrição
Fluxos Alternativos	FA01: Item sem layout efetivo 1. No passo 2 do FB01, o sistema identifica que o item não possui layout próprio e que sua coleção também não possui layout associado. 2. O sistema exibe a ficha sem Elementos, apresentando apenas o nome e a data de criação do item. 3. O fluxo segue no passo 6 do FB01. FA02: Acesso por visitante 1. No passo 1 do FB01, quem acessa a ficha não é o dono do item. 2. O sistema verifica a privacidade da coleção a que o item pertence e o vínculo entre os dois usuários (RF-08). 3. Sendo o acesso permitido, o sistema exibe os mesmos Elementos do FB01, inclusive os vazios, e omite o passo 6: nenhuma opção de edição é apresentada. FA03: Acesso não permitido 1. No passo 2 do FA02, o sistema identifica que a privacidade da coleção não permite o acesso àquele visitante. 2. O sistema informa que o conteúdo não está disponível e não exibe a ficha. 3. O caso de uso é encerrado.

Continua na próxima página

Quadro 11 – Continuação da página anterior

Item	Descrição
Requisitos Especiais	RNF-02 - A ficha deve ser carregada em até 3 segundos. RNF-03 - A verificação de privacidade ocorre no servidor, e não apenas na interface. RNF-04 - A ficha deve permanecer legível em diferentes resoluções de tela, preservando as posições relativas entre os Elementos.
Condições Prévias	O item deve existir. O acesso à coleção a que ele pertence deve ser permitido a quem visualiza, seja por ser o dono, seja pelo nível de privacidade configurado (RF-08).
Pós-Condições	A ficha do item é exibida em modo leitura, com os Elementos do seu layout efetivo. Nenhum dado do item é alterado. Para o dono, o acesso à edição do layout do item fica disponível; para o visitante, nenhuma ação de edição é oferecida. As ações de alterar os dados do item e de excluí-lo permanecem no menu de ações da listagem da coleção, e não na ficha.
Pontos de Extensão	E01 - Modificar Layout do Item (estende o passo 6 do FB01, conforme RF-17; disponível apenas para o dono do item).

Fonte: Elaborada pelo autor

4.3.2.12 Visualizar Itens da Coleção

Quadro 12 – Especificação do Caso de Uso: Visualizar Itens da Coleção

Item	Descrição
Nome do Caso de Uso	Visualizar Itens da Coleção

Continua na próxima página

Quadro 12 – Continuação da página anterior

Item	Descrição
Descrição Resumida	Este caso de uso permite que o usuário visualize todos os itens de uma coleção, aplicando critérios de ordenação e de filtro. Os filtros disponíveis derivam dos Elementos presentes no layout efetivo dos itens, isto é, o layout da coleção ou, havendo personalização individual, o layout próprio do item (RF-12 e RF-17).
Fluxo Básico	FB01: Visualização dos itens 1. O usuário acessa uma coleção a partir da categoria correspondente. 2. O sistema recupera os itens vinculados à coleção. 3. O sistema identifica o layout efetivo de cada item e, a partir dos Elementos que o compõem, determina os critérios de filtro disponíveis. 4. O sistema exibe os itens na grade, aplicando a ordenação previamente escolhida pelo usuário. 5. O usuário pode alterar o critério de ordenação [E01]. 6. O usuário pode aplicar filtros sobre o conteúdo dos Elementos [E02]. 7. O sistema atualiza a listagem conforme os critérios aplicados. 8. O usuário seleciona um item para abrir sua ficha [include: Exibir Ficha de Item em Modo Leitura].

Continua na próxima página

Quadro 12 – Continuação da página anterior

Item	Descrição
Fluxos Alternativos	FA01: Coleção sem itens 1. No passo 2 do FB01, o sistema identifica que a coleção não possui itens. 2. O sistema exibe mensagem informativa e um atalho para o cadastro do primeiro item. 3. O caso de uso é encerrado. FA02: Nenhum item corresponde aos filtros 1. No passo 7 do FB01, nenhum item satisfaz a combinação de filtros aplicada. 2. O sistema exibe mensagem informando que não há resultados e oferece a opção de limpar os filtros. 3. O sistema retorna ao passo 6 do FB01. FA03: Coleção sem layout associado 1. No passo 3 do FB01, o sistema identifica que a coleção não possui layout associado. 2. O sistema disponibiliza os critérios derivados dos Elementos dos layouts próprios dos itens; não havendo Elementos, restam apenas o nome e a data de criação. 3. O fluxo continua no passo 4 do FB01. FA04: Saída da coleção 1. O usuário navega para fora da coleção. 2. O sistema descarta os filtros aplicados, que são válidos apenas durante a permanência na coleção. 3. O sistema preserva o critério de ordenação, que vale para todas as coleções até ser alterado.

Continua na próxima página

Quadro 12 – Continuação da página anterior

Item	Descrição
Requisitos Especiais	RNF-02 - A listagem deve ser carregada em até 3 segundos, com carregamento progressivo acima de 20 itens. RNF-04 - A grade deve permanecer legível em diferentes resoluções de tela.
Condições Prévias	O usuário deve estar autenticado. A coleção deve existir e ser visível para ele, seja por ser de sua propriedade, seja pelo nível de privacidade configurado (RF-08).
Pós-Condições	O usuário visualiza os itens da coleção conforme os critérios aplicados. O critério de ordenação escolhido é persistido para as demais coleções; os filtros são descartados ao sair da coleção.
Pontos de Extensão	E01 - Ordenar Itens (estende o passo 5 do FB01, conforme RF-12). E02 - Filtrar Itens (estende o passo 6 do FB01, conforme RF-12; os critérios derivam dos Elementos do layout efetivo de cada item).

Fonte: Elaborada pelo autor

4.3.2.13 Visualizar Dashboard de Categorias

Quadro 13 – Especificação do Caso de Uso: Visualizar Dashboard de Categorias

Item	Descrição
Nome do Caso de Uso	Visualizar Dashboard de Categorias

Continua na próxima página

Quadro 13 – Continuação da página anterior

Item	Descrição
Descrição Resumida	Este caso de uso permite que o usuário visualize todas as suas categorias em uma tela dedicada, com informações resumidas de cada uma (nome, ícone, número de coleções e número total de itens) e acesso rápido às coleções que pertencem a cada categoria. O dashboard é o ponto central de navegação do sistema e permite alternar entre modos de visualização, aplicar filtros e ordenações (RF-13).
Fluxo Básico	FB01: Visualização do dashboard 1. O usuário acessa o menu principal do sistema. 2. O usuário seleciona a opção “Minhas Categorias” ou equivalente. 3. O sistema recupera todas as categorias do usuário no banco de dados. 4. O sistema exibe o dashboard com cards de todas as categorias, aplicando as preferências de visualização previamente salvas (modo, ordenação e filtros). 5. Para cada categoria, o sistema exibe: nome, ícone, contagem de coleções e contagem total de itens (somando todas as coleções). 6. O usuário pode alternar entre modo grade e modo lista [E01]. 7. O usuário pode aplicar filtros e ordenações (por nome, data de criação ou tipo/tema da categoria) [E02]. 8. O usuário clica em uma categoria para acessar suas coleções.

Continua na próxima página

Quadro 13 – Continuação da página anterior

Item	Descrição
Fluxos Alternativos	FA01: Usuário sem categorias criadas 1. No passo 3 do FB01, o sistema identifica que o usuário não possui categorias criadas. 2. O sistema exibe mensagem informativa: “Você ainda não criou nenhuma categoria. Crie sua primeira categoria para começar!” 3. O sistema exibe botão destacado “Criar Nova Categoria”. 4. Se o usuário clicar no botão, o sistema aciona o caso de uso “Criar Categoria”. FA02: Erro ao carregar categorias 1. No passo 3 do FB01, o sistema encontra erro ao recuperar as categorias do banco de dados. 2. O sistema exibe mensagem de erro: “Não foi possível carregar suas categorias. Por favor, tente novamente.” 3. O sistema oferece opção para recarregar a página. FA03: Categoria vazia ao acessar 1. No passo 8 do FB01, o usuário clica em uma categoria que não possui coleções criadas. 2. O sistema exibe a página da categoria com mensagem: “Esta categoria ainda não tem coleções. Crie sua primeira coleção para começar a adicionar itens.” 3. O sistema exibe atalho destacado “Criar Primeira Coleção”. 4. Se o usuário clicar no atalho, o sistema aciona o caso de uso “Criar Coleção”.

Continua na próxima página

Quadro 13 – Continuação da página anterior

Item	Descrição
Requisitos Especiais	RNF-02 - O sistema deve carregar as categorias em até 3 segundos. RNF-04 - O dashboard deve ser responsivo em dispositivos móveis. RNF-09 - A interface deve seguir padrões de acessibilidade.
Condições Prévias	O usuário deve estar autenticado no sistema.
Pós-Condições	O usuário visualiza todas as suas categorias. O usuário pode navegar para qualquer categoria e, dentro dela, para qualquer coleção ou item. O estado de visualização (grade/lista, filtros, ordenação) é persistido para futuras sessões.
Pontos de Extensão	E01 - Alternar modo de visualização (grade/lista, conforme RF-13). E02 - Filtrar e Ordenar Categorias (permite organizar por diferentes critérios, conforme RF-13).

Fonte: Elaborada pelo autor

4.3.2.14 Criar Layout

Quadro 14 – Especificação do Caso de Uso: Criar Layout

Item	Descrição
Nome do Caso de Uso	Criar Layout

Continua na próxima página

Quadro 14 – Continuação da página anterior

Item	Descrição
Descrição Resumida	Este caso de uso permite que o usuário crie um layout personalizado no editor visual, acrescentando à área de edição os Elementos que comporão a ficha dos itens e definindo a posição, o tamanho e a aparência de cada um. O layout pode ser criado do zero ou por duplicação de um layout já existente na biblioteca e, posteriormente, associado a uma ou mais coleções, conforme RF-14 e RF-15.

Continua na próxima página

Quadro 14 – Continuação da página anterior

Item	Descrição
Fluxo Básico	FB01: Criação de layout 1. O usuário acessa a área de gerenciamento de layouts. 2. O usuário seleciona a opção “Criar Layout”. 3. O sistema exibe o editor com a área de edição vazia e o painel de Elementos ao lado. 4. O usuário insere um nome para o layout. 5. O usuário insere uma descrição para o layout (opcional). 6. O usuário acrescenta Elementos à área de edição pelo painel, escolhendo entre os seis tipos disponíveis: texto, imagem, forma, ícone, classificação e data [E01]. 7. O usuário posiciona e redimensiona cada Elemento arrastando-o pela área de edição e ajustando sua alça, e define a camada em que ele se sobrepõe aos demais. 8. O usuário escreve o conteúdo de um Elemento diretamente sobre ele, na própria área de edição. 9. O usuário ajusta as propriedades visuais do Elemento selecionado: no texto, fonte, tamanho, peso, itálico, sublinhado, riscado, alinhamento horizontal e vertical, nível de título, entrelinha, espaçamento entre letras e apresentação em lista; na forma e na imagem, forma do elemento, arredondamento e borda (cor, espessura e traço); na forma, ainda, curva e ponta da linha; na imagem, ajuste; na classificação, nota máxima e ícone; na data, formato e estética; e, em qualquer Elemento, cor, cor de fundo, opacidade e rotação. 10. O sistema grava cada alteração concluída automaticamente, sem exigir confirmação, e avisa o usuário caso ele tente sair da página com alteração ainda por gravar. 11. O usuário retorna à lista de layouts, na qual o novo layout já aparece disponível para associação a coleções (RF-15).

Continua na próxima página

Quadro 14 – Continuação da página anterior

Item	Descrição
Fluxos	FA01: Nome do layout já existe
Alternativos	1. No passo 4 do FB01, o sistema identifica que já existe um layout com o mesmo nome na biblioteca do usuário. 2. O sistema exibe mensagem de erro: “Já existe um layout com este nome. Por favor, escolha outro nome.” 3. O sistema retorna ao passo 4 do FB01. FA02: Layout sem Elementos 1. No passo 11 do FB01, o sistema identifica que nenhum Elemento foi acrescentado ao layout. 2. O sistema informa que o layout precisa de ao menos um Elemento para ser associado a uma coleção. 3. O sistema retorna ao passo 6 do FB01. FA03: Duplicação de layout existente 1. No passo 2 do FB01, o usuário pode optar por “Duplicar Layout Existente” [E02]. 2. O sistema exibe a biblioteca de layouts do usuário, que reúne os layouts criados por ele, os layouts padrão oferecidos pelo sistema e os layouts importados de outros usuários (RF-16). 3. O usuário seleciona um layout para duplicar. 4. O sistema carrega a estrutura do layout selecionado no editor, pré-preenchendo o nome com o original acrescido de “(cópia)”. 5. O fluxo continua no passo 4 do FB01, com o usuário podendo renomear e ajustar a cópia livremente. 6. Alterações realizadas no layout duplicado não refletem no layout de origem. FA04: Desfazer uma alteração 1. Em qualquer momento entre os passos 6 e 10 do FB01, o usuário pode desfazer a última alteração, e também refazê-la. 2. O sistema restaura o estado anterior da área de edição e o grava, percorrendo o histórico de alterações da sessão de edição. 3. O fluxo retorna ao passo em que o usuário estava. 4. Caso o usuário feche a página havendo alterações ainda não gravadas, o sistema o avisa antes de sair.

Quadro 14 – Continuação da página anterior

Item	Descrição
Requisitos Especiais	RNF-01 - O editor deve ser intuitivo e facilitar a personalização. RNF-07 - As alterações devem ser gravadas e refletidas na prévia sem recarregar a página e sem travamentos. RNF-04 - Compatibilidade com navegadores modernos.
Condições Prévias	O usuário deve estar autenticado. Para duplicação (FA03), deve existir ao menos um layout na biblioteca do usuário.
Pós-Condições	Um novo layout é criado e armazenado no banco de dados, com os Elementos acrescentados pelo usuário, suas posições, tamanhos, camadas, conteúdos e propriedades visuais. O layout está disponível para associação a uma ou mais coleções, conforme RF-15, e aparece na biblioteca de layouts do usuário. Se criado por duplicação, o layout de origem permanece inalterado.
Pontos de Extensão	E01 - Editar Layout (estende o passo 6 do FB01, permitindo acrescentar, alterar e remover Elementos a qualquer momento, conforme RF-14). E02 - Duplicar Layout (estende o passo 2 do FB01, permitindo criar um layout a partir de outro já existente na biblioteca, conforme RF-14).

Fonte: Elaborada pelo autor

4.3.2.15 Compartilhar Layout

Quadro 15 – Especificação do Caso de Uso: Compartilhar Layout

Item	Descrição
Nome do Caso de Uso	Compartilhar Layout
Descrição Resumida	Este caso de uso permite que um usuário compartilhe um layout de sua biblioteca com outra pessoa, por meio de um link que contém o identificador do layout, e que o destinatário o importe, recebendo uma cópia independente em sua própria biblioteca (RF-16).

Continua na próxima página

Quadro 15 – Continuação da página anterior

Item	Descrição
Fluxo Básico	FB01: Compartilhamento e importação 1. O usuário proprietário acessa sua biblioteca de layouts. 2. O usuário seleciona o layout desejado e escolhe a opção de compartilhar. 3. O sistema apresenta o link de compartilhamento, contendo o identificador do layout. 4. O usuário envia o link ao destinatário pelo meio que preferir, externo à plataforma. 5. O destinatário, autenticado, acessa o link recebido. 6. O sistema verifica se o layout existe e se não pertence ao próprio destinatário. 7. O sistema exibe uma prévia da estrutura do layout e solicita confirmação da importação. 8. O destinatário confirma a importação. 9. O sistema cria, na biblioteca do destinatário, uma cópia integral da estrutura do layout, incluindo Elementos, conteúdo, estilos, posições e tamanhos. 10. O sistema registra a cópia com a finalidade de <i>template</i> importado e guarda a referência ao layout de origem. 11. O sistema exibe mensagem de sucesso e apresenta o layout importado na biblioteca do destinatário.

Continua na próxima página

Quadro 15 – Continuação da página anterior

Item	Descrição
Fluxos Alternativos	FA01: Layout de origem inexistente 1. No passo 6 do FB01, o sistema identifica que o layout foi excluído pelo proprietário. 2. O sistema exibe mensagem informando que o layout não está mais disponível. 3. O caso de uso é encerrado. FA02: Importação do próprio layout 1. No passo 6 do FB01, o sistema identifica que o layout já pertence ao usuário que acessou o link. 2. O sistema informa que, para obter outra cópia de um layout próprio, deve-se utilizar a duplicação (RF-14). 3. O caso de uso é encerrado. FA03: Nome de layout já existente na biblioteca do destinatário 1. No passo 9 do FB01, o sistema identifica que o destinatário já possui um layout com o mesmo nome. 2. O sistema solicita um nome alternativo para a cópia. 3. O fluxo continua no passo 9 do FB01. FA04: Alteração posterior no layout de origem 1. Após a importação, o proprietário altera ou exclui o layout de origem. 2. As cópias já importadas permanecem inalteradas, pois são independentes. 3. Na exclusão, o sistema apenas zera a referência à origem, preservando a cópia.

Continua na próxima página

Quadro 15 – Continuação da página anterior

Item	Descrição
Requisitos Especiais	RNF-03 - A importação deve exigir usuário autenticado, e o compartilhamento não deve expor dados do proprietário além da estrutura do layout. RNF-02 - A cópia deve ser concluída em até 3 segundos.
Condições Prévias	Ambos os usuários devem estar cadastrados. O layout de origem deve existir e pertencer a quem compartilha. Não é exigido vínculo de amizade entre os dois.
Pós-Condições	Uma cópia independente do layout passa a existir na biblioteca do destinatário, utilizável e editável como se tivesse sido criada por ele. O layout de origem permanece inalterado, e nenhum item ou personalização individual do proprietário é compartilhado.
Pontos de Extensão	E01 - Visualizar Prévia do Layout (estende o passo 7 do FB01 – exibe a estrutura antes de confirmar a importação).

Fonte: Elaborada pelo autor

4.3.2.16 Adicionar Amigo

Quadro 16 – Especificação do Caso de Uso: Adicionar Amigo

Item	Descrição
Nome do Caso de Uso	Adicionar Amigo

Continua na próxima página

Quadro 16 – Continuação da página anterior

Item	Descrição
Descrição Resumida	Este caso de uso permite que o usuário envie solicitações de amizade para outros usuários da plataforma. A busca pode ser realizada pelo código de usuário ou pelo nome, e a solicitação fica pendente até que o destinatário a aceite ou recuse (RF-20 e RF-24).

Continua na próxima página

Quadro 16 – Continuação da página anterior

Item	Descrição
Fluxo Básico	FB01: Envio de solicitação de amizade 1. O usuário acessa a área de busca de usuários ou área social. 2. O usuário insere o código do usuário ou o nome da pessoa que deseja adicionar. 3. O sistema busca usuários correspondentes no banco de dados, conforme o critério informado. 4. O sistema exibe uma lista de resultados com nome, foto e código de usuário. 5. O usuário identifica a pessoa desejada e clica em “Adicionar Amigo”. 6. O sistema verifica se já existe uma relação de amizade ou solicitação pendente. 7. O sistema cria uma solicitação de amizade pendente no banco de dados. 8. O sistema gera notificação in-app ao destinatário informando sobre a solicitação [include: Gerenciar Notificações do Sistema]. 9. O sistema exibe mensagem de sucesso: “Solicitação de amizade enviada.” 10. O sistema atualiza o botão para “Solicitação Pendente” no perfil do usuário destinatário.

Continua na próxima página

Quadro 16 – Continuação da página anterior

Item	Descrição
Fluxos Alternativos	FA01: Usuário não encontrado 1. No passo 3 do FB01, o sistema não encontra nenhum usuário correspondente ao critério informado. 2. O sistema exibe mensagem: “Nenhum usuário encontrado.” 3. O sistema retorna ao passo 2 do FB01. FA02: Já são amigos 1. No passo 6 do FB01, o sistema identifica que os usuários já são amigos. 2. O sistema exibe mensagem: “Vocês já são amigos.” 3. O sistema oferece opção de visualizar o perfil do amigo. 4. O caso de uso é encerrado. FA03: Solicitação já existe 1. No passo 6 do FB01, o sistema identifica que já existe uma solicitação pendente. 2. O sistema exibe mensagem: “Você já enviou uma solicitação de amizade para este usuário.” 3. O sistema oferece opção de cancelar a solicitação pendente. 4. O caso de uso é encerrado. FA04: Solicitação recebida do mesmo usuário 1. No passo 6 do FB01, o sistema identifica que existe uma solicitação pendente enviada pelo usuário que está sendo adicionado. 2. O sistema exibe mensagem: “Este usuário já enviou uma solicitação de amizade para você.” 3. O sistema oferece opção de aceitar a solicitação existente. 4. Se o usuário aceitar, o sistema aciona o caso de uso “Gerenciar Solicitações de Amizade”.

Continua na próxima página

Quadro 16 – Continuação da página anterior

Item	Descrição
Requisitos Especiais	RNF-02 - A busca e envio de solicitação devem ocorrer em até 3 segundos. RNF-08 - O sistema deve enviar notificações em tempo real. RNF-03 - A privacidade dos usuários deve ser respeitada.
Condições Prévias	O usuário deve estar autenticado no sistema. O usuário destinatário deve existir e ter um perfil ativo.
Pós-Condições	Uma solicitação de amizade é criada com status “pendente”. O destinatário recebe notificação da solicitação (RF-24). A solicitação aparece na lista de pendentes de ambos os usuários. O botão no perfil do destinatário muda para “Solicitação Pendente”.
Pontos de Extensão	Nenhum ponto de extensão identificado para este caso de uso.

Fonte: Elaborada pelo autor

4.3.2.17 Gerenciar Solicitações de Amizade

Quadro 17 – Especificação do Caso de Uso: Gerenciar Solicitações de Amizade

Item	Descrição
Nome do Caso de Uso	Gerenciar Solicitações de Amizade

Continua na próxima página

Quadro 17 – Continuação da página anterior

Item	Descrição
Descrição Resumida	Este caso de uso permite que o usuário visualize, aceite ou recuse solicitações de amizade recebidas de outros usuários da plataforma. Ao aceitar, o vínculo de amizade é criado mutuamente e o solicitante é notificado. Ao recusar, a solicitação é marcada como recusada sem notificação ao solicitante (RF-20 e RF-24).
Fluxo Básico	FB01: Aceitação de solicitação de amizade 1. O usuário acessa a área de notificações ou de solicitações pendentes. 2. O sistema exibe a lista de solicitações de amizade com status “pendente”, contendo nome, foto e data de envio de cada solicitante. 3. O usuário seleciona a solicitação desejada e clica em “Aceitar”. 4. O sistema atualiza o status da solicitação para “aceita” no banco de dados. 5. O sistema cria o vínculo de amizade mútuo entre os dois usuários. 6. O sistema envia notificação ao solicitante informando que a solicitação foi aceita [include: Gerenciar Notificações do Sistema]. 7. O sistema exibe mensagem de sucesso: “Vocês agora são amigos!” 8. O sistema atualiza a lista de solicitações, removendo a solicitação aceita.

Continua na próxima página

Quadro 17 – Continuação da página anterior

Item	Descrição
Fluxos Alternativos	FA01: Recusa de solicitação 1. No passo 3 do FB01, o usuário clica em “Recusar” na solicitação desejada. 2. O sistema atualiza o status da solicitação para “recusada” no banco de dados. 3. O sistema não gera notificação ao solicitante; a recusa é silenciosa. 4. O sistema remove a solicitação da lista de pendentes do usuário. 5. O caso de uso é encerrado. FA02: Nenhuma solicitação pendente 1. No passo 2 do FB01, o sistema identifica que não há solicitações pendentes. 2. O sistema exibe mensagem: “Você não possui solicitações de amizade pendentes.” 3. O caso de uso é encerrado. FA03: Solicitação cancelada pelo remetente 1. No passo 3 do FB01, o usuário tenta aceitar uma solicitação que foi cancelada pelo remetente antes da resposta. 2. O sistema exibe mensagem: “Esta solicitação não está mais disponível.” 3. O sistema remove a solicitação da lista de pendentes. 4. O caso de uso é encerrado.

Continua na próxima página

Quadro 17 – Continuação da página anterior

Item	Descrição
Requisitos Especiais	RNF-08 - Notificações de aceite devem ser enviadas em tempo real; recusas não geram evento. RNF-02 - A operação deve ser concluída em até 3 segundos.
Condições Prévias	O usuário deve estar autenticado. Deve existir ao menos uma solicitação de amizade com status “pendente” endereçada ao usuário.
Pós-Condições	Se aceita: vínculo de amizade mútuo criado; ambos os usuários aparecem na lista de amigos um do outro e o solicitante é notificado. Se recusada: status da solicitação atualizado para “recusada”; nenhum vínculo ou notificação é criado.
Pontos de Extensão	Nenhum ponto de extensão identificado para este caso de uso.

Fonte: Elaborada pelo autor

4.3.2.18 Visualizar Perfil de Usuário

Quadro 18 – Especificação do Caso de Uso: Visualizar Perfil de Usuário

Item	Descrição
Nome do Caso de Uso	Visualizar Perfil de Usuário

Continua na próxima página

Quadro 18 – Continuação da página anterior

Item	Descrição
Descrição Resumida	Este caso de uso permite que o usuário acesse o perfil público de outro usuário (amigo ou não), visualizando suas informações básicas e as coleções disponíveis conforme o nível de privacidade configurado em cada uma delas. As coleções visíveis são agrupadas pelas respectivas categorias, e a navegação nas coleções e itens do usuário visitado é restrita ao modo somente leitura (RF-23 e RF-08).

Continua na próxima página

Quadro 18 – Continuação da página anterior

Item	Descrição
Fluxo Básico	FB01: Visualização do perfil 1. O usuário acessa a lista de amigos ou realiza busca pelo código do usuário desejado. 2. O usuário seleciona o perfil desejado. 3. O sistema recupera as informações públicas do perfil (nome, foto, biografia). 4. O sistema recupera as coleções do usuário visitado com privacidade “pública” ou “somente amigos” (esta última apenas quando há vínculo de amizade confirmada), agrupando-as pelas respectivas categorias. 5. O sistema exibe o perfil com suas informações e as coleções acessíveis, agrupadas por categoria. 6. Coleções com privacidade “privada” não são listadas nem mencionadas. Categorias que, para esse visitante, não tenham nenhuma coleção visível também são ocultadas integralmente. 7. O usuário navega pelas coleções visíveis e pelos itens contidos nelas, em modo somente leitura. 8. O usuário pode seguir uma coleção pública independentemente de amizade; para coleções “somente amigos”, o acompanhamento exige amizade confirmada [E01].

Continua na próxima página

Quadro 18 – Continuação da página anterior

Item	Descrição
Fluxos Alternativos	FA01: Perfil sem coleções acessíveis 1. No passo 4 do FB01, o sistema identifica que todas as coleções do usuário visitado são privadas ou que ele não possui coleções acessíveis ao visitante. 2. O sistema exibe o perfil básico do usuário visitado com a mensagem: “Este usuário não possui coleções públicas no momento.” 3. O caso de uso continua sem exibir coleções ou categorias. FA02: Tentativa de acesso a coleção privada por link direto 1. O usuário tenta acessar uma coleção privada de outro usuário por meio de link direto. 2. O sistema verifica a privacidade da coleção. 3. O sistema exibe mensagem: “Esta coleção não está disponível.” 4. O sistema não exibe nenhum dado da coleção, nem mesmo o nome. FA03: Usuário acessado não é amigo confirmado 1. No passo 4 do FB01, o sistema identifica que não existe vínculo de amizade confirmado entre os usuários. 2. O sistema exibe apenas as coleções com privacidade “pública” do perfil visitado, agrupadas pelas respectivas categorias. 3. Coleções com privacidade “somente amigos” não são exibidas. 4. Categorias sem nenhuma coleção pública visível são ocultadas integralmente.

Continua na próxima página

Quadro 18 – Continuação da página anterior

Item	Descrição
Requisitos Especiais	RNF-03 - A privacidade das coleções deve ser respeitada rigorosamente em toda a navegação. RNF-02 - O perfil deve ser carregado em até 3 segundos. RNF-04 - A visualização deve ser responsiva em dispositivos móveis.
Condições Prévias	O usuário deve estar autenticado. O usuário visitado deve existir na plataforma.
Pós-Condições	O usuário visualiza o perfil e as coleções acessíveis do usuário visitado, agrupadas por categoria. Nenhuma alteração é feita nos dados do usuário visitado. O usuário pode iniciar o fluxo de seguir uma coleção a partir desta tela.
Pontos de Extensão	E01 - Seguir Coleção (estende o passo 8 do FB01, conforme RF-21 – permite ao usuário acompanhar uma coleção pública ou somente amigos a partir do perfil visitado).

Fonte: Elaborada pelo autor

4.3.2.19 Visualizar Feed de Atualizações

Quadro 19 – Especificação do Caso de Uso: Visualizar Feed de Atualizações

Item	Descrição
Nome do Caso de Uso	Visualizar Feed de Atualizações

Continua na próxima página

Quadro 19 – Continuação da página anterior

Item	Descrição
Descrição Resumida	Este caso de uso permite que o usuário acompanhe em tempo real as atualizações das coleções que segue, visualizando eventos de adição, edição e remoção de itens realizados por seus proprietários, ordenados cronologicamente do mais recente ao mais antigo (RF-22).
Fluxo Básico	FB01: Exibição do feed 1. O usuário acessa a área de feed de atualizações. 2. O sistema identifica todas as coleções que o usuário segue ativamente (conforme RF-21). 3. O sistema recupera os eventos de atualização associados a essas coleções (adição/edição/remoção de itens), ordenados do mais recente ao mais antigo. 4. O sistema verifica a privacidade atual de cada coleção antes de exibir seus eventos (RF-08). 5. O sistema exibe o feed com os eventos, contendo: nome do autor da ação, tipo de evento, nome do item afetado, nome da coleção, nome da categoria pai e data/hora relativa do evento. 6. O feed é atualizado automaticamente em tempo real sem necessidade de recarregar a página. 7. O usuário pode rolar o feed para carregar eventos mais antigos progressivamente (paginação).

Continua na próxima página

Quadro 19 – Continuação da página anterior

Item	Descrição
Fluxos Alternativos	FA01: Usuário não segue nenhuma coleção 1. No passo 2 do FB01, o sistema identifica que o usuário não segue nenhuma coleção. 2. O sistema exibe mensagem: “Seu feed está vazio. Siga coleções para acompanhar suas atualizações.” 3. O sistema exibe atalho para a área de descoberta de perfis e coleções públicas. FA02: Coleção seguida alterada para privada 1. No passo 4 do FB01, o sistema identifica que uma coleção anteriormente seguida foi alterada para privada. 2. O sistema não exibe os eventos dessa coleção, incluindo eventos anteriores à mudança de privacidade. 3. O sistema encerra automaticamente o acompanhamento dessa coleção. 4. Os demais eventos do feed continuam sendo exibidos normalmente. FA03: Erro ao carregar o feed 1. No passo 3 do FB01, o sistema encontra erro ao recuperar os eventos. 2. O sistema exibe mensagem: “Não foi possível carregar o feed. Tente novamente.” 3. O sistema oferece botão para recarregar.

Continua na próxima página

Quadro 19 – Continuação da página anterior

Item	Descrição
Requisitos Especiais	RNF-07 - O feed deve ser atualizado em tempo real sem recarregar a página. RNF-02 - O carregamento inicial do feed deve ocorrer em até 3 segundos. RNF-03 - Eventos de coleções privadas não devem ser exibidos em nenhuma circunstância.
Condições Prévias	O usuário deve estar autenticado. O usuário deve seguir ao menos uma coleção com privacidade pública ou somente amigos (RF-21).
Pós-Condições	O usuário visualiza os eventos recentes das coleções que segue. O feed permanece atualizado em tempo real durante a navegação. Coleções tornadas privadas são removidas do feed imediatamente, com seu acompanhamento encerrado.
Pontos de Extensão	Nenhum ponto de extensão identificado para este caso de uso.

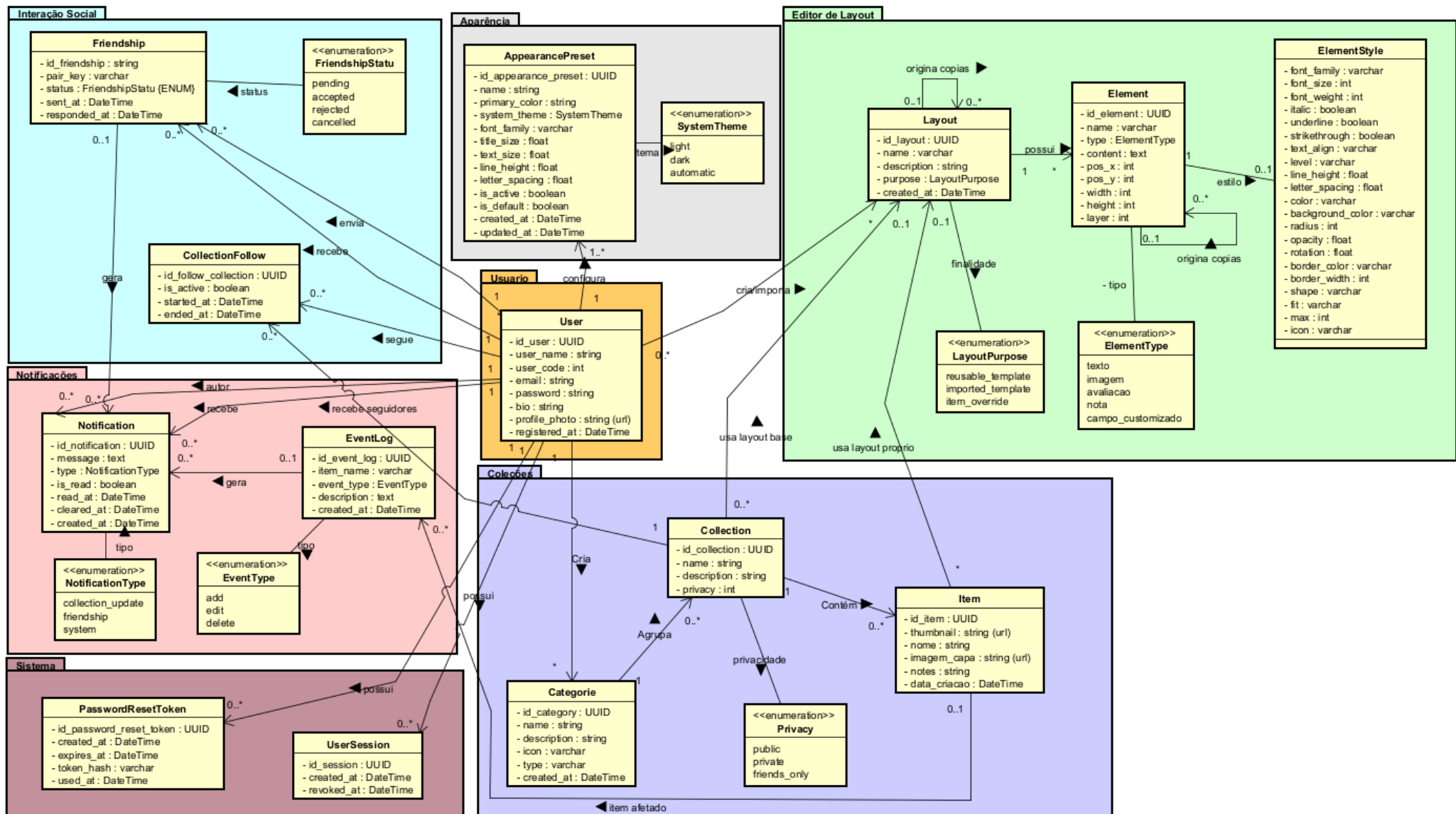
Fonte: Elaborada pelo autor

4.4 Modelo de Domínio

De acordo com (LARMAN, 2007), o modelo de domínio é uma representação visual das classes conceituais ou objetos do mundo real em um domínio de interesse, ilustrando conceitos significativos, atributos e associações entre eles.

A Figura 8 apresenta o modelo de domínio desenvolvido para o escopo deste trabalho, evidenciando a organização nos sete pacotes descritos na Subseção 4.1.1, bem como as principais classes conceituais do sistema e as associações que estabelecem entre si.

Figura 8 – Modelo de Domínio



Fonte: Elaborada pelo autor

5 PROJETO DE SOFTWARE

Este capítulo apresenta a arquitetura técnica e as especificações de projeto que fundamentam o sistema de gerenciamento de coleções. Nele, detalham-se as interfaces, a modelagem de dados e as regras de persistência utilizadas no desenvolvimento da aplicação.

5.1 Projeto de Interface

As interfaces do sistema foram projetadas na ferramenta **Figma** e adotam um partido visual deliberadamente simples, construído sobre poucos componentes reutilizáveis, uma tipografia única e uma paleta reduzida, ancorada em um tom de roxo. A escolha é coerente com a proposta do produto: como o conteúdo das fichas é inteiramente definido pelo usuário no editor de *layouts*, a interface que o cerca deve permanecer discreta, para não competir visualmente com aquilo que o próprio usuário compôs. Convém registrar que os protótipos aqui descritos foram elaborados na fase de concepção, antes da implementação, e representam a intenção de projeto: definem a organização e o comportamento pretendidos para as telas, e não seu resultado final, que pode incorporar ajustes decorrentes do desenvolvimento.

Os esquemas de tela elaborados nessa fase são apresentados nas Figuras 9 a 18. Eles registram a organização pretendida para cada tela e serviram de base para o desenvolvimento, sem a pretensão de corresponder ponto a ponto à interface final.

O sistema divide-se em duas regiões visualmente distintas. Antes da autenticação, na página inicial, no cadastro e nas telas de *login* e de recuperação de senha, o conteúdo aparece centralizado e isolado, sem elementos de navegação, já que ainda não há acervo a percorrer. A Figura 9 mostra esse arranjo na tela de entrada, reduzida à identidade da plataforma e às duas ações possíveis. Após a autenticação, as telas passam a compartilhar uma mesma moldura: uma barra lateral fixa, que dá acesso às categorias do usuário e à área social; uma barra superior, que reúne a

identificação da tela atual, o acesso às notificações e o menu do usuário; e a área central, onde o conteúdo é efetivamente apresentado.

Figura 9 – Esquema da tela inicial



Fonte: Elaborada pelo autor

A navegação pela hierarquia de três níveis é o eixo da área central. Categorias, coleções e itens são exibidos em uma grade de *cards*, e uma trilha de navegação no topo indica a posição do usuário, por exemplo “Categorias > Livros > Lidos”, permitindo retornar a qualquer nível anterior com um clique. Os *cards* adaptam-se ao que representam: os de coleção exibem o indicador de privacidade e a contagem de itens, enquanto os de item privilegiam a imagem de capa. Acima da grade situam-se os controles de modo de exibição, ordenação e filtro previstos nos requisitos, e um botão flutuante, no canto inferior, cria um novo registro no contexto em que o usuário se encontra.

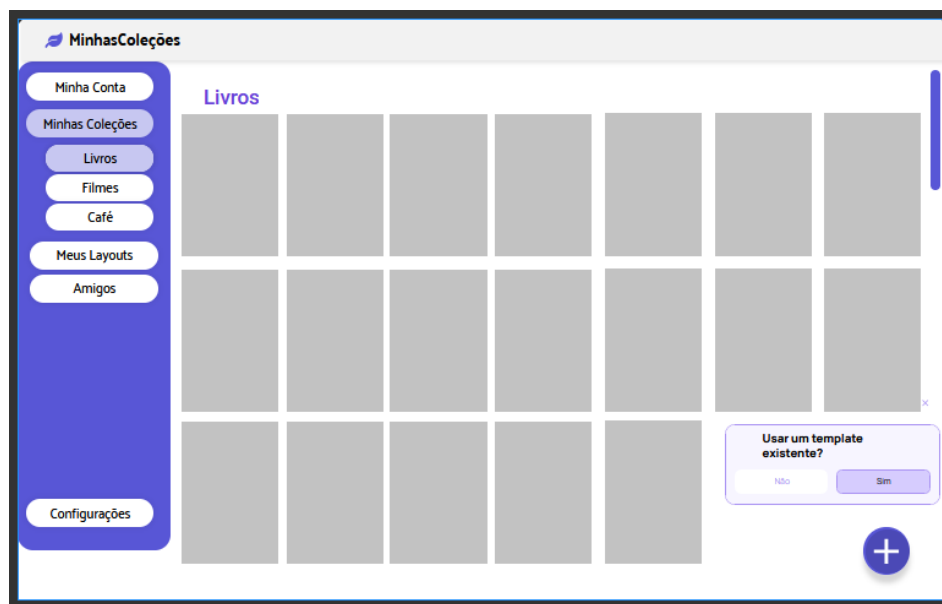
A Figura 10 apresenta o esquema da tela que reúne as coleções do usuário, agrupadas sob as categorias a que pertencem, com a barra lateral dando acesso direto a cada uma delas. A Figura 11, por sua vez, mostra a listagem de itens de uma coleção, com a grade de *cards* ocupando a área central e o botão flutuante de criação no canto inferior direito.

Figura 10 – Esquema da tela de coleções do usuário



Fonte: Elaborada pelo autor

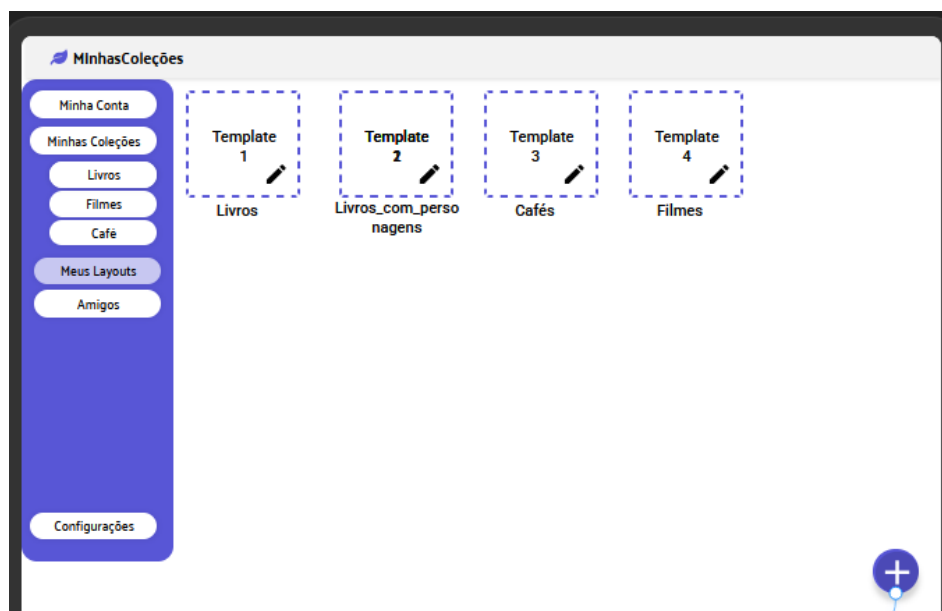
Figura 11 – Esquema da listagem de itens de uma coleção



Fonte: Elaborada pelo autor

A biblioteca de layouts, apresentada na Figura 12, reúne em uma única tela os layouts disponíveis ao usuário, cada um identificado pelo nome que ele lhe deu e acompanhado do atalho para edição.

Figura 12 – Esquema da biblioteca de layouts



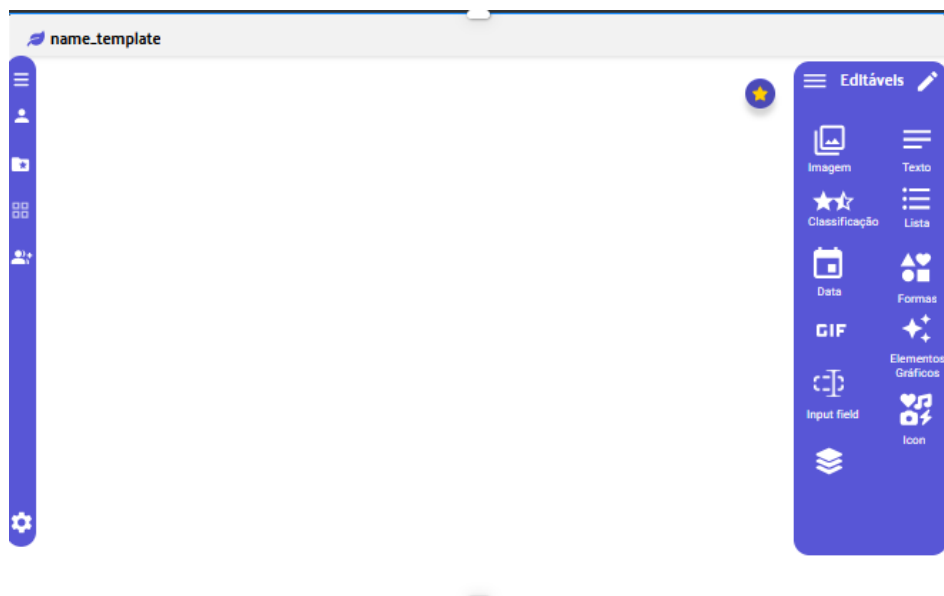
Fonte: Elaborada pelo autor

O editor de *layouts* recebeu tratamento próprio, por ser a tela em que a proposta da plataforma se materializa. Divide-se entre uma área de edição, que representa a ficha em construção, e um painel lateral com os tipos de Elemento disponíveis. O usuário acrescenta a peça à área de edição pelo painel e, uma vez ali, posiciona e redimensiona o Elemento livremente, escrevendo seu conteúdo sobre ele próprio. O editor não possui botão de gravação: cada alteração concluída é gravada automaticamente, cabendo ao desfazer e ao refazer o papel de reverter uma alteração indesejada, e ao aviso de saída o de alertar quando ainda há alteração por gravar. Também não há alternância entre modos de edição e de prévia: a própria área de edição desenha a ficha como ela aparecerá em modo leitura, de modo que cada ajuste é conferido no lugar em que é feito.

As Figuras 13 e 14 mostram as duas situações do editor. Na primeira, a área de edição está vazia, como no início da montagem de um layout; na segunda,

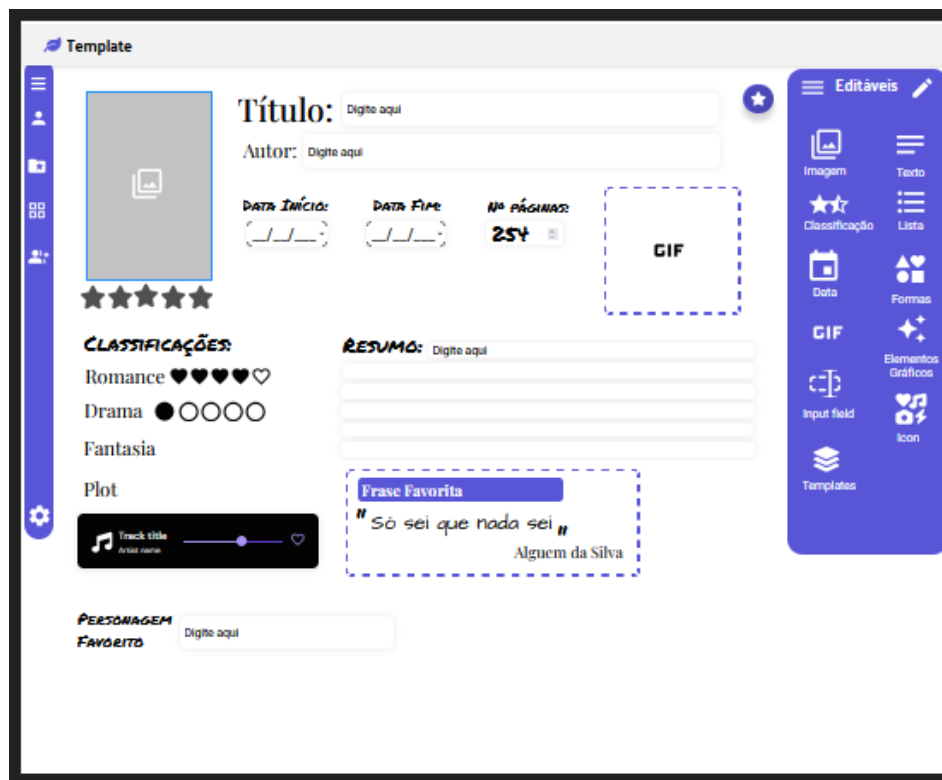
ela exibe uma ficha de livro já composta, com Elementos de imagem, texto, data e classificação dispostos livremente. Cabe observar que o painel lateral desses esquemas reflete o conjunto de peças cogitado na concepção, anterior à definição dos seis tipos consolidados no Capítulo 4.

Figura 13 – Esquema do editor de layout com a área de edição vazia



Fonte: Elaborada pelo autor

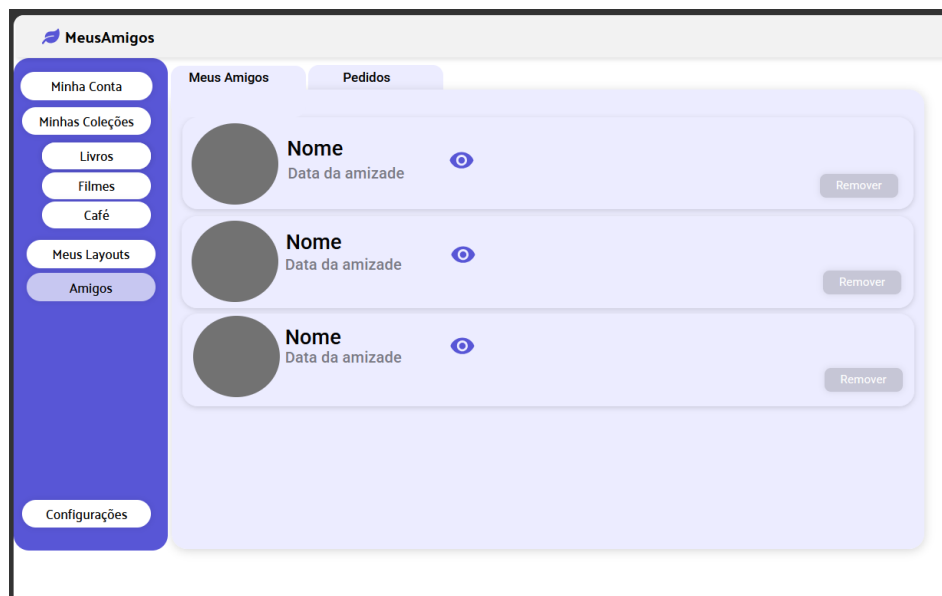
Figura 14 – Esquema do editor de layout com uma ficha composta



Fonte: Elaborada pelo autor

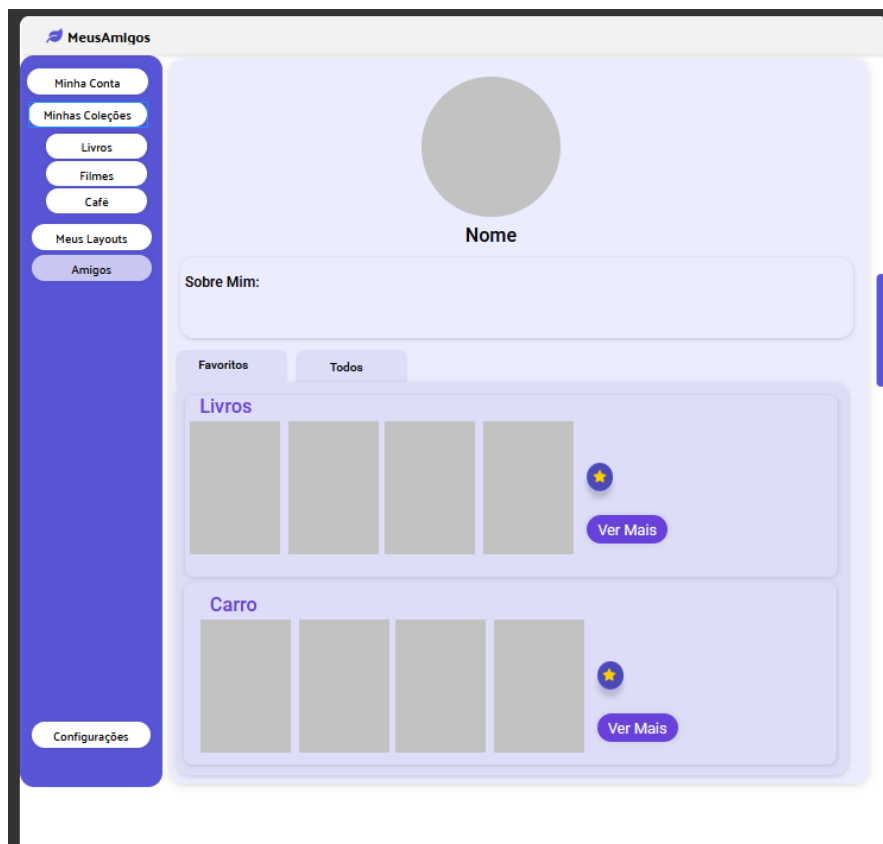
As telas de interação social seguem a mesma moldura. A área de amigos, na Figura 15, separa em abas os vínculos já estabelecidos e as solicitações pendentes, e oferece a busca por código de usuário ou por nome prevista no RF-20. No perfil de outro usuário, na Figura 16, exibem-se os dados públicos e as coleções visíveis conforme o vínculo existente, cada uma acompanhada da ação de seguir ou deixar de seguir.

Figura 15 – Esquema da tela de amigos



Fonte: Elaborada pelo autor

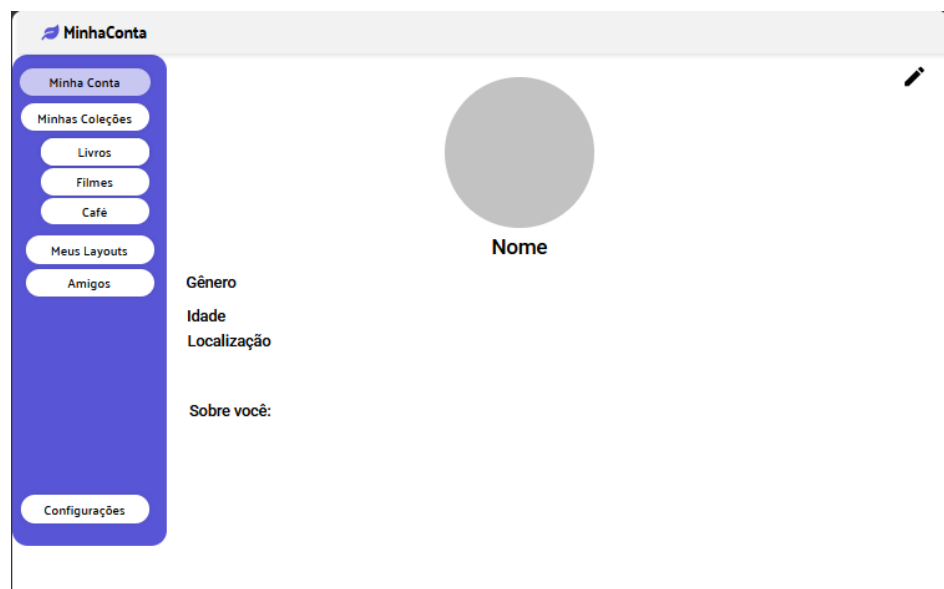
Figura 16 – Esquema do perfil de outro usuário



Fonte: Elaborada pelo autor

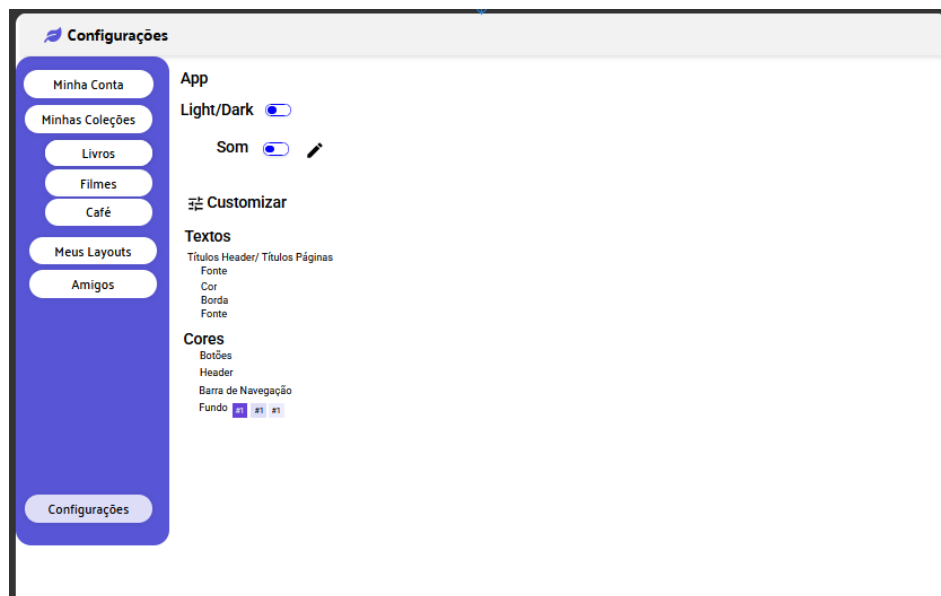
Completam o conjunto as telas de conta e de aparência. O perfil do próprio usuário, na Figura 17, reúne a foto, o nome e os demais dados de identificação, com o atalho para edição no canto superior. A tela de configurações, na Figura 18, concentra a personalização visual global prevista nos RF-18 e RF-19: a alternância entre os temas claro e escuro e o ajuste de tipografia e de cores da interface. Na especificação consolidada no Capítulo 4, essa personalização passou a se organizar como uma biblioteca de paletas, cada uma com cores e tipografia próprias, e a escolha do estilo claro, escuro ou automático foi deslocada para o menu da conta; o esquema registra a concepção inicial.

Figura 17 – Esquema do perfil do próprio usuário



Fonte: Elaborada pelo autor

Figura 18 – Esquema da tela de configurações de aparência



Fonte: Elaborada pelo autor

5.2 Projeto de Dados

A persistência de informações utiliza o Sistema Gerenciador de Banco de Dados (SGBD) **PostgreSQL 16**, executado em ambiente isolado através de **Docker**. A manipulação e a administração dos dados são realizadas por meio do **DBeaver**, cliente gráfico de administração de bancos de dados relacionais.

O banco de dados foi estruturado para suportar tipos de dados modernos e performáticos, incluindo:

- **UUID (Universally Unique Identifier):** Utilizado para todas as chaves primárias e estrangeiras;
- **JSONB:** Para o armazenamento eficiente de configurações dinâmicas de interface;
- **Varchar(n):** Para cadeias de caracteres de tamanho limitado, com restrição imposta no nível do banco;
- **Text:** Para cadeias de caracteres extensas, sem limite explícito de tamanho;

- **Timestamp:** Para o registro cronológico de eventos e criações;
- **Float:** Para valores numéricos fracionários, utilizados em ajustes tipográficos;
- **Boolean:** Para o registro de estados binários (verdadeiro/falso);
- **Enum:** Para campos com domínio restrito a um conjunto pré-definido de valores.

5.2.1 Mapeamento Objeto-Relacional

O mapeamento entre o modelo de domínio (TypeScript) e o modelo relacional (PostgreSQL) é gerenciado pelo **Prisma ORM**. Para garantir a integridade dos dados e a fidelidade aos requisitos, adotaram-se as seguintes regras de mapeamento:

- **Idioma dos identificadores:** Os nomes conceituais empregados no texto desta monografia estão em português (Elemento, EstiloDeElemento, Tipo-Elemento, Categoria, Coleção), ao passo que os identificadores do código e do banco permanecem em inglês (`Element`, `ElementStyle`, `ElementType`, `categories`, `collections`). A correspondência entre uns e outros é direta, e adotou-se essa separação para que o texto permaneça legível em português sem divergir dos nomes efetivamente encontrados no código-fonte;
- **Nomenclatura:** As entidades em *camelCase* no código são mapeadas para tabelas em *snake_case* no banco de dados através da diretiva `@@map`;
- **Tipos Físicos:** Os atributos do tipo `String` no Prisma são explicitamente mapeados para os tipos físicos `VARCHAR(n)` ou `TEXT` no PostgreSQL através das diretivas `@db.VarChar(n)` e `@db.Text`, conforme o tamanho esperado do conteúdo;
- **Chaves Primárias:** Todos os modelos possuem um identificador único do tipo `UUID` gerado automaticamente;
- **Relações 1:N:** Implementadas via chaves estrangeiras (*foreign keys*) obrigatórias, com integridade referencial garantida por regras de *cascade* ou *set null* conforme a semântica da relação;

- **Relações N:N:** Implementadas via tabela associativa, como `collection_follows`, que vincula usuários às coleções que acompanham e é protegida por chave única composta, impedindo a duplicação do par, e `_CollectionToTag`, que liga as coleções às suas tags;
- **Relações Reflexivas:** Aparecem em dois contextos. Nas interações sociais, a amizade é mapeada como relação nomeada, para distinguir os papéis de remetente e destinatário. No editor de *layouts*, tanto `layouts` quanto `elements` referenciam a própria tabela para registrar de qual registro a cópia se originou, com regra *set null*, de modo que excluir o original apenas zera a referência e preserva a cópia;
- **Índices:** Foram criados índices secundários sobre as chaves estrangeiras de maior frequência de consulta, visando a otimização das operações de leitura.

Cabe registrar que nem toda regra de unicidade prevista nos requisitos está expressa como restrição do banco. As unicidades de nome de categoria e de nome de *layout*, ambas por usuário, são garantidas por chave única. Já a proibição de dois itens homônimos na mesma coleção e de duas coleções homônimas na mesma categoria é verificada na camada de lógica de negócio, antes da escrita. A diferença decorre da ordem em que os requisitos foram implementados e não altera o comportamento observado pelo usuário, mas convém explicitá-la, pois a leitura isolada do esquema não a revelaria.

5.2.2 Estrutura das Tabelas no Banco de Dados

Nesta seção, descreve-se o padrão adotado para a nomenclatura dos objetos do banco de dados, incluindo chaves primárias, chaves estrangeiras e índices únicos. A padronização visa garantir a manutenibilidade do sistema e a integridade referencial. Foi convencionado que os nomes dos objetos devem obedecer aos padrões definidos na Tabela 20.

Tabela 20 – Convenção para Nome dos Objetos no Banco de Dados

Objeto	Padrão Adotado (com exemplo)
Chave Primária	nome_da_tabela_pkey (ex.: users_pkey)
Chave Estrangeira	nome_da_tabela_nome_do_campo_fkey (ex.: elements_layout_id_fkey)
Chave Única (Unique)	nome_da_tabela_nome_do_campo_key (ex.: users_email_key)
Índices	nome_da_tabela_nome_do_campo_idx (ex.: elements_layout_id_idx)

Fonte: Elaborada pelo autor

Para melhor compreensão da arquitetura de dados, as 13 tabelas propostas neste trabalho estão consolidadas na Tabela 21, que serve como índice para o detalhamento físico apresentado na subseção seguinte. A ordem de apresentação acompanha os pacotes do modelo de domínio definidos no Capítulo 4.

Tabela 21 – Tabelas Identificadas neste Trabalho

Tabela do Banco de Dados	Referência no Documento
users	Tabela 22
user_sessions	Tabela 23
password_reset_tokens	Tabela 24
categories	Tabela 25
layouts	Tabela 26
collections	Tabela 27
tags	Tabela 28
elements	Tabela 29
items	Tabela 30
notifications	Tabela 31
event_logs	Tabela 32
friendships	Tabela 33
collection_follows	Tabela 34
appearance_presets	Tabela 35

Fonte: Elaborada pelo autor

5.2.2.1 Detalhamento da Estrutura das Tabelas

Nesta subseção, apresenta-se o detalhamento físico das 13 tabelas que compõem o banco de dados do sistema, especificando tipos de dados, chaves e restrições. Para a leitura das tabelas de detalhamento, foram adotadas as seguintes convenções de cabeçalho:

- **Campo:** Nome técnico do atributo mapeado na tabela do banco de dados;
- **Tipo:** Tipo de dado nativo do PostgreSQL. Para cadeias de caracteres, indica-se *Varch.(n)* para campos com limite, onde *n* é o tamanho máximo, ou *Text* para campos sem restrição;
- **Obr.?:** Indica se o campo é de preenchimento obrigatório (X) ou opcional (vazio);
- **PK?:** Indica se o campo compõe a Chave Primária (*Primary Key*) da tabela;
- **Tab.:** Em caso de Chave Estrangeira (*Foreign Key*), indica a tabela de referência;
- **Cmp.:** Indica o campo específico de referência na tabela estrangeira;
- **Grp.:** Identifica o grupo de Chave Única (*Unique Key*) ao qual o campo pertence (ex: UK_01);
- **Ord.:** Define a sequência do campo dentro de uma composição de chave única ou índice.

O campo `user_code` merece destaque: trata-se do identificador público do usuário, gerado automaticamente no cadastro (RF-01) e imutável, utilizado nas buscas e nas solicitações de amizade (RF-20). Sua unicidade é garantida em toda a plataforma, de modo independente da unicidade do endereço de e-mail.

Já o `default_layouts_at` registra o momento em que o usuário recebeu os *layouts* padrão oferecidos pelo sistema (RF-01), e não é registro histórico: existe para que a entrega aconteça uma única vez. Contas criadas antes dessa funcionalidade recebem os modelos na primeira abertura da biblioteca, e a marca impede que

Tabela 22 – Usuário (users)

Campo	Tipo	Obr.?	PK?	Tab.	Cmp.	Grp.	Ord.
id_user	UUID	X	X				
user_name	Varch.(100)	X					
user_code	Varch.(110)	X				UK_01	1
email	Varch.(150)	X				UK_02	1
password	Varch.(255)	X					
profile_photo	Varch.(500)						
bio	Varch.(500)						
registered_at	Time.	X					
default_layouts_at	Time.						

Fonte: Elaborada pelo autor

um *layout* padrão excluído de propósito reapareça na listagem seguinte — o que aconteceria caso a verificação se apoiasse na simples presença dos modelos pelo nome.

Tabela 23 – Sessão de Usuário (user_sessions)

Campo	Tipo	Obr.?	PK?	Tab.	Cmp.	Grp.	Ord.
id_session	UUID	X	X				
created_at	Time.	X					
revoked_at	Time.						
user_id	UUID	X		users	id_user		

Fonte: Elaborada pelo autor

Esta tabela é o que torna possível revogar um acesso antes de seu prazo de expiração. O identificador da sessão viaja no *token* emitido no login, e a cada requisição autenticada o sistema confere se o registro correspondente ainda tem **revoked_at** nulo. Sem esse controle, um *token* permaneceria válido até expirar, e o *logout* seria apenas uma formalidade do lado do navegador (RF-02).

Sustenta o fluxo de recuperação de senha (RF-04). Armazena-se o *hash* do *token*, e não seu valor original, de modo que o acesso à base não permita forjar um link de redefinição válido. Os campos **expires_at** e **used_at** implementam, respectivamente, o prazo de validade e o uso único exigidos pelo requisito.

Tabela 24 – Token de Redefinição de Senha (password_reset_tokens)

Campo	Tipo	Obr.?	PK?	Tab.	Cmp.	Grp.	Ord.
id_password_reset_token	UUID	X	X				
token_hash	Varch.(64)	X				UK_01	1
created_at	Time.	X					
expires_at	Time.	X					
used_at	Time.						
user_id	UUID	X		users	id_user		

Fonte: Elaborada pelo autor

Tabela 25 – Categoria (categories)

Campo	Tipo	Obr.?	PK?	Tab.	Cmp.	Grp.	Ord.
id_category	UUID	X	X				
name	Varch.(150)	X				UK_01	1
type	Varch.(50)						
description	Text						
icon	Varch.(500)						
cover_image	Varch.(500)						
cover_original	Varch.(500)						
cover_crop_x	Int						
cover_crop_y	Int						
cover_crop_width	Int						
cover_crop_height	Int						
background_color	Varch.(7)						
created_at	Time.	X					
user_id	UUID	X		users	id_user	UK_01	2

Fonte: Elaborada pelo autor

Alguns campos desta tabela envolvem decisões que a leitura isolada do esquema não revelaria. O `icon` guarda o emoji da categoria, exibido sobre o banner do cartão e no modo lista. A imagem do banner tem colunas próprias porque, na categoria, o emoji e a imagem coexistem (RF-06): o `cover_image` guarda o recorte feito pelo usuário, já na proporção do banner, que é o que os cartões carregam, e o `cover_original` guarda o arquivo enviado, intacto, que nenhuma tela exibe. Os quatro campos `cover_crop_*` registram a área recortada, em pixels do original. É esse par, original e área, que permite reabrir o enquadramento onde ele estava e refazê-lo sem novo envio; os campos da área ficam nulos quando a capa

é anterior ao recorte, e nesse caso o enquadramento reabre centralizado. As colunas de arquivo têm 500 caracteres porque guardam a URL pública completa do arquivo armazenado.

Já o `background_color` guarda a cor em notação hexadecimal, e não como índice de uma paleta do sistema. A consequência é que a cor pertence ao usuário, e não ao tema: ela permanece a mesma na alternância entre os modos claro e escuro, comportamento adotado também por ferramentas como o Notion e o Trello nas cores de etiqueta. Como o nome da categoria assenta sobre o banner, seja sobre a cor, seja sobre a capa, a legibilidade fica a cargo da interface: um escurecimento suave na metade inferior do banner e uma sombra discreta no texto branco garantem a leitura sobre qualquer fundo (RNF-05), sem desenhar uma faixa sobre a imagem.

Tabela 26 – Layout Customizado (layouts)

Campo	Tipo	Obr.?	PK?	Tab.	Cmp.	Grp.	Ord.
id_layout	UUID	X	X				
name	Varch.(150)	X				UK_01	2
description	Text						
purpose	Enum	X					
background	Varch.(24)						
created_at	Time.	X					
source_layout_id	UUID			layouts	id_layout		
user_id	UUID	X		users	id_user	UK_01	1

Fonte: Elaborada pelo autor

Três campos desta tabela sustentam requisitos do Editor de Layout. O **purpose** distingue as três finalidades que um *layout* pode ter: *template* reutilizável, criado pelo próprio usuário; *template* importado, resultante do compartilhamento (RF-16); e cópia individual de item, gerada quando o usuário personaliza a ficha de um item específico (RF-17). Já o `source_layout_id` registra de qual *layout* a cópia se originou, tanto na duplicação (RF-14) quanto na importação, sem criar dependência entre os dois: a regra *set null* faz com que excluir o original apenas zere a referência.

Por fim, o `background` guarda o fundo da ficha (RF-14): uma cor hexadecimal ou um degradê de duas cores com ângulo e ponto de início, gravado em formato compacto e validado, e nunca como CSS livre, o que abriria espaço para injeção de

estilo na página de quem visita a ficha. É coluna do *layout*, e não propriedade de um Elemento, porque não pertence a nenhum deles: é a superfície sobre a qual todos são desenhados. Como a ficha própria de um item também é um registro desta tabela (RF-17), a mesma coluna atende aos dois casos, e o valor nulo corresponde ao fundo branco padrão.

Como `user_id` é obrigatório, um *layout* não existe sem dono, e é isso que define onde vivem os *layouts* padrão oferecidos pelo sistema: eles são criados na biblioteca de cada usuário no momento do cadastro (RF-01), com a finalidade de *template* reutilizável, do mesmo modo que as paletas predefinidas de aparência (RF-18). A partir daí comportam-se como qualquer outro *layout* do usuário, podendo ser associados a coleções, editados ou duplicados.

Apenas os *layouts* de finalidade *template*, próprios ou importados, aparecem na biblioteca e podem ser associados a coleções. As cópias individuais de item ficam de fora dessa listagem, pois pertencem à ficha de um item específico e não fazem sentido como modelo a reutilizar.

Tabela 27 – Coleção (collections)

Campo	Tipo	Obr.?	PK?	Tab.	Cmp.	Grp.	Ord.
id_collection	UUID	X	X				
name	Varch.(150)	X					
description	Text						
privacy	Enum	X					
icon	Varch.(500)						
icon_original	Varch.(500)						
icon_crop_x	Int						
icon_crop_y	Int						
icon_crop_width	Int						
icon_crop_height	Int						
background_color	Varch.(7)						
created_at	Time.	X					
category_id	UUID	X		categories	id_cat.		
layout_id	UUID			layouts	id_lay.		

Fonte: Elaborada pelo autor

Na coleção, o ícone é emoji **ou** imagem, e escolher uma forma descarta a outra (RF-07). Por isso, ao contrário da categoria, não há coluna própria para a imagem:

o recorte quadrado feito pelo usuário fica no próprio `icon`, e o `icon_original` e os campos `icon_crop_*` cumprem o mesmo papel que têm na capa da categoria, permitindo refazer o enquadramento sem novo envio.

Tabela 28 – Tag de Coleção (tags)

Campo	Tipo	Obr.?	PK?	Tab.	Cmp.	Grp.	Ord.
<code>id_tag</code>	UUID	X	X				
<code>name</code>	Varch.(30)	X				UK_01	1
<code>created_at</code>	Time.	X					

Fonte: Elaborada pelo autor

A tabela `tags` é o vocabulário compartilhado do RF-07: o `name` é único no sistema inteiro, e não por usuário, de modo que a mesma palavra ocupa uma única linha, qualquer que seja a conta que a escreveu. É isso que permite ao formulário sugerir as tags já existentes. O nome é gravado já na forma canônica (minúsculas, sem espaços nas pontas e sem acentuação), pela mesma rotina que normaliza o tema da categoria; gravar a forma original quebraria a unicidade, pois ‘Fantasia’ e ‘fantasia’ seriam duas linhas. O limite de 30 caracteres, menor que os 50 do tema, reflete o fato de a tag ser um rótulo curto, e várias delas dividirem a largura de um cartão.

A ligação entre coleções e tags é muitos-para-muitos e fica na tabela associativa `_CollectionToTag`, criada e mantida pelo próprio Prisma, com as colunas A (referência a `collections`) e B (referência a `tags`). É o único nome do esquema fora do padrão *snake_case*, justamente por ser gerado pelo ORM; em troca, as tags de uma coleção são lidas diretamente como lista, sem um nível intermediário de objetos. Excluir uma coleção desfaz apenas a ligação: a tag permanece no vocabulário, ao contrário do tema, que é derivado das categorias existentes e desaparece com a última que o utilizava.

Esta é a tabela central do diferencial da plataforma. Nela residem a geometria de cada Elemento (`pos_x`, `pos_y`, `width` e `height`), a camada em que ele se sobrepõe aos demais (`layer`), o conteúdo exibido (`content`) e toda a sua aparência (`style`). Essa concentração é o que permite tratar o Elemento como objeto visual editável, à maneira de um editor gráfico, em vez de separar a estrutura do dado que a preenche.

Tabela 29 – Elemento do Layout (elements)

Campo	Tipo	Obr.?	PK?	Tab.	Cmp.	Grp.	Ord.
id_element	UUID	X	X				
name	Varch.(100)	X					
type	Enum	X					
content	Text						
pos_x	Int	X					
pos_y	Int	X					
width	Int	X					
height	Int	X					
layer	Int	X					
style	JSONB						
source_element_id	UUID			elements	id_element		
layout_id	UUID	X		layouts	id_layout		

Fonte: Elaborada pelo autor

O campo **type** corresponde à enumeração **TipoElemento**, com os seis valores fixos definidos no Capítulo 4.

A coluna **style** corresponde ao **EstiloDeElemento** do modelo de domínio, apresentado na Seção 4.4. Ali ele aparece como uma classe de valor composta no **Elemento**, com uma propriedade visual por atributo, o que torna visível o que o editor oferece; aqui ele é gravado como um único documento **JSONB**. A correspondência é direta, e os nomes das propriedades no documento são os mesmos atributos da classe.

A divisão entre coluna e **JSONB** obedece a um critério: é coluna aquilo que o banco precisa consultar, ordenar ou indexar, e é **JSONB** aquilo que ele apenas guarda e devolve, cabendo ao navegador interpretar. Por esse critério, **content** é coluna, pois a ordenação de itens por classificação (RF-12) lê o valor dali, ao passo que a cor de fundo de um **Elemento** não é: nenhuma consulta pergunta quais **Elementos** são azuis. Uma coluna por propriedade traria o inconveniente oposto: o editor cresce em propriedades visuais, e cada nova opção exigiria alterar o esquema do banco. Ser **JSONB** não afrouxa a validação: o objeto é validado atributo a atributo na camada de aplicação, e uma chave desconhecida faz a requisição ser rejeitada como qualquer outra.

A geometria é expressa em inteiros porque a unidade não é o *pixel*, e sim a milésima parte da largura do quadro da ficha. Como a ficha ocupa a largura disponível e escala junto com ela, uma medida absoluta significaria coisas diferentes em cada tela; com a unidade relativa, mudar a largura disponível altera apenas a escala, e nenhum Elemento se reposiciona.

O campo **layer** não acompanha **pos_x** e **pos_y** como uma terceira coordenada, e por isso não se chama **pos_z**: a ficha é plana, e o que ele determina é a ordem de pintura entre Elementos que ocupam a mesma região. Sem ele, essa ordem viria da ordem de retorno da consulta, que o SGBD não garante, e dois Elementos sobrepostos poderiam trocar de posição entre um carregamento e outro.

Tabela 30 – Item da Coleção (items)

Campo	Tipo	Obr.?	PK?	Tab.	Cmp.	Grp.	Ord.
id_item	UUID	X	X				
name	Varch.(255)	X					
thumbnail	Varch.(500)						
cover_image	Varch.(500)						
background_color	Varch.(7)						
notes	Text						
created_at	Time.	X					
collection_id	UUID	X		collections	id_coll.		
layout_id	UUID			layouts	id_layout		

Fonte: Elaborada pelo autor

O item é a única das três entidades que recebe cor de fundo sem receber **icon**: já dispõe de **thumbnail** e **cover_image** para imagem, e um terceiro campo de imagem seria repetição. A cor atende justamente ao item sem miniatura, que de outro modo apareceria como um retângulo vazio na listagem da coleção.

A tabela distingue dois papéis de usuário: **user_id** é o destinatário da notificação e **author_id** é quem originou o evento, campo que permite compor mensagens como “João Silva enviou uma solicitação de amizade”. As chaves **friendship_id** e **event_log_id** vinculam a notificação ao fato que a gerou, conforme o tipo. Quanto ao estado de leitura, **is_read** indica a situação atual, **read_at** registra o instante em que a leitura ocorreu e **cleared_at** marca as notificações que o usuário optou

Tabela 31 – Notificações (notifications)

Campo	Tipo	Obr.?	PK?	Tab.	Cmp.	Grp.	Ord.
id_notification	UUID	X	X				
message	Text	X					
type	Enum	X					
is_read	Bool	X					
read_at	Time.						
cleared_at	Time.						
created_at	Time.	X					
user_id	UUID	X		users	id_user		
author_id	UUID			users	id_user		
friendship_id	UUID			friendships	id_friendship		
event_log_id	UUID			event_logs	id_event_log		

Fonte: Elaborada pelo autor

por limpar da listagem, ocultando-as de sua área sem excluir o registro, o que preserva o histórico (RF-24).

Tabela 32 – Log de Eventos (event_logs)

Campo	Tipo	Obr.?	PK?	Tab.	Cmp.	Grp.	Ord.
id_event_log	UUID	X	X				
event_type	Enum	X					
item_name	Varch.(255)	X					
description	Text						
created_at	Time.	X					
user_id	UUID	X		users	id_user		
collection_id	UUID	X		collections	id_coll.		
item_id	UUID			items	id_item		

Fonte: Elaborada pelo autor

Esta tabela alimenta o *feed* de atualizações (RF-22). O vínculo com a coleção é obrigatório, pois é por ele que se determina quais usuários devem receber o evento; já o vínculo com o item é opcional e adota a regra *set null*. A razão está no campo **item_name**: como eventos de remoção precisam permanecer legíveis depois que o item deixa de existir, guarda-se o nome do item no próprio registro do evento. Assim, excluir um item zera a referência sem apagar o histórico, e o *feed* continua exibindo a informação de forma compreensível. Os campos estruturados são a fonte primária dos dados apresentados; **description** guarda apenas texto auxiliar.

Tabela 33 – Relacionamentos (friendships)

Campo	Tipo	Obr.?	PK?	Tab.	Cmp.	Grp.	Ord.
id_friendship	UUID	X	X				
pair_key	Varch.(73)	X				UK_01	1
status	Enum	X					
sent_at	Time.	X					
responded_at	Time.						
sender_id	UUID	X		users	id_user		
receiver_id	UUID	X		users	id_user		

Fonte: Elaborada pelo autor

A unicidade desta tabela recai sobre **pair_key** e não sobre o par **sender_id**–**receiver_id**, e a diferença é deliberada. Uma chave única composta pelas duas colunas impediria a repetição da mesma solicitação, mas não a solicitação inversa: se A convida B e, antes da resposta, B convida A, os dois registros seriam aceitos, e passariam a existir dois vínculos entre as mesmas duas pessoas. O **pair_key** resolve

isso ao combinar os dois identificadores em uma ordem fixa, independentemente de quem enviou, de modo que qualquer par de usuários corresponda a um único valor possível.

Tabela 34 – Seguidores de Coleção (collection_follows)

Campo	Tipo	Obr.?	PK?	Tab.	Cmp.	Grp.	Ord.
id_follow_collection	UUID	X	X				
is_active	Bool	X					
started_at	Time.	X					
ended_at	Time.						
user_id	UUID	X		users	id_user	UK_01	1
collection_id	UUID	X		collections	id_coll.	UK_01	2

Fonte: Elaborada pelo autor

Tabela 35 – Preferências de Aparência (appearance_presets)

Campo	Tipo	Obr.?	PK?	Tab.	Cmp.	Grp.	Ord.
id_appearance_preset	UUID	X	X				
name	Varch.(50)	X				UK_01	2
primary_color	Varch.(9)	X					
subcolors	JSONB	X					
system_theme	Enum	X					
font_family	Varch.(100)	X					
title_size	Float	X					
text_size	Float	X					
line_height	Float	X					
letter_spacing	Float	X					
font_weight	Int	X					
font_italic	Bool	X					
is_active	Bool	X					
is_default	Bool	X					
created_at	Time.	X					
updated_at	Time.	X					
user_id	UUID	X		users	id_user	UK_01	1

Fonte: Elaborada pelo autor

Cada paleta pertence a um usuário, e seu nome não se repete entre as paletas da mesma conta, o que a chave única composta por `user_id` e `name` garante; usuários diferentes podem ter paletas de mesmo nome. O campo `subcolors`

emprega o tipo JSONB para guardar as doze subcores em ordem fixa, na qual cada posição corresponde a um papel na interface: fundo da página, cartões, títulos, texto nos cartões, barra lateral, cabeçalho e modais, seguidos do texto no fundo da página, na barra lateral, no cabeçalho, nos modais e sobre o botão de destaque. As cinco últimas foram acrescentadas ao fim da lista, para que as sete primeiras não mudassem de posição. A quantidade exata é garantida pela validação na camada de aplicação, pois uma paleta é sempre completa (RF-18). O `system_theme` guarda o estilo em vigor enquanto a paleta está em uso: nasce do lado para o qual ela foi desenhada e é atualizado quando o usuário altera o estilo no menu da conta. A tipografia (`font_family`, `title_size`, `text_size`, `line_height`, `letter_spacing`, `font_weight` e `font_italic`) também é gravada em cada paleta (RF-19). Os dois indicadores booleanos têm papéis distintos: `is_active` aponta a paleta em uso, e um índice único parcial garante uma única por usuário; `is_default` indica apenas que a paleta foi oferecida pelo sistema no cadastro (RF-01), sem protegê-la de edição ou exclusão. No cadastro, fica ativa a paleta ‘Claro’ ou a ‘Escuro’, conforme o tema do sistema operacional naquele momento, e a conta nunca fica sem paleta porque a paleta em uso não pode ser excluída.

6 IMPLEMENTAÇÃO

Este capítulo descreve como o sistema especificado nos Capítulos 4 e 5 foi construído. São apresentados o ambiente de desenvolvimento, o servidor de dados, o servidor de aplicação, a organização do código do *backend* e da aplicação cliente, a forma como a implementação foi verificada ao longo do desenvolvimento e as limitações conhecidas da versão atual. As ferramentas empregadas já foram caracterizadas na Seção 2.3; aqui o foco recai sobre as versões adotadas, a configuração do ambiente e os mecanismos implementados.

6.1 Ambiente de Desenvolvimento

O sistema é composto por dois projetos independentes, versionados em repositórios Git distintos: o *backend*, uma API REST escrita em TypeScript sobre o NestJS, e o *frontend*, uma aplicação de página única escrita em TypeScript com React. Ambos são executados sobre o Node.js e têm suas dependências gerenciadas pelo pnpm, cuja versão fica registrada no próprio arquivo `package.json` de cada projeto, de modo que qualquer máquina que instale as dependências utilize a mesma versão do gerenciador. O Quadro 42 reúne as principais tecnologias e as versões empregadas.

Durante o desenvolvimento, o sistema é executado localmente por três processos: o contêiner do PostgreSQL, na porta 5432; a API, na porta 3000; e o servidor de desenvolvimento do Vite, na porta 3001. A porta do *frontend* é fixada na configuração do Vite por uma razão funcional, e não de conveniência: é a origem que a API autoriza a realizar requisições pelo navegador, e servir a interface em outra porta faria o navegador bloquear todas as chamadas por violação da política de CORS.

Quadro 42 – Tecnologias e versões utilizadas na implementação

Camada	Tecnologia e versão
Ambiente de execução	Node.js 22.15 e gerenciador de pacotes pnpm 11.20
Linguagem	TypeScript 5.7 no <i>backend</i> e 6.0 no <i>frontend</i>
Servidor de aplicação	NestJS 11, com Passport e JWT para autenticação, class-validator para validação, @nestjs/throttler para limitação de requisições e Nodemailer para envio de e-mail
Persistência	PostgreSQL 16, executado em contêiner Docker, e Prisma ORM 7.8 com o adaptador para o <i>driver pg</i>
Armazenamento de arquivos	Disco local ou Cloudflare R2, este acessado pelo SDK S3 da AWS
Interface	React 19.2, Vite 8, React Router 8, TanStack Query 5 e Axios 1.19
Testes	Jest 30 com ts-jest
Qualidade de código	ESLint e Prettier no <i>backend</i> ; Oxlint no <i>frontend</i>

Fonte: Elaborada pelo autor

A configuração do *backend* é feita por variáveis de ambiente, lidas de um arquivo `.env`. O repositório inclui o modelo `.env.example`, que relaciona as variáveis sem seus valores: a cadeia de conexão com o banco (`DATABASE_URL`), o segredo de assinatura dos *tokens* (`JWT_SECRET`), o endereço do *frontend* (`FRONTEND_URL`), os parâmetros do servidor de e-mail (`SMTP_*`) e as credenciais do armazenamento em nuvem (`R2_*`). Manter esses valores fora do código evita que credenciais sejam versionadas e permite que o mesmo código seja executado em ambientes diferentes apenas com a troca do arquivo de configuração.

6.2 Servidor de Dados

O PostgreSQL é provisionado pelo Docker Compose, a partir de um arquivo `docker-compose.yml` mantido no repositório do *backend*. O arquivo declara um único serviço, baseado na imagem oficial `postgres:16`, que expõe a porta 5432 e grava os dados em um volume nomeado. O volume desacopla os dados do ciclo de vida do contêiner: recriar ou atualizar o contêiner não apaga o banco. O serviço conta ainda com uma verificação de saúde que executa o utilitário `pg_isready`

periodicamente, o que permite saber quando o banco está de fato pronto para aceitar conexões, e não apenas quando o contêiner foi iniciado.

O esquema do banco é definido com o Prisma e fica dividido em sete arquivos no diretório `prisma/schema`: um arquivo principal e seis arquivos por assunto (usuário, coleções, *layout*, aparência, interação social e notificações), que acompanham os pacotes do modelo de domínio apresentados no Capítulo 4. Cada alteração no esquema gera uma migração versionada, contendo o SQL correspondente; no momento da redação deste trabalho, o histórico reúne 36 migrações, que reconstroem o banco do zero de forma determinística. Quando uma mudança de modelo afetava dados já existentes, a migração foi escrita para convertê-los em vez de descartá-los. Foi o caso da redução dos tipos de Elemento: como o PostgreSQL não permite remover valores de um tipo enumerado, a migração cria o novo tipo, converte a coluna e só então descarta o tipo antigo, preservando o conteúdo de todos os Elementos.

O cliente do Prisma, que oferece acesso tipado às tabelas, é gerado automaticamente a cada instalação de dependências, por meio do *script* `postinstall`. Na aplicação, ele é encapsulado em um serviço do NestJS (`PrismaService`), injetado nos módulos que acessam o banco. As operações que precisam gravar vários registros de forma indivisível são executadas em transações. É o caso da criação da ficha própria de um item (RF-17), que copia o *layout* de origem e todos os seus Elementos: ou a cópia é criada por inteiro, ou nada é gravado. O mapeamento entre o modelo de objetos e as tabelas está detalhado no Capítulo 5.

6.3 Servidor de Aplicação

A API é uma aplicação NestJS que atende na porta 3000, sem prefixo de rota. No momento da redação, ela é composta por 16 controladores, que expõem 81 rotas: 27 de consulta (GET), 24 de criação (POST), 15 de alteração parcial (PATCH), uma de substituição (PUT) e 14 de exclusão (DELETE). Toda requisição percorre uma sequência fixa de etapas antes de alcançar a regra de negócio, configurada nos arquivos `main.ts` e `app.module.ts`:

1. **CORS:** somente a origem informada em `FRONTEND_URL` é autorizada a chamar a API pelo navegador.
2. **Limitação de requisições:** um *guard* global limita cada endereço de origem a 100 requisições por minuto. As rotas sensíveis sobrescrevem esse limite: o *login* aceita cinco tentativas por minuto, e a solicitação de recuperação de senha, três a cada dez minutos, já que cada chamada dispara um e-mail.
3. **Autenticação:** um segundo *guard* global exige um *token* JWT válido em todas as rotas. Uma rota só fica aberta quando é marcada explicitamente como pública, o que faz com que o esquecimento dessa marcação resulte em uma rota fechada, e não em uma rota exposta sem que ninguém perceba.
4. **Validação da sessão:** além de verificar a assinatura e a validade do *token*, a estratégia de autenticação consulta a tabela de sessões e confirma que a sessão indicada no *token* não foi revogada. Com isso, o *logout* tem efeito imediato: um *token* copiado antes da saída deixa de funcionar no mesmo instante, ao custo de uma consulta ao banco por requisição autenticada.
5. **Validação dos dados:** o corpo, os parâmetros e a consulta de cada requisição são validados contra classes de transferência de dados (DTOs). A validação é estrita: além de rejeitar valores inválidos, ela rejeita qualquer campo não previsto, o que torna visível uma divergência de contrato entre *frontend* e *backend* no momento em que ela surge.
6. **Regra de negócio:** validada a requisição, o controlador repassa a operação ao serviço do módulo, que aplica as regras do domínio e acessa o banco pelo Prisma.

A ordem entre as etapas 2 e 3 é deliberada. Como os *guards* executam na ordem em que são registrados, a limitação de requisições atua antes da autenticação, de modo que um volume excessivo de chamadas é barrado antes de consumir a consulta ao banco que valida a sessão, que é justamente o recurso que se deseja proteger. Os erros produzidos em qualquer etapa são convertidos em respostas padronizadas por um filtro de exceções global, com mensagens em português que a interface exhibe diretamente ao usuário.

6.3.1 Armazenamento de Arquivos

As imagens enviadas pelos usuários não são gravadas no banco de dados: o banco guarda apenas o caminho público do arquivo. O armazenamento em si é definido por uma interface com duas operações, gravar e remover, e por duas implementações dessa interface, uma que grava no disco da máquina e outra que envia o arquivo ao Cloudflare R2, serviço de armazenamento de objetos compatível com a API do Amazon S3. A escolha entre elas é feita em um único ponto, no módulo de armazenamento, a partir das credenciais presentes na configuração: havendo as variáveis do R2, os arquivos seguem para a nuvem; faltando qualquer uma delas, o sistema grava em disco e registra um aviso na inicialização. Como os demais módulos dependem apenas da interface, a troca de destino não exige alteração em nenhum controlador ou serviço.

O formato das imagens é verificado a partir do conteúdo do arquivo, e não da declaração enviada pelo cliente. O cabeçalho **Content-Type** de um envio é informado pelo próprio navegador ou por quem monta a requisição, e pode ser falsificado trivialmente. Por isso, a aplicação lê a assinatura presente nos primeiros *bytes* do arquivo, que identifica os formatos JPEG e PNG exigidos pelos requisitos, e é essa assinatura que decide se o arquivo é aceito e qual extensão ele recebe. O nome do arquivo gravado é sempre gerado pelo servidor, o que impede que um nome malicioso enviado pelo cliente direcione a gravação para fora do diretório previsto.

6.4 Organização do Código do *Backend*

O código do *backend* segue a organização modular do NestJS, conforme o Quadro 43. Cada funcionalidade do domínio corresponde a um módulo, e todos os módulos adotam a mesma estrutura interna: um diretório **controller**, que recebe as requisições HTTP; um diretório **service**, que concentra as regras de negócio; e um diretório **dto**, que define e valida os formatos de entrada.

Quadro 43 – Estrutura de diretórios do *backend*

```
src/  
  main.ts  
  app.module.ts  
  common/  
    dto/  
    filters/  
    validators/  
  modules/  
    auth/  
    user/  
    category/  
    collection/  
    item/  
    layout/  
    appearance-preset/  
    social/  
    notification/  
    storage/  
    infra/database/prisma/  
prisma/  
  schema/  
  migrations/
```

Fonte: Elaborada pelo autor

Os módulos espelham os pacotes do modelo de domínio descritos na Subseção 4.1.1: o pacote Usuário corresponde ao módulo **user**; o pacote Sistema, ao módulo **auth**, que trata de *login*, *logout*, sessões e redefinição de senha; o pacote Coleções, aos módulos **category**, **collection** e **item**; o pacote Editor de Layout, ao módulo **layout**; o pacote Aparência, ao módulo **appearance-preset**; o pacote Interação Social, ao módulo **social**; e o pacote Notificações, ao módulo **notification**. Os módulos **storage** e **infra** não têm correspondente no domínio, pois oferecem infraestrutura, respectivamente o armazenamento de arquivos e o acesso ao banco, a todos os demais. Os arquivos de teste ficam ao lado do código que verificam, identificados pelo sufixo **.spec.ts**.

Duas decisões de implementação merecem registro por não serem visíveis no esquema do banco. A primeira diz respeito à ficha própria de um item (RF-17). Ao iniciar a personalização, o serviço responsável cria um novo *layout*, de finalidade exclusiva daquele item, e copia para ele os Elementos do *layout* da

coleção, guardando em cada cópia a referência ao original. Quando a coleção não possui *layout*, a cópia é criada vazia, e o usuário monta a ficha a partir do zero. A partir desse momento, qualquer alteração afeta apenas a cópia, e a exclusão do original somente anula a referência, sem apagar as fichas que dele derivaram.

A segunda decisão diz respeito ao conteúdo que toda conta recebe no cadastro: os *layouts* padrão (RF-14) e as paletas predefinidas (RF-18). Esse conteúdo não fica armazenado em registros de um usuário especial do sistema, mas definido no próprio código, nos arquivos `layouts-padrao.ts` e `paletas-padrao.ts`, e copiado para cada nova conta na mesma operação que cria o usuário. A razão é a portabilidade: guardados apenas no banco de desenvolvimento, esses modelos não existiriam em nenhuma outra instalação, e a funcionalidade pareceria defeituosa sem estar. Definidos em código, eles acompanham o repositório, e suas alterações ficam registradas no histórico de versões. As cópias, e não referências a um original compartilhado, são o que permite ao usuário editar e excluir esses modelos como os seus próprios, sem afetar os de outras pessoas. Para as contas criadas antes da existência dos *layouts* padrão, a entrega ocorre na primeira abertura da biblioteca, e é controlada por uma atualização condicional no banco, que registra a entrega apenas se ela ainda não tiver sido feita. Assim, duas requisições simultâneas não produzem cópias em duplicidade: o próprio banco decide qual delas realiza a entrega.

6.5 Aplicação Cliente

O *frontend* é uma aplicação de página única, empacotada pelo Vite, cuja estrutura de diretórios é apresentada no Quadro 44. O diretório `pages` reúne as 13 telas da aplicação, uma por rota: cadastro, *login*, solicitação e redefinição de senha, perfil, categorias, coleções, listagem de itens, ficha do item, biblioteca e editor de *layouts*, e biblioteca e editor de paletas. As partes reutilizáveis dessas telas ficam em `components`; a lógica de estado compartilhada entre telas, em `hooks`; as funções puras, como os cálculos de cor e de geometria do editor, em `lib`; as chamadas à API, em `api`; e os estilos, em `styles`, organizados em uma camada de variáveis visuais, uma de estilos base e uma de componentes.

Quadro 44 – Estrutura de diretórios do *frontend*

```
src/  
  api/  
  assets/  
  components/  
  dados/  
  hooks/  
  lib/  
  pages/  
  styles/  
  types/
```

Fonte: Elaborada pelo autor

6.5.1 Comunicação com a API

Toda a comunicação com o *backend* passa por uma única instância do Axios, configurada com dois interceptadores. O primeiro atua antes de cada requisição e acrescenta o *token* de acesso ao cabeçalho **Authorization**, formato exigido pela estratégia de autenticação da API. O segundo atua sobre as respostas com erro: converte cada falha em uma mensagem exibível, dando preferência à mensagem em português enviada pelo servidor, e, quando a API responde que a sessão expirou ou foi revogada, remove o *token* armazenado e conduz o usuário de volta à tela de *login*. Centralizar esse tratamento evita que cada tela precise repetir a montagem do cabeçalho e a interpretação dos erros.

As leituras de dados são gerenciadas pelo TanStack Query, que guarda em memória as respostas já obtidas, identificadas por uma chave que inclui os parâmetros da consulta. Como a API recebe a ordenação e os filtros como parâmetros, cada combinação corresponde a uma requisição distinta; com a memória por chave, voltar a uma combinação já consultada exibe o resultado imediatamente. As operações de gravação, por sua vez, sinalizam quais consultas ficaram desatualizadas, e as listagens se atualizam sem que a tela precise solicitar os dados novamente. No *logout*, essa memória é esvaziada junto com o *token*, para que nenhum dado da sessão encerrada fique disponível a quem utilizar o mesmo navegador em seguida.

6.5.2 Editor de Layouts

O editor de *layouts* é a parte mais extensa da aplicação cliente, e três decisões de implementação sustentam seu funcionamento. A primeira é a gravação automática. O editor não possui botão de salvar: cada alteração concluída, como soltar um Elemento após arrastá-lo ou terminar de editar um texto, é enviada à API após uma pausa de 400 milissegundos, intervalo que agrupa movimentos feitos em sequência em uma única requisição. A gravação automática só é segura porque vem acompanhada do desfazer e do refazer, acionados pelos atalhos **Ctrl+Z** e **Ctrl+Shift+Z** e apoiados em um histórico de 50 passos; sem eles, um movimento equivocado se tornaria permanente no instante em que fosse feito. Enquanto houver alteração ainda não gravada, o navegador avisa o usuário caso ele tente fechar a página.

A segunda decisão é a separação entre a identidade de um Elemento na tela e sua identidade no banco. Durante a edição, cada Elemento recebe um identificador gerado no próprio navegador, que nunca muda, e o identificador do banco é mantido à parte, em um mapa consultado apenas pela sincronização. Essa separação é o que permite desfazer uma exclusão: o Elemento restaurado precisa ser criado novamente no banco, com um novo identificador, mas permanece o mesmo na tela. A sincronização compara o estado da tela com uma cópia do último estado confirmado pelo servidor e envia somente as diferenças: cria os Elementos novos, altera apenas os campos modificados dos existentes e remove os que deixaram de existir. Quando uma requisição falha, a cópia de referência não é atualizada, de modo que a tentativa seguinte reenvia automaticamente o que ficou pendente.

A terceira decisão diz respeito ao desenho dos Elementos do tipo forma, que são construídos como gráficos vetoriais (SVG) em vez de caixas estilizadas por CSS. O SVG permite aplicar preenchimento, cor, espessura e tracejado da borda a qualquer uma das doze silhuetas disponíveis, o que o recorte de caixas por CSS não oferece. A área de desenho de cada forma adota as mesmas dimensões do Elemento, de modo que a espessura do traço é expressa na mesma unidade relativa das posições e dos tamanhos, descrita no Capítulo 5, e acompanha a escala da ficha quando a largura disponível muda.

6.5.3 Aparência e Tipografia

A aparência da interface é aplicada por meio de variáveis CSS. Uma função pura converte a paleta em uso no conjunto de variáveis que colore a interface, e a mesma função pinta a área de prévia do editor de paletas e as miniaturas da biblioteca, o que garante que a prévia nunca diverge do resultado aplicado. Quando o estilo global em vigor é o oposto ao da paleta em uso, a claridade de todas as cores é invertida no espaço de cor OKLCH, que, ao contrário do HSL, representa a claridade de acordo com a percepção humana, preservando o matiz e a saturação de cada cor.

A preferência de estilo é persistida no banco, por fazer parte da paleta, e replicada no armazenamento local do navegador. A cópia local atende aos momentos em que a API não pode responder: as telas anteriores ao *login*, que ainda não sabem de qual conta se trata, e o primeiro quadro de cada carregamento da página. Ela é lida por um pequeno *script* no cabeçalho do documento, executado antes do carregamento dos estilos, o que evita que a página seja exibida por um instante com o estilo errado. Havendo divergência entre as duas fontes, prevalece o valor do banco.

As fontes tipográficas oferecidas além das do sistema operacional são servidas pelo próprio site, e não carregadas de serviços externos como o Google Fonts. Carregar uma fonte de um serviço de terceiro faz o navegador de cada visitante enviar a esse serviço seu endereço IP, sem base legal que autorize esse tratamento nos termos da Lei Geral de Proteção de Dados Pessoais. Servindo os arquivos do mesmo domínio, não há terceiro envolvido. O navegador continua baixando cada família apenas quando algum texto exibido a utiliza.

6.6 Verificação Durante a Implementação

A implementação foi acompanhada por testes automatizados no *backend*, escritos com o Jest. No momento da redação deste trabalho, o projeto conta com 51 conjuntos de testes, que somam 359 casos, todos aprovados. Os testes verificam serviços, controladores e DTOs de todos os módulos, substituindo o acesso ao banco e ao armazenamento por objetos simulados, de modo que cada regra de negócio

seja exercitada de forma isolada e sem depender de um banco em execução. No *frontend*, a etapa de construção executa a verificação de tipos do TypeScript antes de gerar o pacote de produção, o que impede que uma divergência entre os tipos da interface e os formatos da API passe despercebida. Complementarmente, cada funcionalidade concluída foi verificada manualmente na interface, com apoio do DBeaver para inspecionar o estado do banco. Os procedimentos e os resultados dos testes são detalhados no capítulo dedicado aos testes.

6.7 Limitações da Implementação

A versão atual do sistema possui limitações conhecidas, registradas a seguir:

- **Limitação de requisições em memória:** os contadores de requisições são mantidos na memória do processo da API. Reiniciar o servidor os zera, e várias instâncias da aplicação não compartilhariam esses contadores. Em uma implantação com múltiplas instâncias, seria necessário um armazenamento compartilhado.
- **Ausência de comunicação em tempo real:** a API é exclusivamente REST, sem canal de comunicação contínua com o navegador. O *feed* de atualizações e as notificações são obtidos quando a interface os solicita, e não enviados pelo servidor no momento em que os eventos ocorrem.
- **Ausência de documentação automática da API:** não há documentação no padrão OpenAPI; a referência das rotas é mantida manualmente no repositório do *backend*.
- **Armazenamento em disco sem credenciais de nuvem:** na ausência das credenciais do R2, os arquivos enviados ficam no disco da máquina que executa a API e não estão disponíveis em outra máquina, perdendo-se em ambientes de hospedagem cujo sistema de arquivos é recriado a cada publicação.

7 TESTE

Este capítulo apresenta os testes realizados para verificar as regras de negócio do sistema, conforme a estratégia descrita no Capítulo 2. A verificação automatizada concentra-se no *backend*, onde residem as regras de acesso, privacidade, sessão e consistência dos dados, e foi conduzida por meio de testes de unidade. São descritos o ambiente em que os testes são executados, uma visão geral do conjunto de testes por módulo, a especificação dos principais casos de teste e os resultados obtidos.

7.1 Ambiente e Execução dos Testes

Os testes foram escritos com o *framework* Jest, na versão 30, com o *ts-jest* responsável por interpretar o código TypeScript. Cada arquivo de teste fica ao lado do código que verifica e é identificado pelo sufixo `.spec.ts`, e o conjunto completo é executado pelo comando `pnpm test`, na raiz do projeto do *backend*.

Os testes são de unidade: cada um exercita uma parte do sistema de forma isolada. Para isso, as dependências externas, como o serviço de acesso ao banco de dados e o serviço de armazenamento de arquivos, são substituídas por objetos simulados, cujas respostas são definidas pelo próprio teste. Essa abordagem permite reproduzir situações difíceis de obter manualmente, como duas requisições concorrentes sobre o mesmo registro, e dispensa um banco de dados em execução. Em contrapartida, o teste verifica as chamadas que a regra de negócio faz ao banco, e não o resultado dessas chamadas em um banco real.

Os testes se dividem em três alvos. Os testes de serviços verificam as regras de negócio propriamente ditas. Os testes de controladores verificam que cada operação é executada em nome do usuário da sessão autenticada, e não de um identificador enviado na requisição. Os testes de DTOs verificam a validação dos dados de entrada, como formatos, limites e campos obrigatórios.

7.2 Visão Geral dos Testes

O Quadro 45 apresenta a distribuição dos testes pelos módulos do *backend*, com os requisitos funcionais a que cada módulo se relaciona. Os módulos **storage** e **common** não correspondem a um requisito específico, pois reúnem funcionalidades transversais: o armazenamento de arquivos, a validação dos Elementos e das imagens enviadas e a regra de mescla de fichas.

Quadro 45 – Distribuição dos testes automatizados por módulo

Módulo	Requisitos relacionados	Arquivos	Testes
social	RF-20 a RF-23	20	103
user	RF-01, RF-03 e RF-05	3	41
common	Transversal (validação de Elementos e de imagens e mescla de fichas)	4	41
item	RF-09, RF-10, RF-12, RF-17 e RF-22	4	38
layout	RF-14 e RF-16	3	27
category	RF-06 e RF-13	4	24
appearance-preset	RF-18 e RF-19	3	23
notification	RF-24	3	23
storage	Transversal (armazenamento de arquivos)	2	16
auth	RF-02 e RF-04	4	12
collection	RF-07, RF-08 e RF-15	1	11
Total		51	359

Fonte: Elaborada pelo autor

O módulo social concentra o maior número de testes porque reúne as regras com mais combinações possíveis: o que cada usuário pode ver e acompanhar depende, ao mesmo tempo, do nível de privacidade da coleção e da existência de uma amizade aceita com o dono. Parte desses testes é parametrizada, isto é, um mesmo teste é executado para cada combinação de nível de acesso e privacidade, o que garante que nenhuma delas fique sem verificação.

7.3 Casos de Teste

Os casos de teste a seguir foram especificados segundo a estrutura proposta por Marques (2018), composta por título, objetivo, pré-condição, passos e resultados esperados. Como os casos são executados automaticamente pelo Jest, os passos descrevem as ações realizadas pelo teste automatizado: preparar o estado simulado, acionar a operação e verificar o retorno. Os resultados esperados descrevem o comportamento do sistema com verbos no presente, conforme recomenda o autor. Ao final de cada caso, são indicados os testes automatizados que o implementam, identificados pelo nome registrado no código. Foram selecionadas as regras de maior impacto sobre a segurança e a consistência dos dados, apresentadas nos Quadros 46 a 53.

Quadro 46 – Caso de teste CT01: validar a revogação da sessão no *logout*

Objetivo: Verificar se o <i>logout</i> revoga a sessão atual e se um <i>token</i> associado a uma sessão revogada deixa de ser aceito (RF-02).
Pré-condição: Usuário cadastrado, com uma sessão ativa registrada no banco e um <i>token</i> de acesso que referencia essa sessão.
Passos: 1. Acionar o <i>logout</i> informando o usuário e a sessão atual. 2. Verificar a atualização enviada à tabela de sessões. 3. Validar um <i>token</i> cuja sessão está ativa. 4. Validar um <i>token</i> cuja sessão não existe ou foi revogada.
Resultados esperados: 1. O sistema registra a data de revogação somente na sessão atual do usuário, e apenas se ela ainda não estiver revogada. 2. O sistema aceita o <i>token</i> da sessão ativa e devolve os dados do usuário autenticado. 3. O sistema recusa o <i>token</i> da sessão inexistente ou revogada, respondendo como acesso não autorizado.
Testes automatizados: <code>auth.service.spec.ts</code> : “revokes only the current session on logout”; <code>jwt.strategy.spec.ts</code> : “accepts an active session” e “rejects a missing or revoked session”.

Fonte: Elaborada pelo autor

Quadro 47 – Caso de teste CT02: validar a redefinição de senha por *token* temporário

Objetivo: Verificar se a recuperação de senha gera um <i>token</i> temporário armazenado de forma protegida, não revela se um e-mail está cadastrado e invalida o <i>token</i> e as sessões após o uso (RF-04).
Pré-condição: Usuário cadastrado com e-mail conhecido.
Passos: 1. Solicitar a recuperação de senha para o e-mail cadastrado. 2. Solicitar a recuperação de senha para um e-mail não cadastrado. 3. Redefinir a senha com um <i>token</i> válido e uma nova senha confirmada. 4. Redefinir a senha com um <i>token</i> inválido ou expirado.
Resultados esperados: 1. O sistema envia por e-mail um <i>token</i> de 64 caracteres, grava no banco apenas o seu <i>hash</i> SHA-256, com data de expiração, e responde com uma mensagem neutra, iniciada por “Se o e-mail estiver cadastrado”. 2. O sistema responde com a mesma mensagem neutra, sem criar <i>token</i> e sem enviar e-mail. 3. O sistema grava a nova senha criptografada com bcrypt, marca o <i>token</i> como utilizado somente se ele ainda não tiver sido usado nem expirado, e revoga todas as sessões ativas do usuário. 4. O sistema recusa a requisição como inválida e não altera a senha.
Testes automatizados: <code>auth.service.spec.ts</code> : “creates a hashed, temporary password reset token”, “uses the same response without creating a token for an unknown email”, “resets the password, consumes the token and revokes all sessions” e “rejects an invalid or expired reset token”.

Fonte: Elaborada pelo autor

Quadro 48 – Caso de teste CT03: validar o encerramento de acompanhamentos ao restringir a privacidade

Objetivo: Verificar se, ao tornar uma coleção mais restrita, o sistema encerra os acompanhamentos dos usuários que perderam acesso a ela, preservando os demais (RF-08 e RF-21).
Pré-condição: Coleção pertencente ao usuário autenticado, com acompanhamentos ativos de outros usuários.
Passos: 1. Alterar a privacidade da coleção para privada. 2. Alterar a privacidade da coleção para somente amigos. 3. Alterar a privacidade de uma coleção somente para amigos para pública. 4. Verificar, em cada caso, as atualizações enviadas às tabelas de coleções e de acompanhamentos.
Resultados esperados: 1. Ao tornar a coleção privada, o sistema altera a privacidade e encerra todos os acompanhamentos ativos na mesma transação, registrando a data de encerramento. 2. Ao tornar a coleção somente para amigos, o sistema encerra apenas os acompanhamentos de usuários sem amizade aceita com o dono, mantendo os dos amigos. 3. Ao tornar a coleção menos restrita, o sistema altera a privacidade sem modificar nenhum acompanhamento.
Testes automatizados: <code>collection.service.spec.ts</code> : “ends every active follow when a collection becomes private”, “keeps friend follows and ends non-friend follows when becoming friends-only” e “does not touch follows when visibility becomes less restrictive”.

Fonte: Elaborada pelo autor

Quadro 49 – Caso de teste CT04: validar o acesso a coleções conforme privacidade e amizade

Objetivo: Verificar se cada usuário visualiza e acompanha somente as coleções permitidas pelo seu vínculo com o dono e pelo nível de privacidade de cada coleção (RF-08, RF-21 e RF-23).
Pré-condição: Dois usuários cadastrados: o dono das coleções e um visitante autenticado, com ou sem amizade aceita entre eles.
Passos: 1. Determinar o nível de acesso quando o visitante é o próprio dono. 2. Determinar o nível de acesso quando há amizade aceita entre visitante e dono. 3. Determinar o nível de acesso quando não há amizade, ou quando a solicitação está pendente, recusada ou cancelada. 4. Avaliar, para cada nível de acesso, a visualização de coleções públicas, somente para amigos e privadas. 5. Solicitar o acompanhamento de uma coleção pública, de uma coleção somente para amigos e de uma coleção do próprio usuário. 6. Consultar uma coleção inexistente e uma coleção que o visitante não pode ver.
Resultados esperados: 1. O sistema concede ao dono acesso às três privacidades, sem consultar amizades. 2. O sistema concede ao amigo acesso às coleções públicas e somente para amigos. 3. O sistema concede aos demais usuários acesso apenas às coleções públicas, inclusive quando a solicitação de amizade não foi aceita. 4. O sistema permite que não amigos acompanhem coleções públicas e que amigos acompanhem coleções somente para amigos, e recusa o acompanhamento de uma coleção do próprio usuário. 5. O sistema responde que a coleção não foi encontrada tanto quando ela não existe quanto quando o visitante não tem permissão para vê-la, sem revelar a existência de coleções invisíveis.
Testes automatizados: <code>social-access-policy.service.spec.ts</code>: todos os testes do arquivo, incluindo as combinações parametrizadas de nível de acesso e privacidade; <code>social-content.service.spec.ts</code>: “returns not found for a missing or invisible collection”.

Fonte: Elaborada pelo autor

Quadro 50 – Caso de teste CT05: validar a criação da ficha própria de um item

Objetivo: Verificar se a personalização de um item cria uma cópia própria do <i>layout</i> , com ou sem <i>layout</i> na coleção, e se as alterações afetam somente essa cópia (RF-17).
Pré-condição: Item pertencente ao usuário autenticado, em uma coleção com <i>layout</i> associado ou sem <i>layout</i> .
Passos: 1. Iniciar a personalização de um item cuja coleção possui <i>layout</i> com Elementos. 2. Iniciar a personalização de um item cuja coleção não possui <i>layout</i>. 3. Adicionar um Elemento a um item que já possui ficha própria. 4. Tentar alterar um Elemento de um item que ainda não possui ficha própria.
Resultados esperados: 1. O sistema cria um <i>layout</i> de uso exclusivo do item, com referência ao <i>layout</i> da coleção, copia cada Elemento com seu conteúdo, posição, tamanho, camada e estilo, registrando a referência ao Elemento original, e associa o novo <i>layout</i> ao item. 2. O sistema cria o <i>layout</i> do item vazio, sem referência de origem e sem copiar Elementos, e o associa ao item. 3. O sistema grava o novo Elemento somente no <i>layout</i> próprio do item. 4. O sistema recusa a alteração por conflito de estado e não modifica nenhum Elemento.
Testes automatizados: <code>item-layout-override.service.spec.ts</code> : “creates an editable item layout copy from the collection layout”, “creates an empty override when the collection has no layout”, “adds a visual element only to the item override layout” e “rejects editing item visual elements before an override exists”.

Fonte: Elaborada pelo autor

Quadro 51 – Caso de teste CT06: validar a paleta em uso

Objetivo: Verificar se o cadastro deixa em uso a paleta correspondente ao tema do sistema operacional, se a troca de paleta mantém uma única paleta em uso e se a paleta em uso não pode ser excluída (RF-01 e RF-18).
Pré-condição: Para o cadastro, dados válidos de um novo usuário; para as demais operações, usuário autenticado com paletas na biblioteca.
Passos: 1. Cadastrar um usuário informando que o sistema operacional está no tema escuro. 2. Selecionar como paleta em uso uma paleta do usuário que não está ativa. 3. Tentar excluir a paleta em uso. 4. Excluir uma paleta predefinida que não está em uso.
Resultados esperados: 1. O sistema cria as paletas predefinidas com exatamente uma em uso, a paleta “Escuro”, de tema escuro. 2. O sistema desativa a paleta anterior e ativa a selecionada em uma única transação. 3. O sistema recusa a exclusão por conflito e não remove a paleta. 4. O sistema remove a paleta predefinida, que pertence ao usuário como as demais.
Testes automatizados: <code>user.service.spec.ts</code> : “starts the account on the dark palette when the system is dark”; <code>appearance-preset.service.spec.ts</code> : “activates an owned preset and deactivates the previous one atomically”, “rejects deleting the active preset” e “deletes a system palette that is not in use”.

Fonte: Elaborada pelo autor

Quadro 52 – Caso de teste CT07: validar a resposta a uma solicitação de amizade

Objetivo: Verificar se a resposta a uma solicitação de amizade altera o estado apenas enquanto a solicitação está pendente, notifica o remetente na mesma transação e não produz efeitos duplicados em respostas concorrentes (RF-20 e RF-24).
Pré-condição: Solicitação de amizade enviada por um usuário ao usuário autenticado.
Passos: 1. Aceitar uma solicitação pendente. 2. Recusar uma solicitação que já foi cancelada pelo remetente. 3. Aceitar uma solicitação pendente que outra requisição simultânea já respondeu.
Resultados esperados: 1. O sistema altera a solicitação para aceita somente se ela ainda estiver pendente e destinada ao usuário autenticado, registra a data da resposta e cria, na mesma transação, a notificação “João aceitou sua solicitação de amizade.” para o remetente. 2. O sistema recusa a operação por conflito, sem alterar a solicitação e sem criar notificação. 3. O sistema identifica que nenhum registro foi alterado, recusa a operação por conflito e não cria notificação.
Testes automatizados: <code>friendship.service.spec.ts</code> : “accepts a pending request and notifies its sender transactionally”, “rejects a request that is no longer pending” e “rejects a concurrent second response without creating a notification”.

Fonte: Elaborada pelo autor

Quadro 53 – Caso de teste CT08: validar o formato real da imagem enviada

Objetivo: Verificar se o sistema identifica o formato de uma imagem pelo seu conteúdo, e não pelo nome ou pelo tipo declarado, e se recusa arquivos que apenas aparentam ser imagens (RF-09).
Pré-condição: Item pertencente ao usuário autenticado, sem imagem de capa.
Passos: 1. Verificar o formato de arquivos que começam com as assinaturas de JPEG e de PNG. 2. Verificar o formato de um arquivo de texto renomeado como imagem. 3. Enviar como imagem de capa do item um arquivo cujo conteúdo é texto.
Resultados esperados: 1. O sistema reconhece os formatos JPEG e PNG pelas assinaturas no início do arquivo. 2. O sistema não reconhece o arquivo de texto como imagem. 3. O sistema recusa o envio como requisição inválida e não grava nenhum arquivo no armazenamento.
Testes automatizados: <code>imagem-real.validator.spec.ts</code> : “recognizes JPEG and PNG by their signature” e “rejects content that only claims to be an image”; <code>item.service.spec.ts</code> : “rejects a file whose bytes are not JPEG or PNG”.

Fonte: Elaborada pelo autor

7.4 Resultados e Limitações

Na execução realizada em 2 de outubro de 2026, os 51 conjuntos de testes, com 359 testes no total, foram aprovados, em aproximadamente 8 segundos. Todos os casos de teste especificados na seção anterior obtiveram, portanto, os resultados esperados.

A estratégia adotada possui limitações que devem ser consideradas na interpretação desses resultados:

- **Ausência de testes de integração:** como o acesso ao banco é simulado, os testes verificam as operações que a regra de negócio solicita ao banco, mas não o comportamento dessas operações em um banco PostgreSQL real, como a aplicação das restrições de unicidade.

- **Ausência de testes de ponta a ponta:** não há testes que percorram a API a partir de requisições HTTP, verificando em conjunto a autenticação, a validação e as respostas de cada rota.
- **Escopo da verificação automatizada:** os testes automatizados concentram-se nas regras de negócio do *backend* e não abrangem os aspectos visuais e de interação da aplicação cliente.

8 IMPLANTAÇÃO

Os Capítulos 5 e 6 trataram o sistema como artefato lógico: a divisão em camadas, a aplicação cliente em React, a API em NestJS e a persistência em PostgreSQL por meio do Prisma. Este capítulo trata de onde cada uma dessas partes é executada. A distinção é necessária porque o desenho em camadas não determina, por si só, a distribuição física: a mesma arquitetura poderia ser publicada em uma única máquina ou repartida entre vários provedores, e a escolha entre essas alternativas responde a critérios de custo, de carga esperada e de esforço de operação, e não a critérios de projeto.

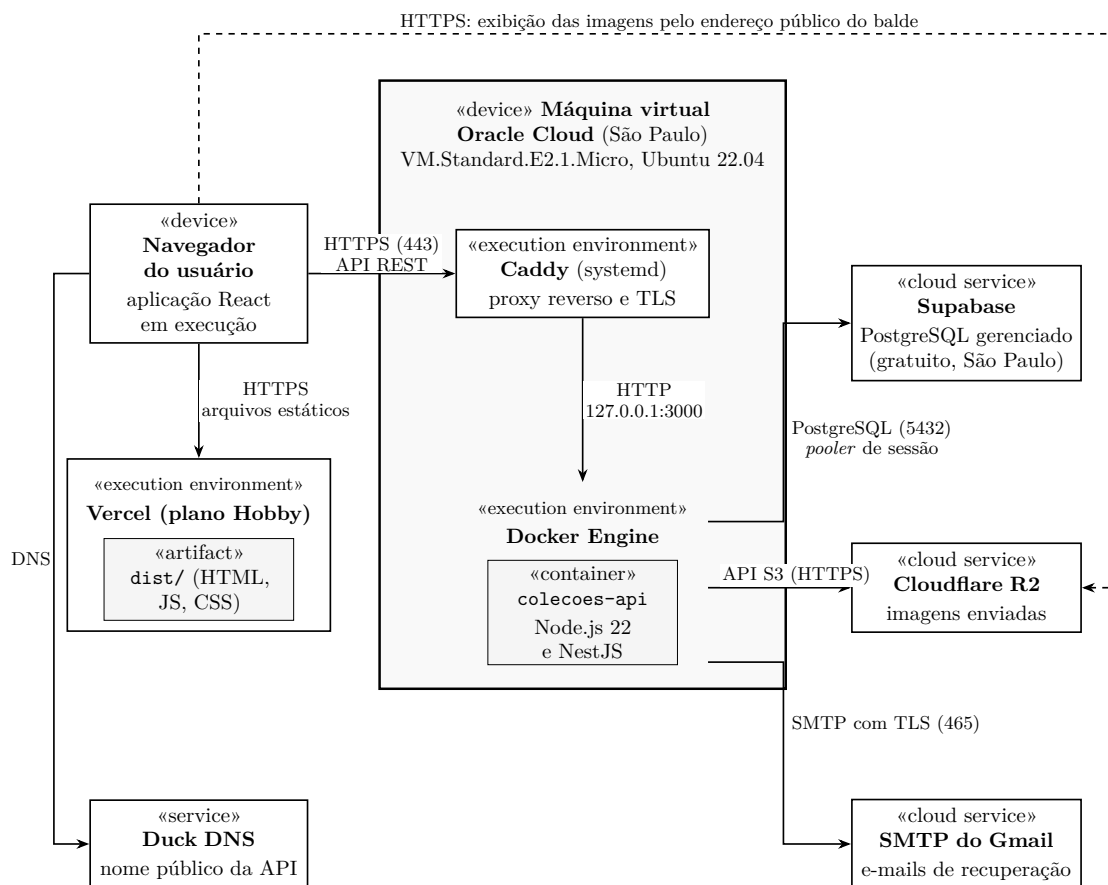
O ambiente aqui descrito é o de produção, acessível pela internet, e foi montado exclusivamente com planos gratuitos dos provedores envolvidos. Essa restrição de custo é o fio que explica a maior parte das decisões registradas a seguir. A Seção 8.1 apresenta os ambientes de execução e a distribuição dos componentes entre eles. A Seção 8.2 descreve como cada componente é configurado e qual superfície da máquina permanece exposta. A Seção 8.3 percorre o caminho que uma nova versão do sistema atravessa até entrar em operação.

8.1 Infraestrutura

A alocação dos componentes segue um critério simples: cada parte do sistema é publicada no tipo de ambiente que corresponde à sua natureza de execução. A aplicação cliente, que se reduz a arquivos estáticos, vai para uma plataforma de hospedagem de arquivos estáticos; a API, que é um processo de longa duração, vai para uma máquina virtual; e os dados, que precisam sobreviver à máquina, ficam em serviços gerenciados. Desse critério decorre a distribuição apresentada na Figura 19.

Aplicação cliente. A aplicação cliente é publicada na Vercel, no plano gratuito (Hobby), sob o endereço <<https://tcc-frontend-git-main-camilaalbierimattos.vercel.app>>. A separação já era imposta pela arquitetura: a aplicação executa no

Figura 19 – Diagrama de implantação do sistema



Fonte: Elaborada pelo autor

navegador do usuário, e o resultado da sua construção (`tsc -b` seguido de `vite build`) é um conjunto de arquivos estáticos, sem estado de servidor. Qualquer hospedagem de arquivos estáticos poderia servi-los; a Vercel foi escolhida porque constrói e publica o projeto diretamente a partir do repositório no GitHub, a cada alteração no ramo principal (Vercel, a), o que dispensa qualquer etapa manual para essa parte do sistema. Servir esses arquivos a partir da mesma máquina que executa a API foi descartado, por consumir a memória escassa dessa máquina em uma função que não exige nenhuma de suas características. Como a navegação entre telas é feita no próprio navegador, um endereço interno aberto diretamente, como o de uma Coleção, precisa ser respondido com o arquivo `index.html`; em aplicações Vite publicadas na Vercel, isso é feito por uma regra de reescrita declarada no

arquivo `vercel.json` do repositório (Vercel, c).

API. A API é executada em uma máquina virtual da Oracle Cloud Infrastructure, criada na região de São Paulo com a forma `VM.Standard.E2.1.Micro`, que integra o nível gratuito permanente (*Always Free*) do provedor. A documentação da Oracle descreve essa forma com um oitavo de OCPU, com possibilidade de usar recursos adicionais de processamento, e 1 GB de memória (Oracle,); o sistema operacional é o Ubuntu 22.04. Diferentemente de arquiteturas baseadas em serviços, em que a decisão de implantação envolve distribuir vários processos entre máquinas, a API deste trabalho é um monólito: um único processo, empacotado em um único contêiner. A questão de distribuição, portanto, não se coloca, e o dimensionamento reduz-se a saber se a máquina comporta esse processo. Com 1 GB, a memória é o recurso que se esgota primeiro, antes do processamento e do disco. O efeito foi observado já na primeira publicação: como a imagem do contêiner é construída na própria máquina, a instalação das dependências e a compilação só foram concluídas depois da criação de uma área de troca (*swap*) em disco.

Banco de dados. O PostgreSQL de produção é fornecido em regime gerenciado pelo Supabase, no plano gratuito, em projeto criado na região de São Paulo, a mesma da máquina virtual, o que mantém curto o percurso de ida e volta entre a API e o banco. Durante o desenvolvimento, o banco local em contêiner Docker, descrito nos Capítulos 2 e 6, continua disponível como alternativa. A escolha de um serviço gerenciado separa duas responsabilidades: manter o processo do banco em execução e atualizado, que passa a ser do provedor, e guardar a única cópia dos dados fora da máquina virtual, que de outro modo dividiria com a API o mesmo gigabyte de memória. A API conecta-se pelo *pooler* de sessão do Supabase, na porta 5432. A escolha não é de desempenho, mas de rede: no plano gratuito, a conexão direta ao banco é feita apenas por IPv6, e o próprio provedor indica o modo sessão do *pooler* como alternativa para clientes em redes apenas IPv4 (Supabase, b), situação da máquina virtual, que dispõe de um endereço público IPv4. A mesma cadeia de conexão atende às migrações e à API.

O plano gratuito impõe três limites que convém registrar. O banco é limitado a 500 MB por projeto (Supabase, a). O provedor não faz cópia de segurança automática de projetos gratuitos e recomenda que eles exportem os dados regularmente

com o comando `db dump` da sua ferramenta de linha de comando, mantendo as cópias fora da plataforma (Supabase, c). E projetos gratuitos com pouca atividade ao longo de sete dias são pausados, podendo ser restaurados pelo painel (Supabase, d). O primeiro limite está longe de ser atingido por uma operação de validação; os dois últimos, ao contrário, afetam diretamente a confiabilidade do ambiente: o que a delegação ao serviço gerenciado removeu foi a operação do banco, e não a garantia de recuperação dos dados, que depende de uma exportação feita pelo próprio responsável pela plataforma.

Arquivos enviados. As imagens enviadas pelos usuários (capas, ícones, fotos de perfil e imagens dos Elementos) são gravadas no Cloudflare R2, acessado pela API S3 por meio do SDK da AWS, conforme o Capítulo 6. A consequência prática é que a máquina virtual não guarda estado: o banco está no Supabase e os arquivos estão no R2, de modo que perder a máquina exige apenas reinstalar a API, sem perda de Coleções, Itens ou imagens. Essa propriedade depende das credenciais do R2 estarem configuradas; sem elas, a API gravaria os arquivos no disco do contêiner, limitação já registrada no Capítulo 6, e eles se perderiam a cada nova construção da imagem.

Envio de e-mail. Os e-mails de recuperação de senha são enviados pelo servidor SMTP do Gmail, com conexão cifrada na porta 465 e autenticação por senha de aplicativo da conta Google. Não há serviço de e-mail próprio: o envio depende apenas das variáveis de configuração descritas na Seção 8.2.

Nome de domínio. Não houve registro de domínio próprio. A aplicação cliente usa o subdomínio fornecido pela Vercel, e a API usa um subdomínio gratuito do Duck DNS, serviço que aponta subdomínios de `duckdns.org` para um endereço IP escolhido pelo usuário (Duck DNS,), no caso, o endereço público da máquina virtual. A necessidade de um nome para a API decorre de uma cadeia de exigências. A aplicação cliente é servida pela Vercel em HTTPS, e requisições `fetch()` ou `XMLHttpRequest` feitas por uma página segura a um endereço HTTP são conteúdo misto bloqueável, recusado pelo navegador (MDN Web Docs,); a API precisa, portanto, responder em HTTPS. Para isso, o Caddy obtém e renova automaticamente certificados TLS para nomes de domínio públicos (Caddy Web Server, a), o que pressupõe um nome que resolva para a máquina. O subdomínio

gratuito atende a essa exigência sem custo. A contrapartida é a dependência de um serviço mantido por terceiros, sem compromisso de disponibilidade: uma falha na resolução do nome tornaria a API inacessível, ainda que a máquina estivesse em funcionamento.

O Quadro 54 sintetiza os alvos de implantação e o motivo da alocação de cada um.

Quadro 54 – Alvos de implantação do sistema

Componente	Onde executa	Plano	Motivo da alocação
Aplicação cliente (React)	Vercel	Hobby (gratuito)	Arquivos estáticos sem estado de servidor; construção e publicação automáticas a partir do repositório
API (NestJS)	Máquina virtual Oracle Cloud, em contêiner Docker, atrás do Caddy	<i>Always Free</i>	Processo de longa duração; um único contêiner, por ser um monólito
Banco de dados (PostgreSQL)	Supabase, região de São Paulo	Gratuito	Operação do banco delegada ao provedor; dados fora da máquina virtual
Arquivos enviados	Cloudflare R2	Gratuito	Mantém a máquina virtual sem estado
E-mail	SMTP do Gmail	Conta Google	Envio dos e-mails de recuperação de senha sem servidor próprio
Nome da API	Duck DNS	Gratuito	Nome público necessário para o certificado TLS emitido pelo Caddy

Fonte: Elaborada pelo autor

8.2 Configuração dos Serviços

A configuração segue um princípio único: o código é o mesmo em todos os ambientes, e tudo o que varia entre eles é fornecido de fora, no momento da execução ou da construção. Na máquina virtual, os valores ficam em um arquivo `.env` ao lado do repositório; na Vercel, nas variáveis de ambiente do projeto. O Quadro 55 relaciona essas variáveis, sem seus valores.

Quadro 55 – Variáveis de ambiente de produção

Variável	Onde	Função
DATABASE_URL	Máquina virtual	Cadeia de conexão do <i>pooler</i> de sessão do Supabase, usada pela API e pelas migrações
JWT_SECRET	Máquina virtual	Segredo de assinatura dos <i>tokens</i> de sessão
FRONTEND_URL	Máquina virtual	Endereço publicado da aplicação cliente; única origem autorizada pelo CORS
PORT	Máquina virtual	Porta em que a API escuta dentro do contêiner
SMTP_*	Máquina virtual	Servidor, porta, cifra, usuário, senha de aplicativo e remetente do Gmail
R2_*	Máquina virtual	Conta, chaves de acesso, balde e endereço público do Cloudflare R2
API_URL	Vercel	Endereço HTTPS da API, embutido nos arquivos estáticos durante a construção

Fonte: Elaborada pelo autor

A variável `API_URL` tem natureza diferente das demais. Como a aplicação cliente não tem servidor próprio, não há momento de execução em que ela pudesse ler a configuração: o endereço da API é embutido nos arquivos durante a construção, e alterá-lo exige uma nova publicação. O nome sem o prefixo `VITE_`, que o Vite exige por padrão, é aceito por uma configuração explícita do prefixo no `vite.config.ts`, restrita a esse nome exato para que nenhuma outra variável do ambiente seja exposta nos arquivos públicos por acidente.

8.2.1 Contêiner da API

A API é construída e executada pelo Docker, a partir de um `Dockerfile` e de um `docker-compose.yml` versionados no repositório do *backend*. O `Dockerfile`, reproduzido no Quadro 56, é dividido em três estágios. O estágio `base` parte da imagem `node:22-slim` e acrescenta o OpenSSL, exigido pelo motor do Prisma e ausente da imagem reduzida. O estágio `build` instala as dependências, o que gera o cliente do Prisma, compila o código e aplica as migrações pendentes ao banco. O estágio `runtime`, que é o único que compõe a imagem final, copia apenas o `package.json`, as dependências e o código compilado, sem o código-fonte.

Quadro 56 – Dockerfile da API

```
1 FROM node:22-slim AS base
2 RUN apt-get update && apt-get install -y --no-install-recommends
   openssl \
3   && rm -rf /var/lib/apt/lists/*
4 RUN corepack enable
5 WORKDIR /app
6
7 FROM base AS build
8 COPY package.json pnpm-lock.yaml pnpm-workspace.yaml prisma.config
   .ts ./
9 COPY prisma ./prisma
10 RUN pnpm install --frozen-lockfile
11 COPY . .
12 RUN pnpm build
13 RUN --mount=type=secret,id=backend_env,target=/app/.env \
14   pnpm exec prisma migrate deploy
15
16 FROM base AS runtime
17 ENV NODE_ENV=production
18 COPY --from=build /app/package.json ./
19 COPY --from=build /app/node_modules ./node_modules
20 COPY --from=build /app/dist ./dist
21 EXPOSE 3000
22 CMD ["node", "dist/src/main.js"]
```

Fonte: Elaborada pelo autor

Dois pontos desse arquivo merecem comentário. O primeiro é a forma como a migração recebe a cadeia de conexão. Argumentos de construção e variáveis de ambiente são inadequados para passar segredos ao processo de construção, porque persistem na imagem final; a montagem de segredo (`RUN --mount=type=secret`) torna o arquivo disponível apenas durante a instrução que o utiliza (Docker, a). O `.env` é declarado no `docker-compose.yml` como segredo de construção e montado somente no passo da migração, de modo que nenhuma camada da imagem contém as credenciais. Em execução, o mesmo arquivo é entregue ao contêiner como conjunto de variáveis de ambiente (`env_file`).

O segundo é o momento da migração. Executada durante a construção, ela altera o esquema do banco antes que o contêiner novo substitua o antigo, e o contêiner antigo continua atendendo requisições sobre o esquema já alterado até ser

substituído. A vinculação tem uma consequência desejável e uma que precisa ser observada. A desejável é que uma versão cujo esquema não pôde ser aplicado não chega a entrar em execução, porque a falha da migração interrompe a construção. A que precisa ser observada é que, durante a janela de construção, o código antigo opera sobre o esquema novo: em migrações que apenas acrescentam tabelas ou colunas, o efeito é nulo; em migrações que removem ou renomeiam estruturas, abre-se uma janela de erro. Manter cada migração compatível com a versão anterior do código passa, assim, a ser condição do procedimento tal como ele está construído.

8.2.2 Superfície Exposta

O Quadro 57 reproduz a declaração do contêiner e a configuração do Caddy. A porta da API é publicada apenas na interface de retorno da máquina (127.0.0.1), o que a torna inalcançável de fora; o único caminho até ela passa pelo Caddy, instalado diretamente no sistema operacional e executado como serviço do **systemd**. A restrição no próprio mapeamento de portas não é redundante em relação ao *firewall*: quando o Docker publica a porta de um contêiner, o tráfego é desviado antes de passar pelas regras do UFW, que ficam, na prática, sem efeito para essa porta (Docker, b). Publicada em todas as interfaces, a API ficaria acessível sem cifra, contornando o Caddy, ainda que o UFW não a liberasse.

Quadro 57 – Declaração do contêiner da API e configuração do Caddy

```
# docker-compose.yml (trecho)
services:
  api:
    build:
      context: .
      dockerfile: Dockerfile
      secrets:
        - backend_env
    container_name: colecoes-api
    restart: unless-stopped
    env_file:
      - .env
    ports:
      - "127.0.0.1:3000:3000"

# /etc/caddy/Caddyfile
[PREENCHER].duckdns.org {
  reverse_proxy localhost:3000
}
```

Fonte: Elaborada pelo autor

O bloco do Caddyfile é suficiente para o arranjo inteiro: por se tratar de um nome de domínio público, o Caddy obtém e renova o certificado TLS e redireciona o tráfego HTTP para HTTPS sem configuração adicional (Caddy Web Server, a). A superfície de rede da máquina reduz-se, assim, a três portas, liberadas tanto nas regras de entrada da rede virtual da Oracle quanto no UFW: a 22, para administração por SSH com autenticação por chave; a 443, atendida pelo Caddy; e a 80, que não serve conteúdo algum e responde apenas pelo redirecionamento para HTTPS e pela validação dos certificados.

Ao repassar uma requisição, o Caddy acrescenta o cabeçalho `X-Forwarded-For` com o endereço do cliente (Caddy Web Server, b). A informação é necessária para a limitação de requisições descrita no Capítulo 6, que identifica o cliente pelo endereço IP: sem ela, todas as requisições chegariam à API vindas do próprio Caddy, e o limite de cinco tentativas de *login* por minuto passaria a valer para a plataforma inteira, e não para cada cliente. A API é, por isso, configurada para confiar em exatamente um intermediário, o Caddy. Confiar em qualquer valor do cabeçalho, em vez de em um único salto, permitiria a um

cliente forjar o próprio endereço e escapar do limite. Pela mesma razão de restringir o que é aceito, o CORS da API autoriza uma única origem, a da aplicação cliente publicada, lida da variável `FRONTEND_URL`.

8.2.3 Continuidade de Execução

A recuperação após falhas é delegada a dois supervisores já presentes na máquina. O contêiner é declarado com a política `restart: unless-stopped`, de modo que o Docker o reinicia se o processo terminar inesperadamente, e o Docker e o Caddy são serviços do `systemd` habilitados na inicialização, o que faz a plataforma voltar sozinha depois de uma reinicialização da máquina virtual. Não há, contudo, rota dedicada à verificação de saúde: a forma mais simples de saber se a API está operando é uma requisição sem *token* a uma rota protegida, como `GET /me`, cuja resposta 401 indica que o processo, o roteamento e a verificação de autenticação estão funcionando, embora não comprove o acesso ao banco.

Os segredos e parâmetros da API são mantidos no arquivo `.env`, fora do controle de versão e redigido diretamente na máquina virtual, por acesso remoto. Não há cofre de segredos nem cópia desse arquivo em outro lugar. A consequência é que a máquina guarda a única cópia existente da configuração de produção: perdê-la não implicaria perda de dados, que estão no Supabase e no R2, mas exigiria reconstituir as credenciais de cada serviço. É uma limitação registrada, de correção simples, e não uma decisão de projeto.

8.3 Procedimento de Implantação

A implantação envolve duas atividades distintas: o provisionamento inicial do ambiente, feito uma única vez, e a publicação de novas versões, repetida a cada alteração do sistema.

8.3.1 Provisionamento Inicial

O ambiente foi preparado na seguinte ordem:

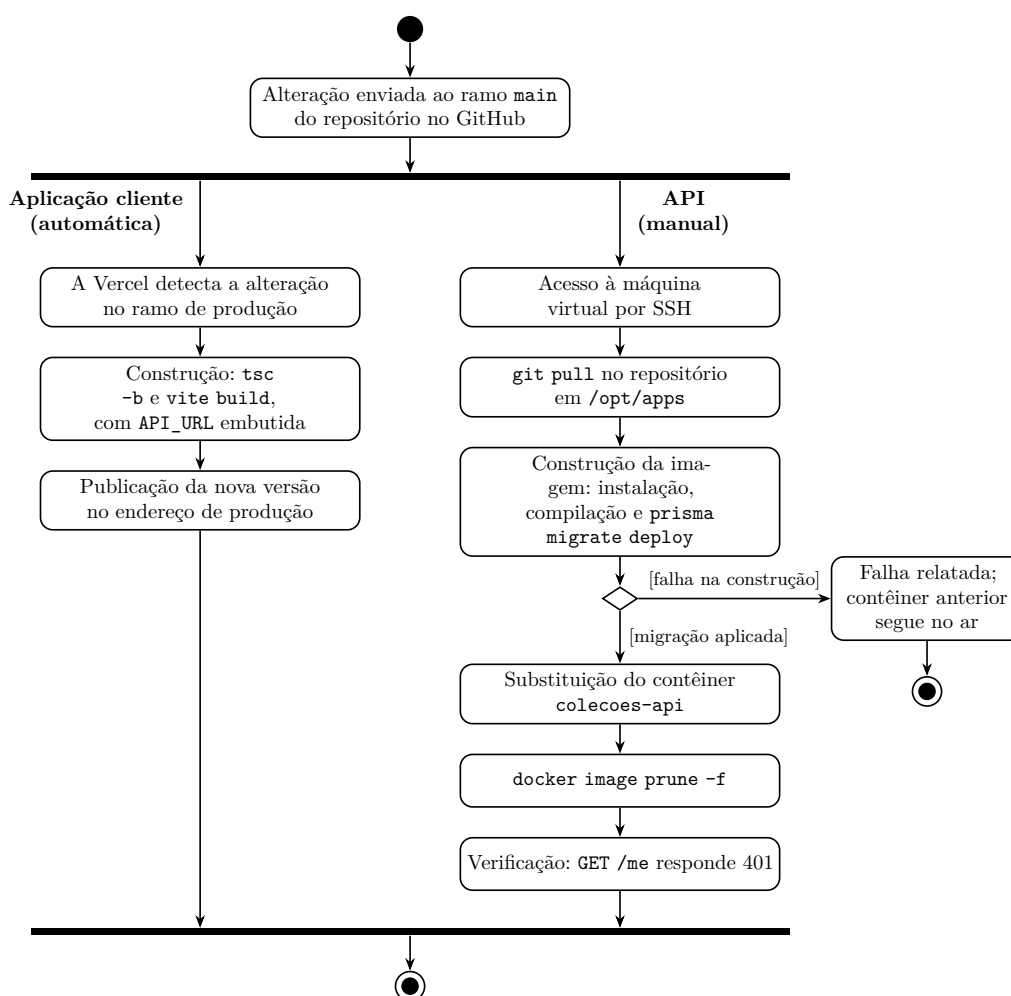
1. **Serviços de dados:** criação do projeto no Supabase, na região de São Paulo, e do balde no Cloudflare R2, com acesso público de leitura para a exibição das imagens; geração da senha de aplicativo da conta Google usada no envio de e-mails.
2. **Máquina virtual:** criação da instância na Oracle Cloud, com acesso por SSH autenticado por chave; liberação das portas 80 e 443 nas regras de entrada da rede virtual; atualização dos pacotes do sistema e ativação do UFW, permitindo apenas as portas 22, 80 e 443.
3. **Área de troca:** criação de um arquivo de *swap*, necessário para que a construção da imagem caiba na memória da máquina.
4. **Docker e Caddy:** instalação do Docker pelo *script* oficial do projeto e do Caddy pelo seu repositório de pacotes, habilitado como serviço do **systemd**.
5. **Nome da API:** cadastro do subdomínio no Duck DNS, apontando para o endereço público da máquina.
6. **Código:** geração de uma chave SSH na máquina, cadastrada como chave de implantação (*deploy key*) no repositório do *backend* no GitHub, e clonagem do repositório no diretório `/opt/apps`.
7. **Configuração:** redação do `.env` a partir do modelo `.env.example` e do `Caddyfile` com o nome da API.
8. **Primeira execução:** construção e início do contêiner com `docker compose up -d --build`, que aplica todas as migrações ao banco vazio, e recarga do Caddy, que obtém o certificado.
9. **Aplicação cliente:** importação do repositório do *frontend* na Vercel e definição da variável `API_URL` com o endereço HTTPS da API, antes da primeira construção.

8.3.2 Publicação de Novas Versões

A publicação de uma nova versão segue caminhos diferentes para as duas partes do sistema, como mostra o diagrama de atividades da Figura 20. A aplicação

cliente é publicada automaticamente: a cada alteração no ramo principal do repositório, a Vercel constrói o projeto e cria uma nova publicação de produção (Vercel, a). A API é publicada manualmente: o responsável acessa a máquina por SSH e executa um *script* de implantação, que obtém a versão mais recente do repositório (`git pull`), reconstrói a imagem e substitui o contêiner (`docker compose up -d --build`) e, por fim, remove as imagens que deixaram de ser usadas (`docker image prune -f`).

Figura 20 – Diagrama de atividades da publicação de uma nova versão



Fonte: Elaborada pelo autor

A diferença de tratamento entre as duas partes decorre do que cada uma exige. A publicação da aplicação cliente não toca em nenhum recurso da máquina

virtual nem em dados, e a plataforma que a recebe já oferece a construção automática; automatizá-la não teve custo. A publicação da API, ao contrário, reconstrói a imagem na mesma máquina que atende à produção e aplica migrações ao banco, e foi mantida manual para que cada publicação seja acompanhada por quem a disparou. A automação por uma esteira de integração contínua, que se conectaria à máquina e executaria o mesmo *script*, é um acréscimo possível que não altera nenhum dos passos descritos.

Duas consequências da publicação da API precisam ser registradas. A primeira é a degradação durante a construção: instalar dependências e compilar o código em uma máquina de 1 GB, com parte da memória em disco, disputa recursos com o contêiner em execução, e as respostas da API tornam-se mais lentas enquanto a construção ocorre. A segunda é a indisponibilidade breve no momento da substituição do contêiner, durante a qual o Caddy não encontra o processo na porta local e a aplicação cliente recebe erro nas requisições. Para a escala de uso do trabalho, ambas são aceitáveis; deixariam de ser em uma operação com compromisso formal de disponibilidade.

8.3.3 Reversão

A reversão também segue caminhos distintos. Na aplicação cliente, a Vercel permite devolver a produção a uma publicação anterior sem nova construção, recurso que, no plano Hobby, alcança a publicação imediatamente anterior (Vercel, b). Na API, a reversão percorre o mesmo caminho da publicação: o código é revertido no repositório e o *script* de implantação é executado novamente. Como o último passo da publicação remove as imagens que deixaram de ser usadas, não há artefato preservado da versão substituída, e o tempo de reversão é igual ao de uma publicação completa, e não menor. Uma migração já aplicada, por sua vez, não é desfeita pela reversão do código; por isso, a compatibilidade entre cada migração e a versão anterior do código, discutida na Seção 8.2, é também o que torna a reversão segura.

As decisões descritas neste capítulo (a máquina gratuita de 1 GB, a construção da imagem no próprio ambiente de produção, a publicação manual da API, o banco sem cópia de segurança automática e a configuração mantida apenas na

máquina) têm a mesma natureza: trocam garantias operacionais por custo zero, em um contexto em que o objetivo da operação é validar o sistema, e não sustentar uma exploração comercial. Nenhuma delas decorre da arquitetura descrita nos Capítulos 5 e 6, e todas podem ser revistas por acréscimo, sem alteração do código da aplicação.

9 RESULTADOS E DISCUSSÃO

REFERÊNCIAS BIBLIOGRÁFICAS

Caddy Web Server. **Automatic HTTPS**. Documentação do Caddy. Disponível em: <<https://caddyserver.com/docs/automatic-https>>. Acesso em: 02 out. 2026. Citado 2 vezes nas páginas 194 e 199.

Caddy Web Server. **reverse_proxy (Caddyfile directive)**. Documentação do Caddy. Disponível em: <https://caddyserver.com/docs/caddyfile/directives/reverse_proxy>. Acesso em: 02 out. 2026. Citado na página 199.

Docker. **Build secrets**. Documentação do Docker. Disponível em: <<https://docs.docker.com/build/building/secrets/>>. Acesso em: 05 out. 2026. Citado na página 197.

Docker. **Packet filtering and firewalls**. Documentação do Docker. Disponível em: <<https://docs.docker.com/engine/network/packet-filtering-firewalls/>>. Acesso em: 05 out. 2026. Citado na página 198.

Duck DNS. **Duck DNS: about**. Duck DNS. Disponível em: <<https://www.duckdns.org/about.jsp>>. Acesso em: 05 out. 2026. Citado na página 194.

FIELDING, R. T. **Architectural Styles and the Design of Network-based Software Architectures**. Tese (Doutorado) — University of California, Irvine, 2000. Disponível em: <<https://ics.uci.edu/~fielding/pubs/dissertation/top.htm>>. Citado 2 vezes nas páginas 15 e 22.

FONTELLES, M. J. et al. Metodologia da pesquisa científica: Diretrizes para a elaboração de um protocolo de pesquisa. **Revista Paraense de Medicina**, v. 23, n. 3, p. 1–8, 2009. Disponível em: <<http://files.bvs.br/upload/S/0101-5907/2009/v23n3/a1967.pdf>>. Citado na página 14.

FOWLER, M. **Patterns of Enterprise Application Architecture**. Boston: Addison-Wesley, 2002. Citado 3 vezes nas páginas 15, 17 e 22.

GERHARDT, T. E.; SILVEIRA, D. T. **Métodos de Pesquisa**. Porto Alegre: Editora da UFRGS, 2009. Citado na página 15.

LARMAN, C. **Utilizando UML e Padrões: uma introdução à análise e ao projeto orientados a objetos e ao desenvolvimento iterativo**. 3. ed. Porto Alegre: Bookman, 2007. Citado 4 vezes nas páginas 15, 22, 31 e 142.

MARATHE, S.; SUNDAR, S. S. What drives customization? control or identity? In: **Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '11)**. Vancouver: ACM, 2011. p. 781–790. Citado na página 10.

MARQUES, R. **Melhores práticas na elaboração de casos de teste**. 2018. Blog Cedro Technologies. Disponível em: <<https://www.cedrotech.com/blog/melhores-praticas-na-elaboracao-de-casos-de-teste/>>. Acesso em: 13 set. 2026. Citado na página 182.

MDN Web Docs. **Mixed content**. Mozilla Developer Network. Disponível em: <https://developer.mozilla.org/en-US/docs/Web/Security/Mixed_content>. Acesso em: 05 out. 2026. Citado na página 194.

Oracle. **Always Free Resources**. Documentação da Oracle Cloud Infrastructure. Disponível em: <https://docs.oracle.com/en-us/iaas/Content/FreeTier/freetier_topic-Always_Free_Resources.htm>. Acesso em: 05 out. 2026. Citado na página 193.

PRESSMAN, R. S.; MAXIM, B. R. **Engenharia de software: uma abordagem profissional**. 8. ed. Porto Alegre: AMGH, 2016. v. 8. ISBN 9788580555332. Citado 3 vezes nas páginas 15, 28 e 61.

SOMMERVILLE, I. **Engenharia de Software**. 9. ed. São Paulo: Pearson Prentice Hall, 2011. Tradução: Ivan Bosnic e Kalinka G. de O. Gonçalves. ISBN 9788579361081. Citado 6 vezes nas páginas 15, 21, 28, 33, 34 e 61.

Supabase. **About billing on Supabase**. Documentação do Supabase. Disponível em: <<https://supabase.com/docs/guides/platform/billing-on-supabase>>. Acesso em: 05 out. 2026. Citado na página 193.

Supabase. **Connect to your database**. Documentação do Supabase. Disponível em: <<https://supabase.com/docs/guides/database/connecting-to-postgres>>. Acesso em: 05 out. 2026. Citado na página 193.

Supabase. **Database Backups**. Documentação do Supabase. Disponível em: <<https://supabase.com/docs/guides/platform/backups>>. Acesso em: 05 out. 2026. Citado na página 194.

Supabase. **Project Pausing**. Documentação do Supabase. Disponível em: <<https://supabase.com/docs/guides/platform/free-project-pausing>>. Acesso em: 05 out. 2026. Citado na página 194.

VALENTE, M. T. **Engenharia de Software Moderna: Princípios e Práticas para Desenvolvimento de Software com Produtividade**. Belo Horizonte: Independente, 2020. ISBN 978-6500019504. Citado 2 vezes nas páginas 28 e 74.

Vercel. **Deploying Git Repositories with Vercel**. Documentação da Vercel. Disponível em: <<https://vercel.com/docs/git>>. Acesso em: 05 out. 2026. Citado 2 vezes nas páginas 192 e 202.

Vercel. **Performing an Instant Rollback on a Deployment**. Documentação da Vercel. Disponível em: <<https://vercel.com/docs/instant-rollback>>. Acesso em: 05 out. 2026. Citado na página 203.

Vercel. **Vite on Vercel**. Documentação da Vercel. Disponível em: <<https://vercel.com/docs/frameworks/frontend/vite>>. Acesso em: 02 out. 2026. Citado na página 193.

WATKINS, R. D.; SELLEN, A.; LINDLEY, S. E. Digital collections and digital collecting practices. In: **Proceedings of the 33rd Annual ACM Conference on Human Factors in Computing Systems (CHI '15)**. Seoul: ACM, 2015. p. 3423–3432. Citado na página 9.